

W++ Compiler Pro: Advanced Development Platform



Group Members:

Dur E Adan Sultan (22-CS-88)

Muhammad Yasir (22-CS-66)

Instructor Name:

Dr Rao Wakeel Ahmad

Course Name:

Compiler Construction

Semester:

Spring 2025 (6th Semester Complete)

**Department of Computer Science University of
Engineering and Technology, Taxila**

Introduction

Purpose

The primary purpose of this project is to provide an all-in-one, user-friendly Integrated Development Environment (IDE) for the W++ programming language a language inspired by C and C++. This IDE is designed to make the process of writing, analyzing, and debugging code as smooth and intuitive as possible. By integrating advanced features such as real-time error detection, syntax highlighting, and code analysis, the IDE aims to reduce the learning curve for new programmers and increase productivity for experienced users. Ultimately, the project seeks to foster a deeper understanding of programming concepts by making the structure and behavior of code more transparent and accessible.

Objectives

The main objectives of the W++ IDE project are as follows:

- **Simplify Code Writing:** Provide a modern, intuitive interface that allows users to write code efficiently, with features like auto-completion and typo correction.
- **Automate Error Detection:** Instantly identify and highlight both lexical and syntax errors, helping users correct mistakes before running their code.
- **Enhance Code Understanding:** Offer detailed token and symbol tables, so users can see how their code is broken down and interpreted by the computer.
- **Support Learning and Teaching:** Serve as an educational tool for students and teachers, making it easier to demonstrate programming concepts and debug code in real time.
- **Increase Productivity:** Enable quick testing, analysis, and correction of code, reducing the time spent on manual debugging and error tracking.

Target Users

This IDE is designed for a broad range of users, including:

- **Students:** Those who are learning programming, especially in C/C++-like languages, will benefit from the instant feedback and educational features.
- **Educators:** Teachers and instructors can use the IDE to demonstrate code structure, error types, and debugging techniques in a classroom setting.
- **Beginner Programmers:** Individuals new to coding will find the interface approachable and the error messages helpful for learning.
- **Anyone Interested in Programming:** The IDE is accessible to anyone who wants to experiment with code, regardless of their background.

Brief Description of the Technologies Used

The W++ IDE is built using the Java programming language, leveraging the robust and cross-platform capabilities of Java to ensure compatibility on various operating systems. The graphical user interface (GUI) is developed with Java Swing, which provides a responsive and visually appealing environment for code editing and analysis. The backend of the IDE features a custom lexer and parser, which are responsible for breaking down the code into tokens, building symbol tables, and detecting both lexical and syntax errors. Standard Java libraries are used for file handling, regular expressions, and data management, ensuring reliability and performance. The IDE also incorporates features such as syntax highlighting, auto-correction, and code completion, all implemented using efficient Java algorithms and data structures.

Core Functionalities

The W++ IDE is packed with a range of powerful features designed to make programming, learning, and debugging in the W++ language as smooth and effective as possible. Below, each core function is described in detail, with a focus on its role, operation, and value to the user.

1. Multi-Tabbed Code Editor

Explanation:

The IDE provides a modern, multi-tabbed code editor that allows users to work on multiple files simultaneously. Each tab represents a separate file, making it easy to switch between different pieces of code or projects without losing progress. The editor supports standard text editing features such as cut, copy, paste, undo, and redo, as well as more advanced capabilities like line numbering and customizable font sizes.

Purpose:

This feature enhances productivity by allowing users to organize their work efficiently, compare code side-by-side, and manage multiple coding tasks within a single application window.

2. Syntax Highlighting

Explanation:

As users type or open code in the editor, the IDE automatically applies color-coding to different elements of the W++ language, such as keywords, data types, operators, literals, comments, and identifiers. This is achieved through a custom-built syntax highlighter that analyzes the code in real time and updates the display accordingly.

Purpose:

Syntax highlighting improves code readability, helps users quickly spot errors or inconsistencies, and makes it

3. Automatic Typo Correction

Explanation:

The IDE includes an intelligent typo correction system that detects and automatically fixes common spelling mistakes in keywords and data types as the user types. For example, if a user types “flaot” instead of “float,” the IDE will instantly correct it to the proper keyword.

Purpose:

This feature reduces frustration and saves time by preventing simple mistakes from causing errors later in the coding process. It is especially helpful for new programmers who may not be familiar with all the correct spellings

4. Code Auto-Completion

Explanation:

The editor offers code auto-completion suggestions for keywords, control structures, and common code patterns. As the user types, a popup menu appears with possible completions, which can be selected to quickly insert the full keyword or code snippet (such as an “if-else” block or a “for” loop template).

Purpose:

Auto-completion speeds up coding, reduces the likelihood of syntax errors, and helps users learn the correct structure of W++ statements by example.

5. Tokenization (Lexical Analysis)

Explanation:

When the user clicks the “Tokenize” button, the IDE processes the code using a custom lexer. This lexer breaks the code into tokens—such as keywords, identifiers, operators, literals, and separators—and displays a detailed token table. The table includes the type, value, and count of each token, along with a description.

Purpose:

Tokenization helps users understand how their code is interpreted by the computer, making the process of learning programming concepts more transparent. It also serves as the first step in error detection and code analysis.

6. Symbol Table Generation

Explanation:

Alongside tokenization, the IDE constructs a symbol table that lists all identifiers (such as variable names) found in the code. For each identifier, the table shows its type, size, line of declaration, line of usage, and memory address.

Purpose:

The symbol table is a crucial tool for understanding variable scope, data types, and memory management. It helps

7. Real-Time Error Detection (Lexical and Syntax Errors)

Explanation:

The IDE continuously analyzes the code for both lexical errors (such as invalid tokens or unterminated strings) and syntax errors (such as missing semicolons, unmatched braces, or undeclared variables). Errors are highlighted in the editor and listed in a dedicated error panel, with clear messages indicating the line and nature of each problem.

Purpose:

Immediate feedback on errors allows users to correct mistakes as they code, reducing the time spent debugging and increasing confidence in their work. The detailed error messages also serve as a learning tool, explaining why certain constructs are invalid.

8. Parse (Syntax Analysis) Functionality

Explanation:

After tokenization, users can click the “Parse” button to perform a deeper syntax analysis of their code. The parser checks for correct program structure, valid control flow, and proper use of language constructs. It flags issues such as multiple main functions, invalid variable declarations, and incorrect loop or conditional syntax.

Purpose:

Parsing ensures that the code not only contains valid tokens but also follows the rules of the W++ language. This step is essential for preparing code for compilation or further analysis, and helps users develop good programming habits.

9. Output Simulation

Explanation: The IDE can simulate the output of code by extracting and displaying the results of print statements (such as “cout” or “printf”) directly in a console tab. This allows users to see what their code would output without needing to compile and run it in a separate environment.

Purpose:

Output simulation provides instant gratification and feedback, making it easier to test and debug code. It is particularly useful for beginners who want to see the effects of their code changes immediately.

10. File Management (Open, Save, Rename, Delete)**Explanation:**

Users can create new files, open existing ones, save their work, rename files, and delete files directly from the IDE. The interface ensures that file operations are straightforward and that users are prompted to save changes as needed.

Purpose:

Integrated file management streamlines the workflow, allowing users to focus on coding rather than switching between different applications or worrying about losing their work.

11. Export to CSV**Explanation:**

The IDE allows users to export the contents of the token table, symbol table, or error list to CSV files. This makes it easy to share analysis results, include them in reports, or further process them in spreadsheet software.

Purpose:

Exporting data supports collaboration, documentation, and deeper analysis outside the IDE, making the tool more versatile for both educational and professional use.

12. Theme Customization (Light/Dark Mode)**Explanation:**

Users can switch between light and dark themes to suit their preferences and reduce eye strain. The entire interface, including the editor, output panels, and menus, adapts to the selected theme.

Purpose:

Theme customization enhances user comfort and accessibility, making the IDE pleasant to use for extended periods.

13. Session Logging and Console Output**Explanation:**

The IDE maintains a log of user actions and system messages in a dedicated console tab. This includes information about file operations, error corrections, and analysis results.

Purpose:

Session logging helps users keep track of their workflow, review past actions, and diagnose issues more effectively.

Conclusion

Each of these core functionalities is carefully designed to support users at every stage of the programming process, from writing and understanding code to detecting and correcting errors. Together, they make the W++ IDE a comprehensive, professional, and accessible tool for anyone interested in learning or working with the W++ language.

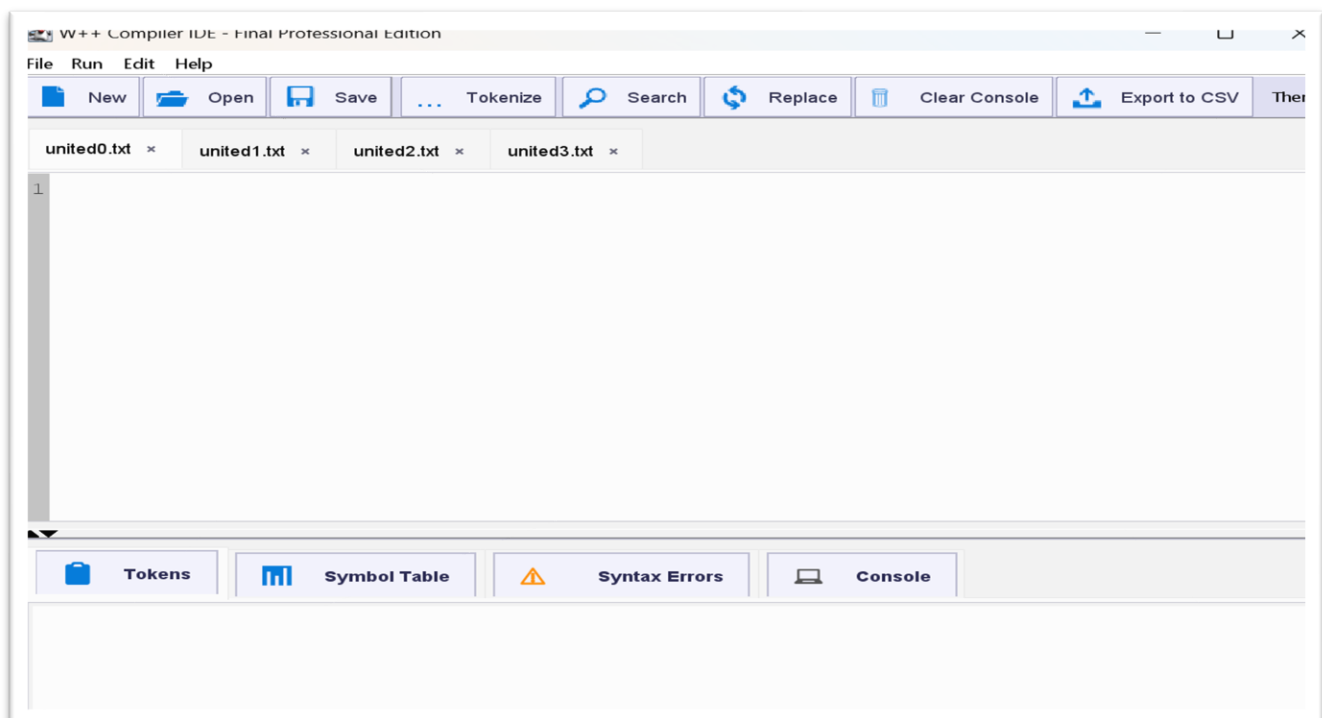
Working Screenshots of Major Functionalities

1. Multi-Tabbed Code Editor

Description:

The IDE allows users to work on multiple files at once, each in its own tab. This makes it easy to switch between different code files and manage several projects simultaneously.

```
private void createNewTab() {
    JTextPane editor = new JTextPane();
    Font font = new Font(Font.MONOSPACED, Font.PLAIN, fontSize);
    editor.setFont(font);
    editor.setBorder(BorderFactory.createEmptyBorder(top:5, left:5, bottom:5, right:5));
    // Add auto-correction functionality
    setupAutoCorrection(editor);
    // Add auto-completion functionality
    setupAutoCompletion(editor);
    JScrollPane scrollPane = new JScrollPane(editor);
    scrollPane.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(new Color(r:200, g:200, b:210), thickness:1),
        BorderFactory.createEmptyBorder(top:0, left:0, bottom:0, right:0)
    ));
    LineNumberView lineNumberView = new LineNumberView(editor);
    scrollPane.setRowHeaderView(lineNumberView);
    int tabIndex = tabbedPane.getTabCount();
    // Generate sequential file name: united0.txt, united1.txt, etc.
    String fileName = "united" + tabIndex + ".txt";
    tabbedPane.addTab(fileName, scrollPane);
    tabbedPane.setSelectedIndex(tabIndex);
    // Create a tab component with close button
    JPanel tabPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, hgap:0, vgap:0));
    tabPanel.setOpaque(isOpaque:false);
    JLabel tabLabel = new JLabel(fileName);
    tabLabel.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, size:12));
    // Create a nicer looking close button
    JButton closeButton = new JButton(text:"✕");
```

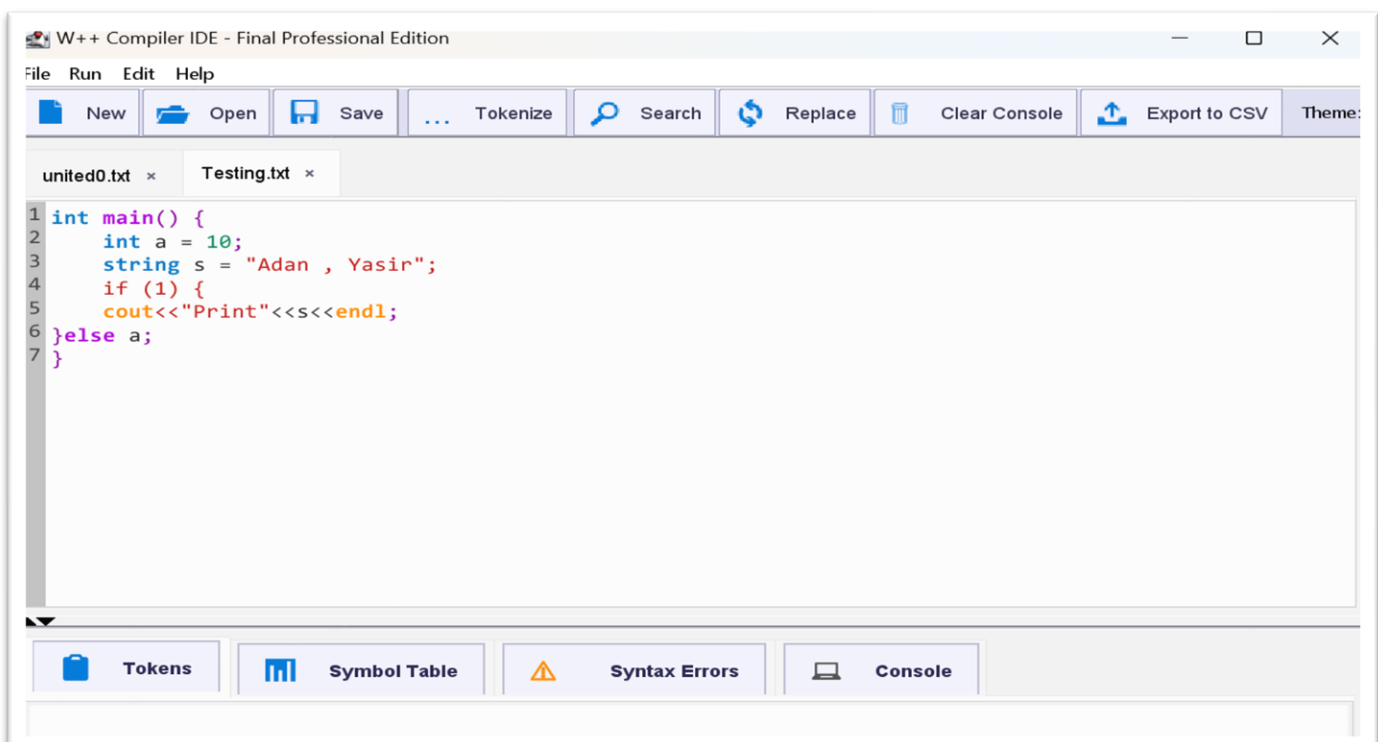


2. Syntax Highlighting

Description:

As you type, the IDE automatically colors keywords, data types, operators, comments, and other code elements. This makes the code easier to read and helps users quickly spot errors.

```
private void applySyntaxHighlighting(JTextPane editor) {
    try {
        isProgrammaticChange = true;
        String text = editor.getText();
        StyledDocument doc = editor.getStyledDocument();
        // Reset foreground and background
        SimpleAttributeSet defaultStyle = new SimpleAttributeSet();
        StyleConstants.setForeground(defaultStyle, isDarkMode ? DARK_FG : LIGHT_FG);
        StyleConstants.setBackground(defaultStyle, isDarkMode ? DARK_BG : LIGHT_BG);
        doc.setCharacterAttributes(offset:0, text.length(), defaultStyle, replace:true);
        // First highlight comments to prevent them from being tokenized
        highlightComments(text, doc);
        // Then highlight preprocessor directives
        highlightPreprocessor(text, doc);
        // Now tokenize and highlight other elements
        java.util.List<Lexer.TokenInfo> tokens = new ArrayList<>();
        java.util.List<Lexer.LexicalError> lexicalErrors = new ArrayList<>();
        Lexer.tokenizeWithErrors(text, tokens, lexicalErrors);
        // Perform syntax analysis
        java.util.List<Lexer.SyntaxError> syntaxErrors = new ArrayList<>();
        Lexer.analyzeSyntax(tokens, syntaxErrors);
        // Highlight tokens
        List<String> tokenStrings = new ArrayList<>();
        for (Lexer.TokenInfo ti : tokens) tokenStrings.add(ti.token);
        highlightTokens(text, tokenStrings, doc);
        // Highlight lexical errors with red underline
        for (Lexer.LexicalError error : lexicalErrors) {
            highlightErrorToken(doc, text, error.token, error.line, error.column, ERROR_COLOR);
        }
    }
}
```



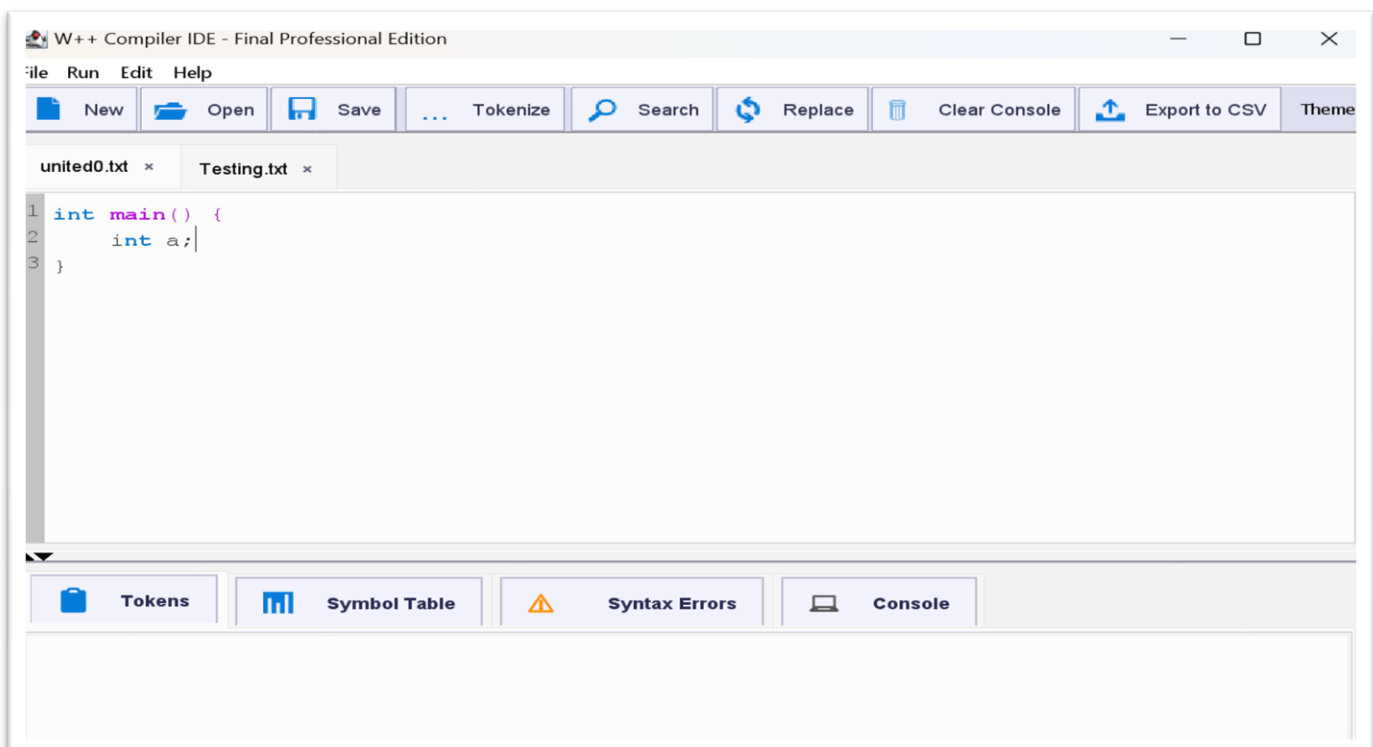
3. Automatic Typo Correction

Description:

The IDE detects and corrects common spelling mistakes in keywords and data types as you type, helping you avoid simple errors and speeding up the coding process.

```
// Add auto-correction functionality
private void setupAutoCorrection(JTextPane editor) {
    editor.getDocument().addDocumentListener(new DocumentListener() {
        private boolean isCorrecting = false;

        @Override
        public void insertUpdate(DocumentEvent e) {
            if (!isCorrecting) {
                try {
                    String text = editor.getText();
                    int caretPos = editor.getCaretPosition();
                    if (caretPos == 0) return;
                    char lastChar = text.charAt(caretPos - 1);
                    // Define separators that should trigger correction
                    boolean isSeparator = !Character.isLetterOrDigit(lastChar);
                    int wordEnd = caretPos;
                    int wordStart = wordEnd;
                    // Always look back for the previous word
                    while (wordStart > 0 && Character.isLetterOrDigit(text.charAt(wordStart - 1)))
                        wordStart--;
                    if (wordStart < wordEnd) {
                        String word = text.substring(wordStart, wordEnd);
                        String correction = TYPO_CORRECTIONS.get(word.toLowerCase());
                        if (correction != null && !correction.equals(word)) {
                            isCorrecting = true;
                            editor.getDocument().remove(wordStart, word.length());
                            editor.getDocument().insertString(wordStart, correction, a:null);
                            // Move caret after the separator if one was typed
                        }
                    }
                } catch (Exception ex) {
                    // Handle exception
                }
            }
        }
    });
}
```



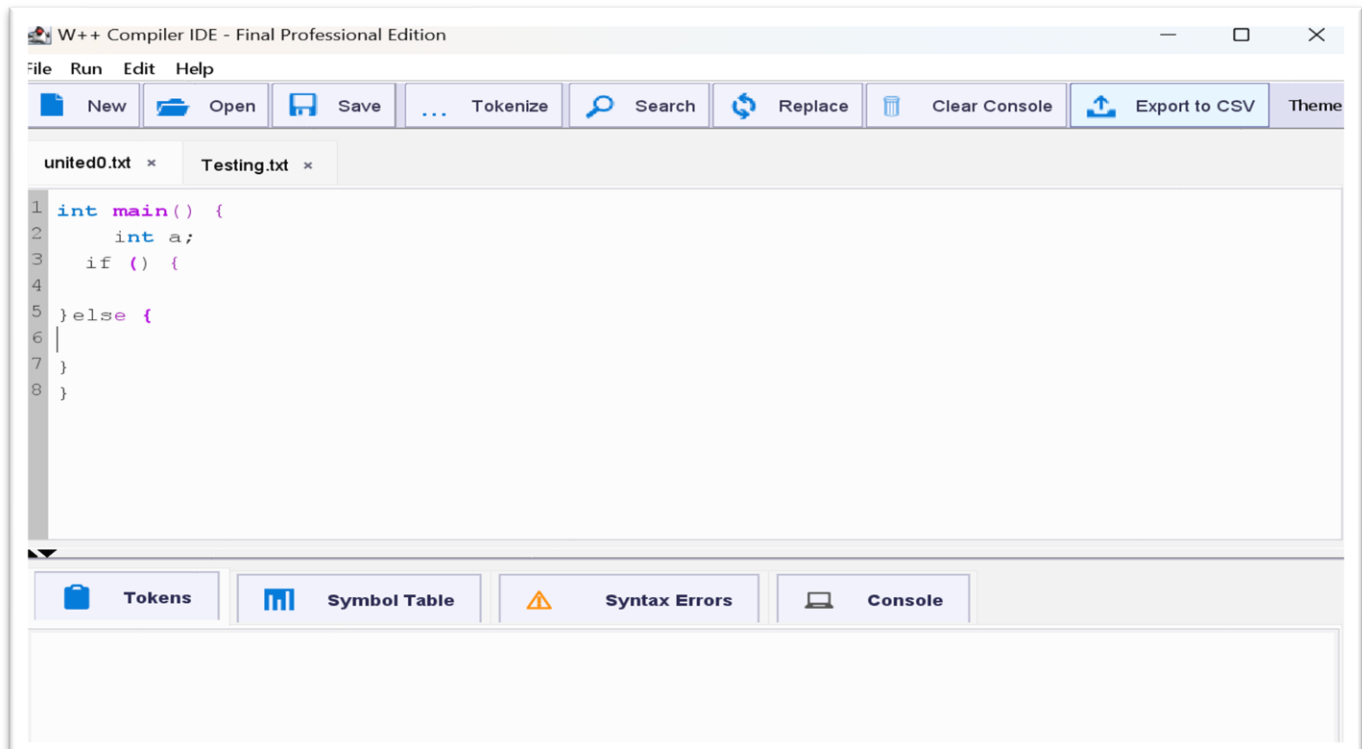
4. Code Auto-Completion

Description:

The editor suggests keywords and code snippets as you type. You can quickly insert common structures like “if-else” or “for” loops, reducing typing and errors.

```
// Auto-complete popup for keywords
private void setupAutoCompletion(JTextPane editor) {
    JPopupMenu suggestionPopup = new JPopupMenu();
    JList<String> suggestionList = new JList<>();
    suggestionList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
    suggestionList.setFont(editor.getFont());
    JScrollPane scrollPane = new JScrollPane(suggestionList);
    scrollPane.setBorder(border:null);
    suggestionPopup.setBorder(BorderFactory.createLineBorder(new Color(r:180,g:180,b:200)));
    suggestionPopup.add(scrollPane);
    suggestionPopup.setFocusable(focusable:false);
    suggestionList.setFocusable(focusable:false);

    // Helper to show suggestions
    Runnable showSuggestions = () -> {
        try {
            int caretPos = editor.getCaretPosition();
            String text = editor.getText(offsets:0, caretPos);
            int wordStart = caretPos;
            while (wordStart > 0 && Character.isLetterOrDigit(text.charAt(wordStart - 1))) {
                wordStart--;
            }
            String prefix = text.substring(wordStart, caretPos);
            if (prefix.length() == 0) {
                suggestionPopup.setVisible(b:false);
                return;
            }
            java.util.List<String> matches = new ArrayList<>();
            for (String kw : KNOWN_WORDS) {
```



5. Tokenization (Lexical Analysis)

Description:

Clicking the “Tokenize” button analyzes your code and displays a table of all tokens, including their type, value, and count. This helps you understand how your code is interpreted.

```
private void runLexer() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText(text:"No tab selected");
        return;
    }
    JTextPane editor = editors.get(selectedTab);
    String code = editor.getText();
    java.util.List<Lexer.TokenInfo> tokens = new ArrayList<>();
    java.util.List<Lexer.LexicalError> lexicalErrors = new ArrayList<>();
    Lexer.tokenizeWithErrors(code, tokens, lexicalErrors);
    Map<String, String[]> symbolTable = new HashMap<>();
    Lexer.processTokensForSymbolTable(tokens, symbolTable);
    Map<Lexer.TokenType, Map<String, Integer>> tokenCounts = new HashMap<>();
    for (Lexer.TokenInfo ti : tokens) {
        String token = ti.token;
        Lexer.TokenType type = Lexer.getTokenType(token);
        tokenCounts.computeIfAbsent(type, k -> new HashMap<>()).merge(token, value:1, Integer::sum);
    }
    // Check for syntax errors
    java.util.List<Lexer.SyntaxError> syntaxErrors = new ArrayList<>();
    Lexer.analyzeSyntax(tokens, syntaxErrors);
    // Display token table
    isProgrammaticChange = true;
    StringBuilder tokenOutput = new StringBuilder();
    tokenOutput.append(str:"W++ LANGUAGE TOKEN ANALYSIS\n");
    tokenOutput.append(str:"=====");
    tokenOutput.append(String.format(format:"%-15s %-20s %-10s %s\n", ...args:"TOKEN TYPE", "LEXEME", "COUNT", "DESCRIPTION"));
}
```

W++ Compiler IDE - Final Professional Edition

File Run Edit Help

New Open Save ... Tokenize Parse Search Replace Clear Console Export t

united0.txt x Testing.txt x

```
1 int main() {
2     int a;
3     if () {
4
5 }else {
6
7 }
8 }
```

Tokens Symbol Table Syntax Errors Console

W++ LANGUAGE TOKEN ANALYSIS

TOKEN TYPE	LEXEME	COUNT	DESCRIPTION
KEYWORD	else	1	Conditional
KEYWORD	main	1	Reserved word
KEYWORD	if	1	Conditional
KEYWORD	int	2	Data type
IDENTIFIER	a	1	User-defined identifier
SEPARATOR	(	2	Parenthesis
SEPARATOR	)	2	Parenthesis
SEPARATOR	,	1	Statement terminator

6. Symbol Table Generation

Description:

The IDE builds a symbol table listing all variables and identifiers, showing their type, size, line of declaration, and memory address.

```
// Display symbol table
StringBuilder symbolOutput = new StringBuilder();
symbolOutput.append(str:"W++ LANGUAGE SYMBOL TABLE\n");
symbolOutput.append(str:"=====");
symbolOutput.append(str:"Name          Type          Size    Dimension    Line of Declaration");
symbolOutput.append(str:"-----");
for (Map.Entry<String, String[]> entry : symbolTable.entrySet()) {
    String name = entry.getKey();
    String[] info = entry.getValue();
    String type = info[0];
    String value = info[1].equals(anObject:"Not initialized") ? "Uninitialized" : info[1];
    int size = Lexer.TYPE_SIZES.getDefault(type, defaultValue:0);
    String line = info[2];
    String lineOfUsage = line; // Placeholder, same as declaration
    String address = "0x" + Integer.toHexString(name.hashCode() & 0xFFFF);
    symbolOutput.append(String.format(format:"%-15s %-15s %-8d %-12d %-18s %-15s %-10s\n", name,
    }
symbolTableArea.setText(symbolOutput.toString());
statusLabel.setText(text:"Lexical analysis complete");
logToConsole("[INFO] Tokenization complete. " + tokens.size() + " tokens found.");
if (!lexicalErrors.isEmpty()) logToConsole("[WARNING] " + lexicalErrors.size() + " lexical error

// Update sessionTokens/sessionLexicalErrors/sessionSyntaxErrors:
sessionTokens = tokens.size();
sessionLexicalErrors = lexicalErrors.size();
isProgrammaticChange = false;
```

The screenshot shows the W++ Compiler IDE interface. The top menu bar includes File, Run, Edit, and Help. Below the menu is a toolbar with buttons for New, Open, Save, Tokenize, Parse, Search, Replace, Clear Console, and Export to. The main editor area displays a C++ code snippet in `Testing.txt`:

```
1 int main() {
2     int a;
3     if () {
4
5     } else {
6
7     }
8 }
```

At the bottom, there is a tabbed interface with four tabs: Tokens, Symbol Table, Syntax Errors, and Console. The **Symbol Table** tab is active, displaying the following table:

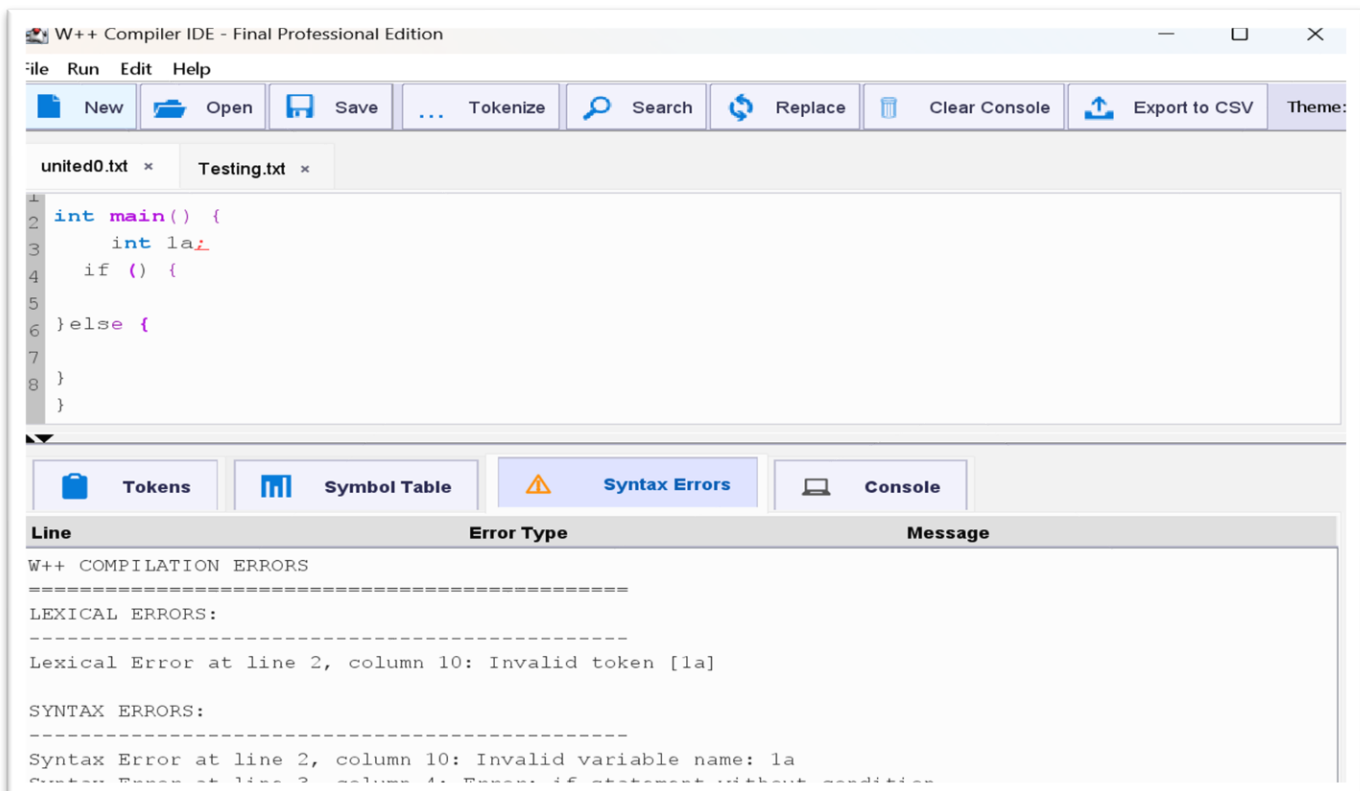
Name	Type	Size	Dimension	Line of Declaration	Line of Usage	Address
a	int	4	0	1	1	0x61

7. Real-Time Error Detection

Description:

The IDE highlights both lexical and syntax errors as you type, and lists them in a dedicated error panel with clear messages and line numbers.

```
private void applySyntaxHighlighting(JTextPane editor) {
    try {
        isProgrammaticChange = true;
        String text = editor.getText();
        StyledDocument doc = editor.getStyledDocument();
        // Reset foreground and background
        SimpleAttributeSet defaultStyle = new SimpleAttributeSet();
        StyleConstants.setForeground(defaultStyle, isDarkMode ? DARK_FG : LIGHT_FG);
        StyleConstants.setBackground(defaultStyle, isDarkMode ? DARK_BG : LIGHT_BG);
        doc.setCharacterAttributes(offset:0, text.length(), defaultStyle, replace:true);
        // First highlight comments to prevent them from being tokenized
        highlightComments(text, doc);
        // Then highlight preprocessor directives
        highlightPreprocessor(text, doc);
        // Now tokenize and highlight other elements
        java.util.List<Lexer.TokenInfo> tokens = new ArrayList<>();
        java.util.List<Lexer.LexicalError> lexicalErrors = new ArrayList<>();
        Lexer.tokenizeWithErrors(text, tokens, lexicalErrors);
        // Perform syntax analysis
        java.util.List<Lexer.SyntaxError> syntaxErrors = new ArrayList<>();
        Lexer.analyzeSyntax(tokens, syntaxErrors);
        // Highlight tokens
        List<String> tokenStrings = new ArrayList<>();
        for (Lexer.TokenInfo ti : tokens) tokenStrings.add(ti.token);
        highlightTokens(text, tokenStrings, doc);
        // Highlight lexical errors with red underline
        for (Lexer.LexicalError error : lexicalErrors) {
            highlightErrorToken(doc, text, error.token, error.line, error.column, ERROR_COLOR);
        }
    }
}
```



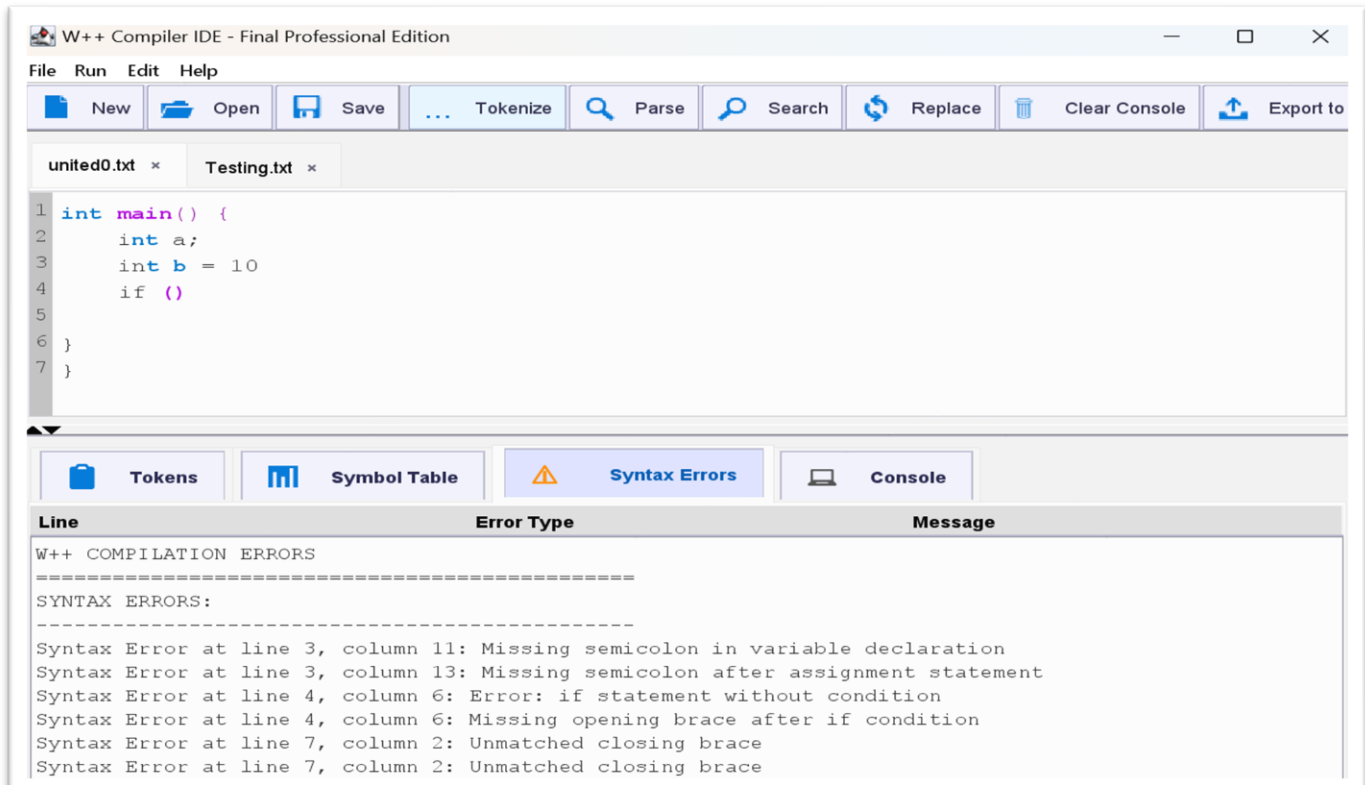
8. Parse (Syntax Analysis) Functionality

Description:

After tokenization, the “Parse” button performs a deeper analysis of your code’s structure, checking for correct syntax and flagging issues like missing semicolons or unmatched braces.

```
private void debugCode() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText(text:"No tab selected");
        return;
    }
    JTextPane editor = editors.get(selectedTab);
    String code = editor.getText();
    tokenTableArea.setText(t:"");
    symbolTableArea.setText(t:"");
    // If the code is empty, clear the error area and return
    if (code.trim().isEmpty()) {
        errorArea.setText(t:"");
        statusLabel.setText(text:"No code to analyze");
        return;
    }

    // Check for duplicate main functions
    int mainCount = 0;
    Pattern mainPattern = Pattern.compile(regex:"\\bmain\\s*\\(");
    Matcher mainMatcher = mainPattern.matcher(code);
    while (mainMatcher.find()) {
        mainCount++;
    }
    if (mainCount > 1) {
        errorArea.setText("W++ COMPILATION ERRORS\n" +
            "=====\n\n" +
            "SYNTAX ERRORS:\n" +
            "\n");
    }
}
```

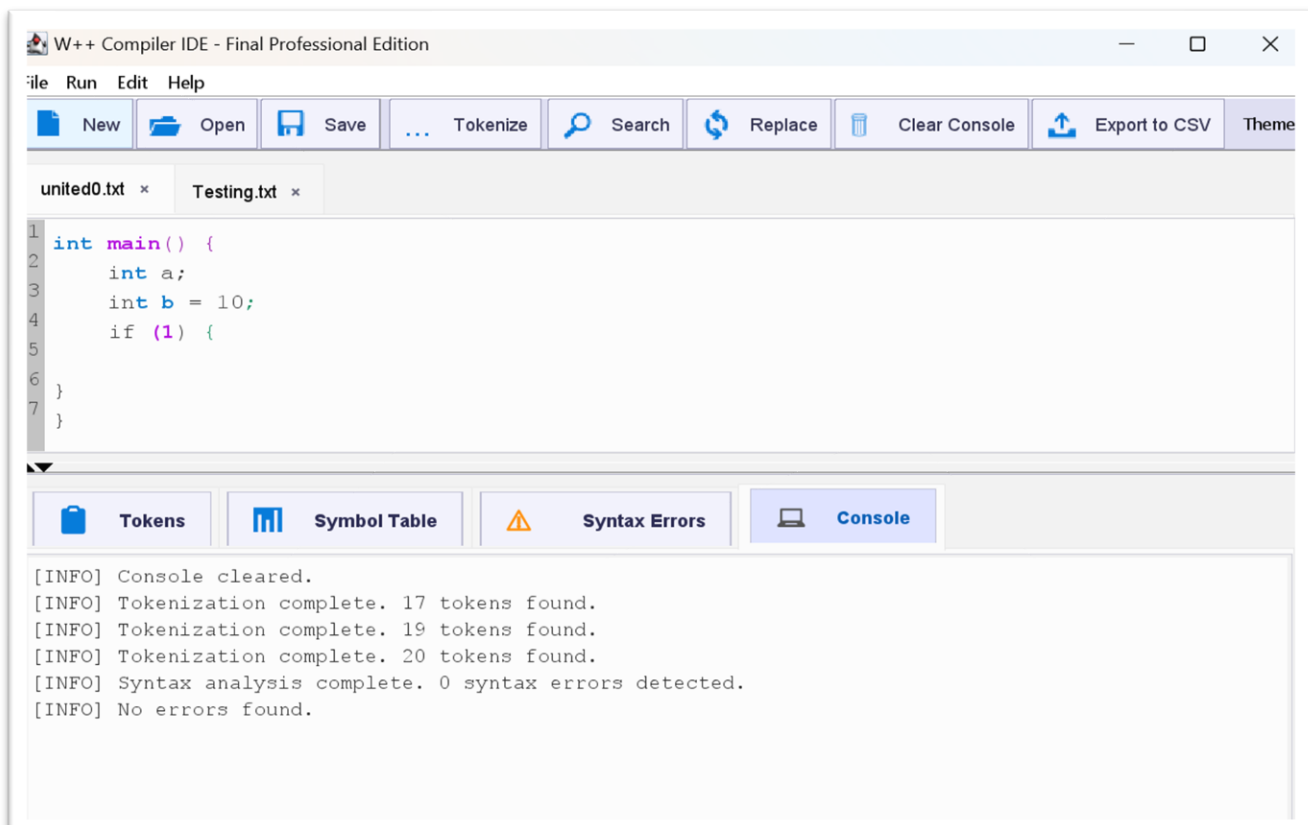


9. Output Simulation

Description:

The IDE simulates the output of your code by displaying the results of print statements in a console tab, so you can see what your code would output without compiling.

```
private void simulateOutput() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText(text:"No tab selected");
        return;
    }
    JTextPane editor = editors.get(selectedTab);
    String code = editor.getText();
    StringBuilder output = new StringBuilder();
    // Simple simulation: extract cout, printf, and print statements
    // C++ cout << "text";
    java.util.regex.Pattern coutPattern = java.util.regex.Pattern.compile(regex:"cout\\s*<<\\s");
    java.util.regex.Matcher coutMatcher = coutPattern.matcher(code);
    while (coutMatcher.find()) {
        output.append(coutMatcher.group(group:1)).append(str:"\n");
    }
    // C printf("text");
    java.util.regex.Pattern printfPattern = java.util.regex.Pattern.compile(regex:"printf\\s*\\(");
    java.util.regex.Matcher printfMatcher = printfPattern.matcher(code);
    while (printfMatcher.find()) {
        output.append(printfMatcher.group(group:1)).append(str:"\n");
    }
    // Python-Like print("text")
    java.util.regex.Pattern printPattern = java.util.regex.Pattern.compile(regex:"print\\s*\\(");
    java.util.regex.Matcher printMatcher = printPattern.matcher(code);
    while (printMatcher.find()) {
        output.append(printMatcher.group(group:1)).append(str:"\n");
    }
}
```

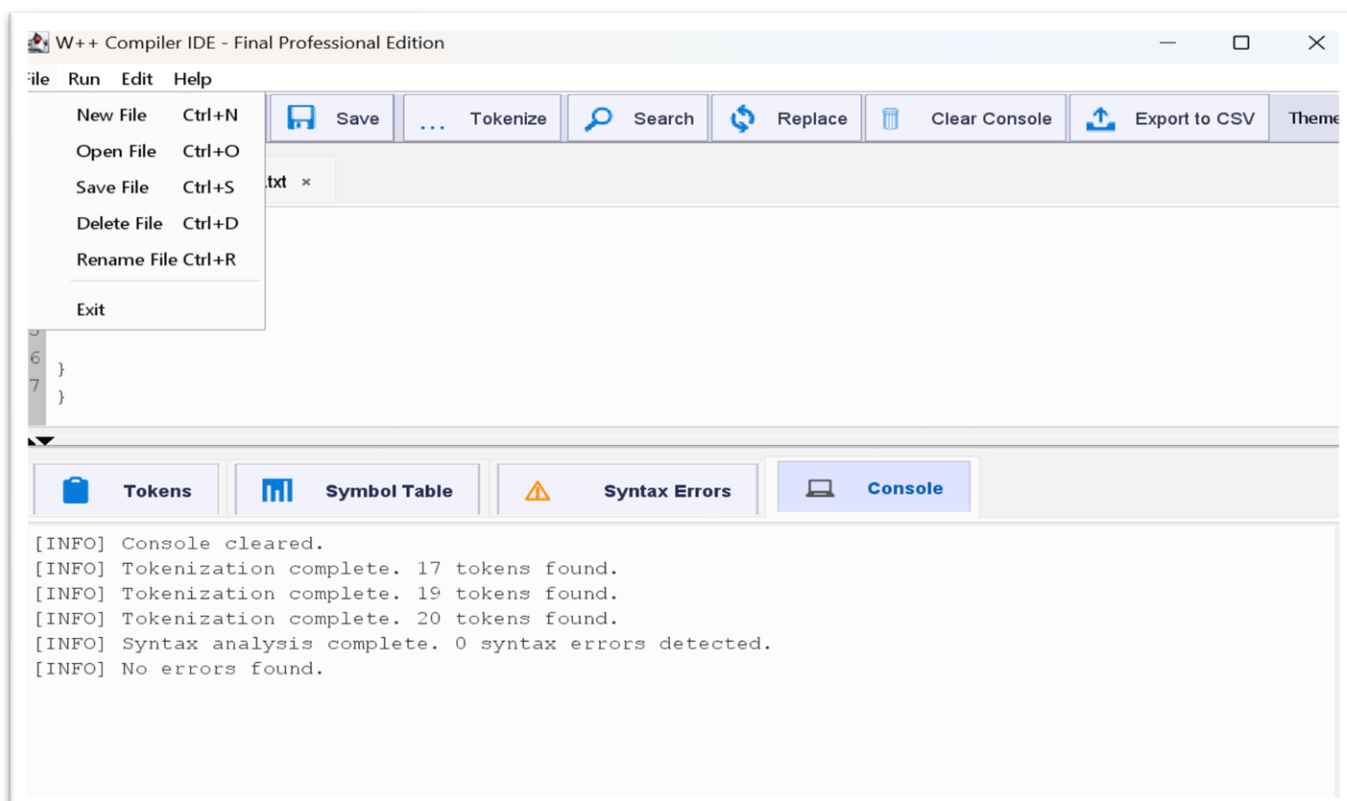


10. File Management (Open, Save, Rename, Delete)

Description:

You can easily create, open, save, rename, and delete files directly from the IDE, making file management simple and efficient.

```
private void openFile() {
    try (BufferedReader reader = new BufferedReader(new FileReader(fileChooser.getSelectedFile()));
        JTextPane editor = new JTextPane();
        Font font = new Font(name:"JetBrains Mono", Font.PLAIN, fontSize);
        if (font.getFamily().equals(anObject:"Dialog")) {
            font = new Font(name:"Consolas", Font.PLAIN, fontSize);
        }
        editor.setFont(font);
        StringBuilder content = new StringBuilder();
        String line;
        while ((line = reader.readLine()) != null) {
            content.append(line).append(str:"\n");
        }
        editor.setText(content.toString());
        // Add auto-correction and auto-completion
        setupAutoCorrection(editor);
        setupAutoCompletion(editor);
        JScrollPane scrollPane = new JScrollPane(editor);
        scrollPane.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(r:200, g:200, b:210), thickness:1),
            BorderFactory.createEmptyBorder(top:0, left:0, bottom:0, right:0)
        ));
        LineNumberView lineNumberView = new LineNumberView(editor);
        scrollPane.setRowHeaderView(lineNumberView);
        int tabIndex = tabbedPane.getTabCount();
        tabbedPane.addTab(fileChooser.getSelectedFile().getName(), scrollPane);
        tabbedPane.setTabComponentAt(tabIndex, createTabComponent(fileChooser.getSelectedFile()));
        editors.put(tabIndex, editor);
    }
}
```

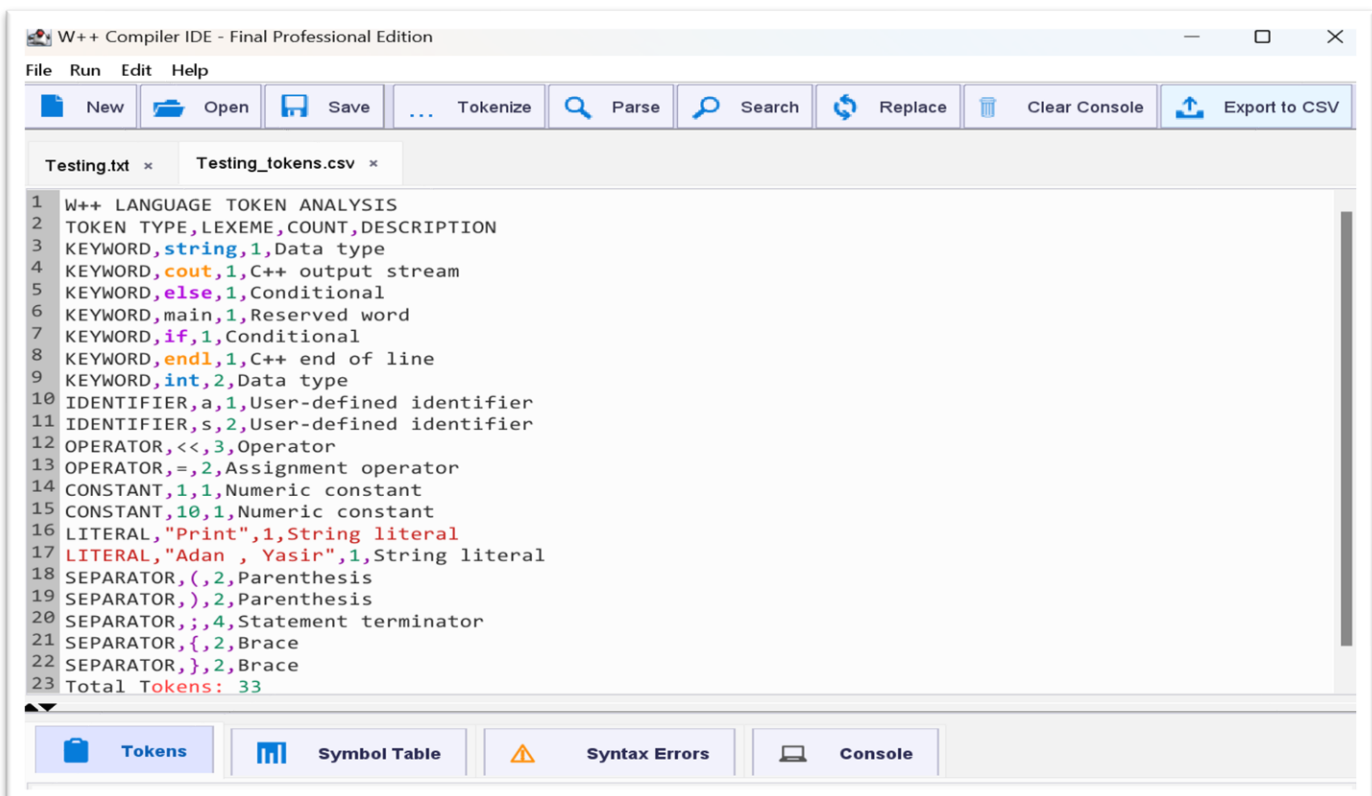


11. Export to CSV

Description:

Export the token table, symbol table, or error list to a CSV file for sharing or further analysis in spreadsheet software.

```
private void exportToCSV() {
    int selectedTab = outputPanel.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText(text:"No output tab selected");
        return;
    }
    // Determine which content to export
    String content = "";
    String fileNameSuffix = "";
    switch (selectedTab) {
        case 0: // Tokens tab
            content = tokenTableArea.getText();
            fileNameSuffix = "tokens";
            break;
        case 1: // Symbol Table tab
            content = symbolTableArea.getText();
            fileNameSuffix = "symbols";
            break;
        case 2: // Errors tab
            content = errorArea.getText();
            fileNameSuffix = "errors";
            break;
        default:
            JScrollPane scrollPane = (JScrollPane)outputPanel.getComponentAt(selectedTab);
            if (scrollPane.getViewPort().getView() instanceof JTextArea) {
                content = ((JTextArea)scrollPane.getViewPort().getView()).getText();
                fileNameSuffix = "output";
            }
    }
}
```

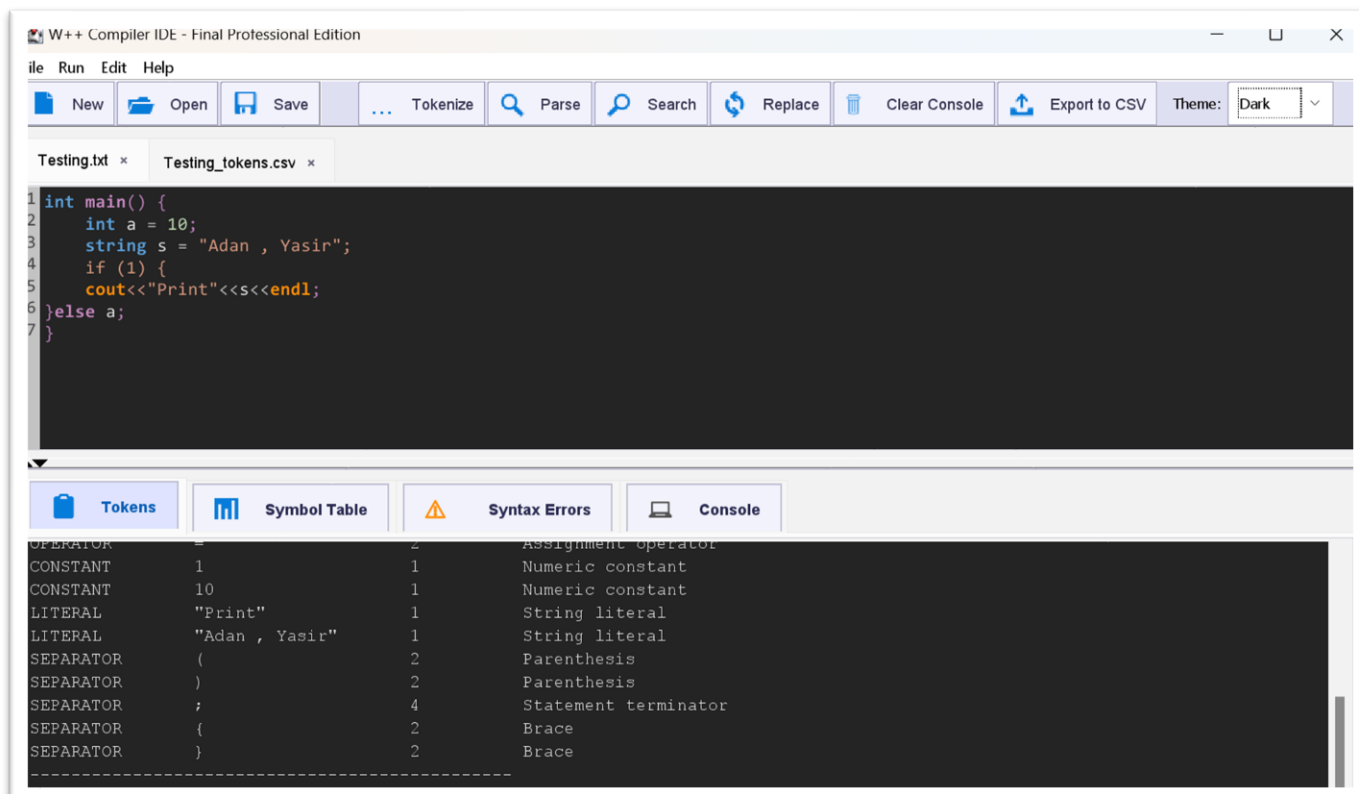


12. Theme Customization (Light/Dark Mode)

Description:

Switch between light and dark themes to suit your preference and reduce eye strain. The entire interface updates instantly.

```
private void applyTheme() {
    // Set colors based on the current theme
    Color bgColor = isDarkMode ? DARK_BG : LIGHT_BG;
    Color fgColor = isDarkMode ? DARK_FG : LIGHT_FG;
    // Set theme-specific syntax colors
    keywordColor = isDarkMode ? KEYWORD_COLOR_DARK : KEYWORD_COLOR_LIGHT;
    typeColor = isDarkMode ? TYPE_COLOR_DARK : TYPE_COLOR_LIGHT;
    stringColor = isDarkMode ? STRING_COLOR_DARK : STRING_COLOR_LIGHT;
    commentColor = isDarkMode ? COMMENT_COLOR_DARK : COMMENT_COLOR_LIGHT;
    numberColor = isDarkMode ? NUMBER_COLOR_DARK : NUMBER_COLOR_LIGHT;
    operatorColor = isDarkMode ? OPERATOR_COLOR_DARK : OPERATOR_COLOR_LIGHT;
    preprocessorColor = isDarkMode ? PREPROCESSOR_COLOR_DARK : PREPROCESSOR_COLOR_LIGHT;
    separatorColor = isDarkMode ? new Color(r:180, g:120, b:180) : new Color(r:140, g:0, b:160);
    // Apply theme to editors
    if (editors != null) {
        for (JTextPane editor : editors.values()) {
            if (editor != null) {
                editor.setBackground(bgColor);
                editor.setForeground(fgColor);
                editor.setCaretColor(fgColor);
                applySyntaxHighlighting(editor);
            }
        }
    }
    // Apply theme to output panels
    if (tokenTableArea != null) {
        tokenTableArea.setBackground(bgColor);
        tokenTableArea.setForeground(fgColor);
    }
}
```



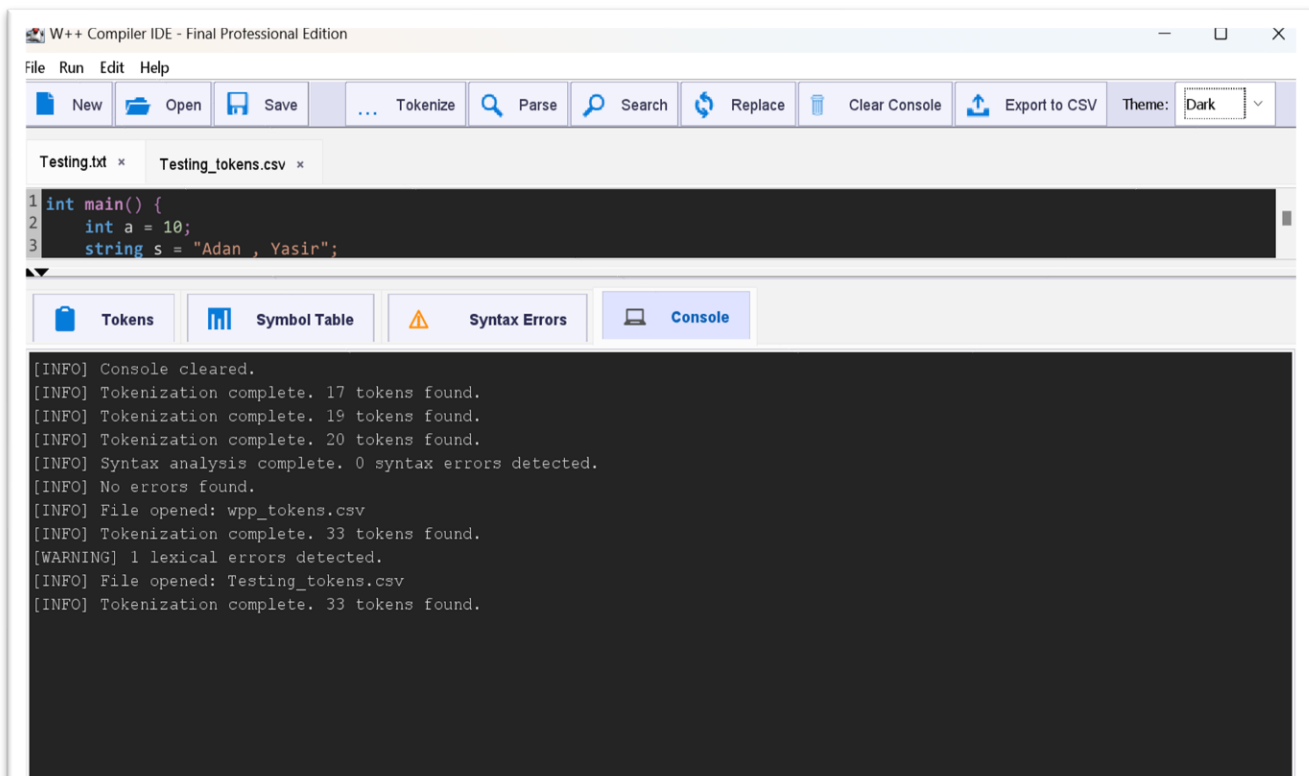
13. Session Logging and Console Output

Description:

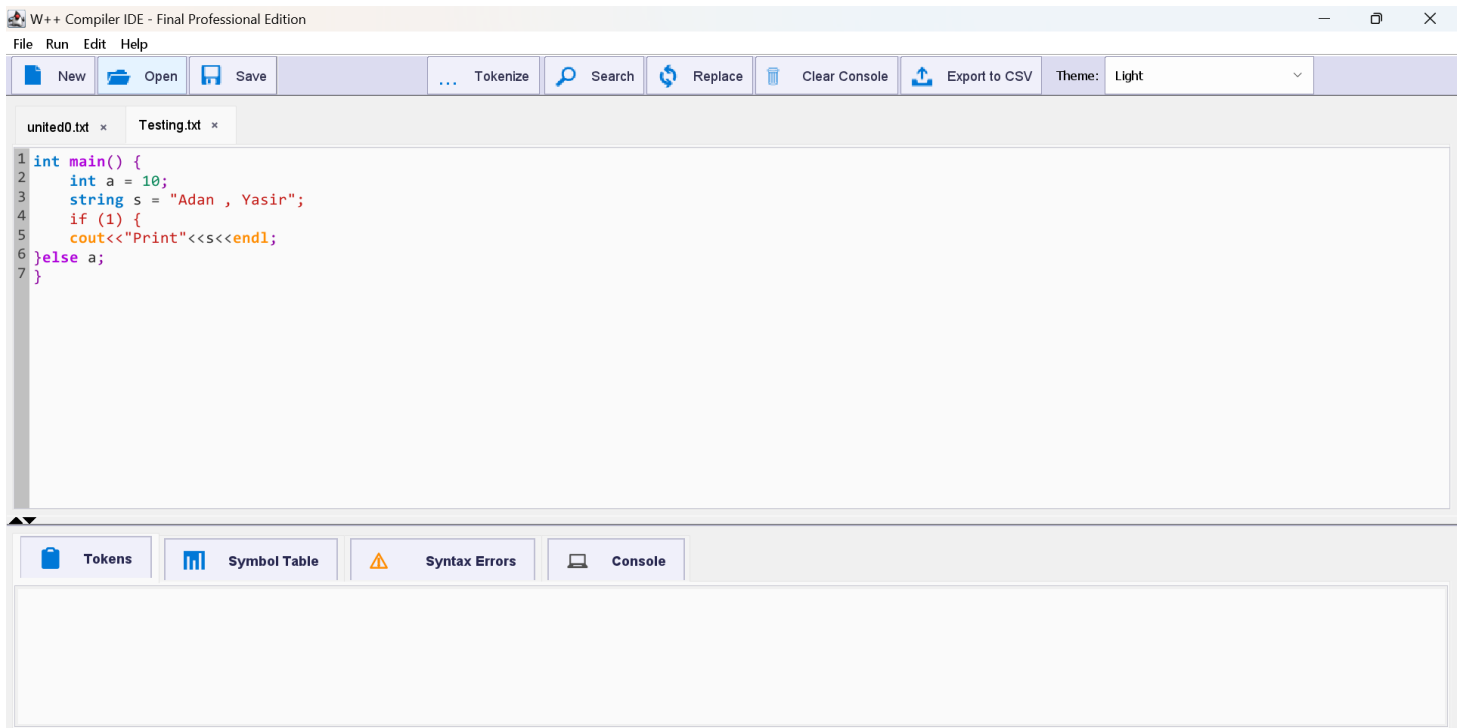
The IDE logs your actions and system messages in a console tab, helping you track your workflow and review past actions.

```
// Add this method to append messages to the Console tab:
private void logToConsole(String message) {
    JScrollPane consoleScroll = (JScrollPane) outputPanel.getComponentAt(index:3);
    JTextArea consoleArea = (JTextArea) consoleScroll.getViewPort().getView();
    if (consoleArea.getText().equals(anObject:"[No output simulated]")) {
        consoleArea.setText("");
    }
    consoleArea.append(message + "\n");
    sessionActions.add(message);
}

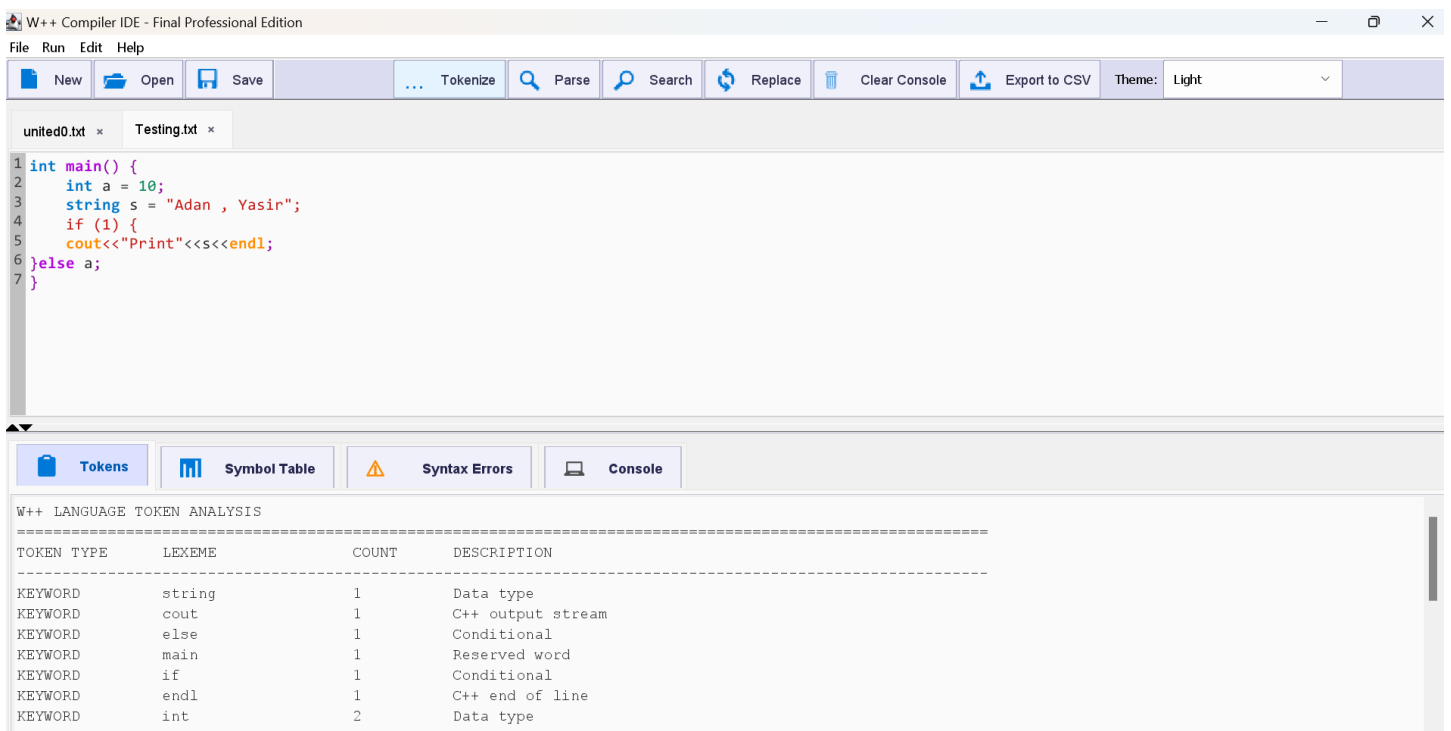
// Add auto-correction functionality
private void setupAutoCorrection(JTextPane editor) {
    editor.getDocument().addDocumentListener(new DocumentListener() {
        private boolean isCorrecting = false;
        @Override
        public void insertUpdate(DocumentEvent e) {
            if (!isCorrecting) {
                try {
                    String text = editor.getText();
                    int caretPos = editor.getCaretPosition();
                    if (caretPos == 0) return;
                    char lastChar = text.charAt(caretPos - 1);
                    // Define separators that should trigger correction
                    boolean isSeparator = !Character.isLetterOrDigit(lastChar);
                    int wordEnd = caretPos;
                    int wordStart = wordEnd;
                    // Always look back for the previous word
                    while (wordStart > 0 && Character.isLetterOrDigit(text.charAt(wordStart -
```



Lexer GUI Interface



Click the Tokenize button. The Parse button will then appear



Source Code

Lexer.java

```
import java.io.*;

import java.util.*;

public class Lexer {

    enum TokenType {

        KEYWORD, IDENTIFIER, OPERATOR, LITERAL, SEPARATOR, CONSTANT, ERROR, UNKNOWN

    }

    static class LexicalError {

        String token;

        String message;

        int line;

        int column;

        LexicalError(String token, String message, int line, int column) {

            this.token = token;

            this.message = message;

            this.line = line;

            this.column = column;

        }

        @Override

        public String toString() {

            return "Lexical Error at line " + line + ", column " + column + ": " + message + " [" + token + "];"

        }

    }

    static class SyntaxError {

        String message;
```

```
int line;
```

```
int column;
```

```
SyntaxError(String message, int line, int column) {
```

```
    this.message = message;
```

```
    this.line = line;
```

```
    this.column = column;
```

```
}
```

```
@Override
```

```
public String toString() {
```

```
    return "Syntax Error at line " + line + ", column " + column + ": " + message;
```

```
}
```

```
}
```

```
static class TokenInfo {
```

```
    String token;
```

```
    int line;
```

```
    int column;
```

```
    TokenInfo(String token, int line, int column) {
```

```
        this.token = token;
```

```
        this.line = line;
```

```
        this.column = column;
```

```
}
```

```
}
```

```
private static final Set<String> KEYWORDS = new HashSet<>(Arrays.asList(
```

```
    "int", "float", "double", "char", "if", "else", "while", "for", "return",
```

```
    "do", "string", "main", "cout", "endl", "std", "cin", "include", "iostream"));
```

```
private static final Set<String> OPERATORS = new HashSet<>(Arrays.asList(
```

```
    "+", "-", "*", "/", "=", "<", ">", "!", "==", "!=", "<=", ">=", "++", "--",
```

```
    "+=", "-=", "*=", "/=", "%=", "&&", "||", "%", "<<", ">>", "::", "&", "|"));
```

```

private static final Map<Character, Set<Character>> COMPOUND_OPS = new HashMap<>();
static {
    COMPOUND_OPS.put('+', new HashSet<>(Arrays.asList('+', '=')));
    COMPOUND_OPS.put('-', new HashSet<>(Arrays.asList('-', '=')));
    COMPOUND_OPS.put('*', new HashSet<>(Arrays.asList('=')));
    COMPOUND_OPS.put('/', new HashSet<>(Arrays.asList('=')));
    COMPOUND_OPS.put('%', new HashSet<>(Arrays.asList('=')));
    COMPOUND_OPS.put('=', new HashSet<>(Arrays.asList('=')));
    COMPOUND_OPS.put('!', new HashSet<>(Arrays.asList('=')));
    COMPOUND_OPS.put('<', new HashSet<>(Arrays.asList('=', '<')));
    COMPOUND_OPS.put('>', new HashSet<>(Arrays.asList('=', '>')));
    COMPOUND_OPS.put('&', new HashSet<>(Arrays.asList('&')));
    COMPOUND_OPS.put('|', new HashSet<>(Arrays.asList('|')));
    COMPOUND_OPS.put(':', new HashSet<>(Arrays.asList(':')));
}

```

```

private static final Set<Character> SEPARATORS = new HashSet<>(Arrays.asList(
    '(', ')', '{', '}', '[', ']', ';', ',', '#', '<', '>'));

```

```

static final Map<String, Integer> TYPE_SIZES = new HashMap<>();
static {
    TYPE_SIZES.put("int", 4);
    TYPE_SIZES.put("float", 4);
    TYPE_SIZES.put("double", 8);
    TYPE_SIZES.put("char", 1);
    TYPE_SIZES.put("string", 0);
}

```

```

public static void main(String[] args) {
    if (args.length == 0) {
        System.out.println("Please provide the path to the input file.");
        return;
    }
}

```

```
}
```

```
String filePath = args[0];
```

```
List<TokenInfo> tokens = new ArrayList<>();
```

```
List<LexicalError> lexicalErrors = new ArrayList<>();
```

```
List<SyntaxError> syntaxErrors = new ArrayList<>();
```

```
try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
```

```
    StringBuilder fileContent = new StringBuilder();
```

```
    String line;
```

```
    while ((line = reader.readLine()) != null) {
```

```
        fileContent.append(line).append("\n");
```

```
    }
```

```
    tokenizeWithErrors(fileContent.toString(), tokens, lexicalErrors);
```

```
    analyzeSyntax(tokens, syntaxErrors);
```

```
} catch (IOException e) {
```

```
    System.err.println("Error reading file: " + e.getMessage());
```

```
    return;
```

```
}
```

```
File outputFile = new File("output.txt");
```

```
if (outputFile.exists()) outputFile.delete();
```

```
try (PrintWriter writer = new PrintWriter(new BufferedWriter(new FileWriter("output.txt")))) {
```

```
    Map<TokenType, Map<String, Integer>> tokenCounts = new HashMap<>();
```

```
    Map<String, String[]> symbolTable = new HashMap<>();
```

```
    processTokensForSymbolTable(tokens, symbolTable);
```

```
    countTokens(tokens, tokenCounts);
```

```
    writer.println("TOKEN TABLE");
```

```
    writer.println("-----");
```

```
    writer.println("Token Type    Lexeme    Count");
```



```

writer.println("-----");
int totalTokens = printTokenTypeGroup(writer, tokenCounts, TokenType.OPERATOR) +
    printTokenTypeGroup(writer, tokenCounts, TokenType.IDENTIFIER) +
    printTokenTypeGroup(writer, tokenCounts, TokenType.KEYWORD) +
    printTokenTypeGroup(writer, tokenCounts, TokenType.CONSTANT) +
    printTokenTypeGroup(writer, tokenCounts, TokenType.LITERAL) +
    printTokenTypeGroup(writer, tokenCounts, TokenType.SEPARATOR);
writer.println("-----");
writer.println("Total Tokens: " + totalTokens);

writer.println("\nSYMBOL TABLE");
writer.println("-----");
writer.println("Name      Type      Size  Dimension  Line of Declaration  Line of Usage  Address
");
writer.println("-----");
for (Map.Entry<String, String[]> entry : symbolTable.entrySet()) {
    String identifier = entry.getKey();
    String[] info = entry.getValue();
    String type = info[0];
    String line = info[2];
    int size = TYPE_SIZES.getOrDefault(type, 0);
    String lineOfUsage = line;
    String address = "0x" + Integer.toHexString(identifier.hashCode() & 0xFFFF);
    writer.printf("%-15s %-15s %-8d %-12d %-18s %-15s %-10s\n", identifier, type, size, 0, line,
lineOfUsage, address);
}

if (!lexicalErrors.isEmpty()) {
    writer.println("\nLexical Errors Found (" + lexicalErrors.size() + "):");
    for (LexicalError error : lexicalErrors) {
        writer.println(error);
    }
}

```

```

if (!syntaxErrors.isEmpty()) {
    writer.println("\nSyntax Errors Found (" + syntaxErrors.size() + "):");
    for (SyntaxError error : syntaxErrors) {
        writer.println(error);
    }
}

if (lexicalErrors.isEmpty() && syntaxErrors.isEmpty()) {
    writer.println("No error found");
}

writer.flush();
System.out.println("Analysis complete. Results written to output.txt");
} catch (IOException e) {
    System.err.println("Error writing to output file: " + e.getMessage());
}
}

static void tokenizeWithErrors(String input, List<TokenInfo> tokenInfos, List<LexicalError> errors) {
    StringBuilder currentToken = new StringBuilder();
    boolean inString = false, inCharLiteral = false, inSingleLineComment = false,
        inMultiLineComment = false, inPreprocessor = false;
    int line = 1, column = 1, tokenStartLine = 1, tokenStartColumn = 1;

    for (int i = 0; i < input.length(); i++) {
        char ch = input.charAt(i);

        if (ch == '\n') {
            line++;
            column = 1;
            if (inSingleLineComment) inSingleLineComment = false;
            if (inPreprocessor) inPreprocessor = false;

```

```

        if (currentToken.length() > 0) {
            addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
            currentToken.setLength(0);
        }
        // Don't add newline as a token
        continue;
    } else {
        column++;
    }

    if (inMultiLineComment) {
        if (ch == '*' && i + 1 < input.length() && input.charAt(i + 1) == '/') {
            inMultiLineComment = false;
            i++;
            column++;
        }
        continue;
    }

    if (inSingleLineComment) continue;

    if (!inString && !inCharLiteral && !inSingleLineComment && !inMultiLineComment) {
        if (ch == '#' && (i == 0 || input.charAt(i - 1) == '\n')) {
            inPreprocessor = true;
            if (currentToken.length() > 0) {
                addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
                currentToken.setLength(0);
            }
            tokenInfos.add(new TokenInfo("#", line, column));
            continue;
        }
    }

```

```

        if (inPreprocessor) continue;
    }

    if (!inString && !inCharLiteral && !inPreprocessor) {
        if (ch == '/' && i + 1 < input.length()) {
            if (input.charAt(i + 1) == '*') {
                if (currentToken.length() > 0) {
                    addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
                    currentToken.setLength(0);
                }
                inMultiLineComment = true;
                i++;
                column++;
                continue;
            } else if (input.charAt(i + 1) == '/') {
                if (currentToken.length() > 0) {
                    addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
                    currentToken.setLength(0);
                }
                inSingleLineComment = true;
                i++;
                column++;
                continue;
            }
        }
    }

    if (ch == '"' && !inCharLiteral) {
        if (!inString) {
            if (currentToken.length() > 0) {
                addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);

```

```

        currentToken.setLength(0);
    }
    tokenStartLine = line;
    tokenStartColumn = column;
    inString = true;
    currentToken.append(ch);
} else {
    currentToken.append(ch);
    addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
    currentToken.setLength(0);
    inString = false;
}
continue;
}

if (ch == "\"" && !inString) {
    if (!inCharLiteral) {
        if (currentToken.length() > 0) {
            addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
            currentToken.setLength(0);
        }
        tokenStartLine = line;
        tokenStartColumn = column;
        inCharLiteral = true;
        currentToken.append(ch);
    } else {
        currentToken.append(ch);
        addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
        currentToken.setLength(0);
        inCharLiteral = false;
    }
}

```

```

        continue;
    }

    if ((inString || inCharLiteral) && ch == '\n') {
        String type = inString ? "string" : "character";
        errors.add(new LexicalError(currentToken.toString(), "Unterminated " + type + " literal",
tokenStartLine, tokenStartColumn));
        currentToken.setLength(0);
        inString = inCharLiteral = false;
        continue;
    }

    if (inString || inCharLiteral) {
        currentToken.append(ch);
        continue;
    }

    if (Character.isWhitespace(ch)) {
        if (currentToken.length() > 0) {
            addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
            currentToken.setLength(0);
        }
        continue;
    }

    if ((COMPOUND_OPS.containsKey(ch) && i + 1 < input.length() &&
COMPOUND_OPS.get(ch).contains(input.charAt(i + 1))) {
        if (currentToken.length() > 0) {
            addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
            currentToken.setLength(0);
        }
        tokenStartLine = line;
    }

```



```

tokenStartColumn = column;

String compoundOp = String.valueOf(ch) + input.charAt(i + 1);
tokenInfos.add(new TokenInfo(compoundOp, line, column));
i++;
column++;
} else if (OPERATORS.contains(String.valueOf(ch)) || SEPARATORS.contains(ch)) {
    if (currentToken.length() > 0) {
        addTokenWithValidation(currentToken.toString(), tokenInfos, errors, tokenStartLine,
tokenStartColumn);
        currentToken.setLength(0);
    }
    tokenStartLine = line;
    tokenStartColumn = column;
    tokenInfos.add(new TokenInfo(String.valueOf(ch), line, column));
} else {
    if (currentToken.length() == 0) {
        tokenStartLine = line;
        tokenStartColumn = column;
    }
    currentToken.append(ch);
}
}

```

```

    if (inString) errors.add(new LexicalError(currentToken.toString(), "Unterminated string literal",
tokenStartLine, tokenStartColumn));

```

```

    if (inCharLiteral) errors.add(new LexicalError(currentToken.toString(), "Unterminated character literal",
tokenStartLine, tokenStartColumn));

```

```

    if (inMultiLineComment) errors.add(new LexicalError("/*", "Unterminated multi-line comment",
tokenStartLine, tokenStartColumn));

```

```

    if (currentToken.length() > 0) addTokenWithValidation(currentToken.toString(), tokenInfos, errors,
tokenStartLine, tokenStartColumn);

```

```

}

```

```

    private static void addTokenWithValidation(String token, List<TokenInfo> tokenInfos, List<LexicalError>
errors, int line, int column) {

```

```

if (token.isEmpty()) return;
TokenType type = getTokenType(token);
if (type == TokenType.ERROR) {
    errors.add(new LexicalError(token, "Invalid token", line, column));
} else if (type == TokenType.IDENTIFIER && !isValidIdentifier(token)) {
    errors.add(new LexicalError(token, "Invalid identifier", line, column));
} else if (type == TokenType.LITERAL && isCharLiteral(token) && token.length() > 3) {
    // Check for char literals with multiple characters
    errors.add(new LexicalError(token, "Invalid char literal: char can only hold a single character", line,
column));
}
tokenInfos.add(new TokenInfo(token, line, column));
}

```

```

private static boolean isValidIdentifier(String token) {
    if (token.isEmpty()) return false;
    if (!Character.isLetter(token.charAt(0)) && token.charAt(0) != '_') return false;
    for (int i = 1; i < token.length(); i++) {
        char c = token.charAt(i);
        if (!Character.isLetterOrDigit(c) && c != '_') return false;
    }
    return true;
}

```

```

private static boolean isCharLiteral(String token) {
    return token.startsWith("\"") && token.endsWith("\"");
}

```

```

static TokenType getTokenType(String token) {
    if (KEYWORDS.contains(token)) return TokenType.KEYWORD;
    if (isValidIdentifier(token)) return TokenType.IDENTIFIER;
    if (token.matches("[0-9]+")) return TokenType.CONSTANT;
    if (token.matches("[0-9]*\\.[0-9]+")) return TokenType.LITERAL;
}

```

```

    if ((token.startsWith("\\'") && token.endsWith("\\'")) || (token.startsWith("'") && token.endsWith("'")))
        return TokenType.LITERAL;
    if (OPERATORS.contains(token)) return TokenType.OPERATOR;
    if (SEPARATORS.contains(token.charAt(0)) && token.length() == 1) return TokenType.SEPARATOR;
    return TokenType.ERROR;
}

static void processTokensForSymbolTable(List<TokenInfo> tokens, Map<String, String[]> symbolTable) {
    String currentType = "Unknown";
    boolean isAfterEquals = false;
    String currentIdentifier = "";
    int currentLine = 1;

    for (int i = 0; i < tokens.size(); i++) {
        TokenInfo tokenInfo = tokens.get(i);
        String token = tokenInfo.token;
        TokenType type = getTokenType(token);

        if (token.equals(";")) currentLine++;

        if (type == TokenType.KEYWORD && isDataType(token)) {
            currentType = token;
            continue;
        }

        if (type == TokenType.IDENTIFIER && !token.equals("main")) {
            currentIdentifier = token;
            if (!symbolTable.containsKey(token)) {
                symbolTable.put(token, new String[]{currentType, "Not initialized", String.valueOf(currentLine)});
            }
            if (i + 1 < tokens.size() && tokens.get(i + 1).token.equals("=")) {
                isAfterEquals = true;
            }
        }
    }
}

```

```

        continue;
    }

    if (isAfterEquals && (type == TokenType.CONSTANT || type == TokenType.LITERAL || type ==
TokenType.IDENTIFIER)) {
        if (!currentIdentifier.isEmpty() && symbolTable.containsKey(currentIdentifier)) {
            symbolTable.get(currentIdentifier)[1] = token;
        }
        isAfterEquals = false;
    }
}

}

private static void countTokens(List<TokenInfo> tokens, Map<TokenType, Map<String, Integer>>
tokenCounts) {
    for (TokenInfo tokenInfo : tokens) {
        String token = tokenInfo.token;
        TokenType type = getTokenType(token);
        tokenCounts.computeIfAbsent(type, k -> new HashMap<>()).merge(token, 1, Integer::sum);
    }
}

private static int printTokenTypeGroup(PrintWriter writer, Map<TokenType, Map<String, Integer>>
tokenCounts, TokenType type) {
    if (!tokenCounts.containsKey(type)) return 0;
    int count = 0;
    for (Map.Entry<String, Integer> entry : tokenCounts.get(type).entrySet()) {
        writer.printf("%-15s %-15s %-5d\n", type, entry.getKey(), entry.getValue());
        count += entry.getValue();
    }
    return count;
}

private static boolean isDataType(String keyword) {

```

```

    return KEYWORDS.contains(keyword) && TYPE_SIZES.containsKey(keyword);
}

```

```

static void analyzeSyntax(List<TokenInfo> tokens, List<SyntaxError> errors) {
    int line = 1;
    int column = 1;
    Stack<String> blockStack = new Stack<>();
    boolean inFunction = false;
    String lastType = null;
    String lastIdentifier = null;
    boolean expectingSemicolon = false;
    boolean hasMainFunction = false;
    boolean inDeclaration = false;
    int parenCount = 0;
    boolean inIfBlock = false;
    boolean inElseBlock = false;
    boolean inComment = false;
    boolean inString = false;
    boolean inChar = false;
    String currentFunction = null;
    boolean inMultiLineComment = false;
    Set<String> declaredVariables = new HashSet<>();
    Set<String> initializedVariables = new HashSet<>();
    boolean inIfStatement = false;
    boolean foundIfCondition = false;
    // --- Strict C++-compliant if-else matching logic ---
    class IfInfo { boolean matched; IfInfo() { matched = false; } }
    Stack<IfInfo> ifStack = new Stack<>();

    for (int i = 0; i < tokens.size(); i++) {
        TokenInfo tokenInfo = tokens.get(i);
        String token = tokenInfo.token;
        TokenType type = getTokenType(token);
    }
}

```

```

// --- Robust check for empty if condition (if () ...) ---
if (token.equals("if")) {
    int idx = i + 1;
    // Skip newlines/whitespace
    while (idx < tokens.size() && tokens.get(idx).token.equals("\n")) idx++;
    if (idx < tokens.size() && tokens.get(idx).token.equals("(")) {
        int idx2 = idx + 1;
        while (idx2 < tokens.size() && tokens.get(idx2).token.equals("\n")) idx2++;
        if (idx2 < tokens.size() && tokens.get(idx2).token.equals("))")) {
            errors.add(new SyntaxError("Error: if statement without condition", tokenInfo.line,
tokenInfo.column));
        }
    }
}

// Track block depth
if (token.equals("{")) {
    blockStack.push("{");
} else if (token.equals("}")) {
    if (blockStack.isEmpty()) {
        errors.add(new SyntaxError("Unmatched closing brace", tokenInfo.line, tokenInfo.column));
    } else {
        blockStack.pop();
    }
    if (inIfBlock) inIfBlock = false;
    if (inElseBlock) inElseBlock = false;
}

// Check for if statement
if (token.equals("if")) {
    inIfStatement = true;
    foundIfCondition = false;

```

```

    ifStack.push(new IfInfo());
}

// Check for else statement
if (token.equals("else")) {
    // Allow else if (should not count as double else)
    boolean isElseIf = (i + 1 < tokens.size() && tokens.get(i + 1).token.equals("if"));
    // Find the nearest unmatched if
    int matchIdx = -1;
    for (int j = ifStack.size() - 1; j >= 0; j--) {
        IfInfo info = ifStack.get(j);
        if (!info.matched) {
            matchIdx = j;
            break;
        }
    }
    if (matchIdx == -1) {
        errors.add(new SyntaxError("Error: else without matching if", tokenInfo.line, tokenInfo.column));
    } else {
        IfInfo matchedIf = ifStack.get(matchIdx);
        if (matchedIf.matched) {
            errors.add(new SyntaxError("Error: multiple else for one if", tokenInfo.line, tokenInfo.column));
        } else {
            matchedIf.matched = true;
        }
    }
}

// --- NEW LOGIC: Check for valid statement or block after else ---
int next = i + 1;
// Skip newlines
while (next < tokens.size() && tokens.get(next).token.equals("\n")) next++;
if (next < tokens.size()) {
    String nextToken = tokens.get(next).token;
    if (!nextToken.equals("{}")) {

```

```

// Not a block, should be a statement ending with semicolon
int stmtEnd = next + 1;
boolean foundSemicolon = false;
while (stmtEnd < tokens.size()) {
    if (tokens.get(stmtEnd).token.equals(";")) {
        foundSemicolon = true;
        break;
    }
    // If we hit a block or another control keyword, break
    if (tokens.get(stmtEnd).token.equals("{") || tokens.get(stmtEnd).token.equals("if") ||
tokens.get(stmtEnd).token.equals("else")) break;
    stmtEnd++;
}
if (!foundSemicolon) {
    errors.add(new SyntaxError("Missing semicolon or invalid statement after else",
tokens.get(next).line, tokens.get(next).column));
}
}
}
}

```

```

// Reset if statement tracking when we find the closing brace

```

```

if (inIfStatement && token.equals("}")) {
    inIfStatement = false;
    foundIfCondition = false;
}

```

```

// --- THIS BLOCK IS NOW THE FIRST LOGIC IN THE LOOP ---

```

```

if (type == TokenType.KEYWORD && i > 0) {
    String prevToken = tokens.get(i - 1).token;
    TokenType prevType = getTokenType(prevToken);
    if (prevType == TokenType.KEYWORD) {
        // Allow only 'int main' as consecutive keywords
    }
}

```



```

        if (!(prevToken.equals("int") && token.equals("main"))) {
            errors.add(new SyntaxError("Invalid consecutive keywords: '" + prevToken + " ' + token + '".
Only 'int main' is allowed as consecutive keywords.", tokenInfo.line, tokenInfo.column));
        }
    }
}

// Track line and column for error reporting
if (token.equals("\n")) {
    line++;
    column = 1;
    continue;
}
column += token.length();

// Handle comments
if (token.equals("//")) {
    inComment = true;
    continue;
}
if (inComment && token.equals("\n")) {
    inComment = false;
    continue;
}
if (token.equals("/*")) {
    inMultiLineComment = true;
    continue;
}
if (token.equals("*/")) {
    inMultiLineComment = false;
    continue;
}
if (inComment || inMultiLineComment) continue;

```

```

// Handle block structure BEFORE skipping string/char
if (token.equals("{")) {
    blockStack.push("{");
} else if (token.equals("}")) {
    if (blockStack.isEmpty()) {
        errors.add(new SyntaxError("Unmatched closing brace", tokenInfo.line, tokenInfo.column));
    } else {
        blockStack.pop();
        if (inIfBlock) inIfBlock = false;
        if (inElseBlock) inElseBlock = false;
    }
}

```

```

// Handle string and char literals
if (token.startsWith("\"")) inString = !inString;
if (token.startsWith("'")) inChar = !inChar;
if (inString || inChar) continue;

```

```

// Special handling for main function
if (token.equals("main") && i > 0 && tokens.get(i-1).token.equals("int")) {
    hasMainFunction = true;
    currentFunction = "main";
    // Skip validation for main function
    continue;
}

```

```

// Check for undeclared variables
if (type == TokenType.IDENTIFIER && !token.equals("main")) {
    // Check if this is a variable declaration
    boolean isDeclaration = i > 0 && isDataType(tokens.get(i-1).token);
    if (!isDeclaration && !declaredVariables.contains(token)) {

```

```

        errors.add(new SyntaxError("Variable '" + token + "' is not declared", tokenInfo.line,
tokenInfo.column));
    }
}

// Check variable declarations
if (isDataType(token)) {
    lastType = token;
    inDeclaration = true;
    if (i + 1 >= tokens.size()) {
        errors.add(new SyntaxError("Incomplete variable declaration", tokenInfo.line, tokenInfo.column));
        continue;
    }
    String nextToken = tokens.get(i + 1).token;
    TokenInfo nextTokenInfo = tokens.get(i + 1);
    // Disallow 'main' as a variable name (unless it's a function declaration)
    boolean isFunctionDecl = nextToken.equals("main") && (i + 2 < tokens.size() && tokens.get(i +
2).token.equals("("));
    if (nextToken.equals("main") && !isFunctionDecl) {
        errors.add(new SyntaxError("'main' is a reserved keyword and cannot be used as a variable name",
nextTokenInfo.line, nextTokenInfo.column));
    } else if (!isValidIdentifier(nextToken) && !nextToken.equals("main")) {
        errors.add(new SyntaxError("Invalid variable name: " + nextToken, nextTokenInfo.line,
nextTokenInfo.column));
    }
    // Only check for missing semicolon if this is NOT a function declaration
    if (!isFunctionDecl) {
        // Check for multiple declarations in the same scope
        if (declaredVariables.contains(nextToken)) {
            errors.add(new SyntaxError("Multiple declaration of variable '" + nextToken + "' in the same
scope", nextTokenInfo.line, nextTokenInfo.column));
        }
        // Check for missing semicolon
        int j = i + 2;

```

```

boolean foundSemicolon = false;
while (j < tokens.size() && !tokens.get(j).token.equals(";")) {
    if (tokens.get(j).token.equals("\n")) break;
    j++;
}
if (j >= tokens.size() || !tokens.get(j).token.equals(";")) {
    // Report error at the end of the declaration line
    int errorLine = nextTokenInfo.line;
    int errorCol = nextTokenInfo.column + nextToken.length();
    errors.add(new SyntaxError("Missing semicolon in variable declaration", errorLine, errorCol));
}
}
continue;
}

// Check variable initialization
if (inDeclaration && type == TokenType.IDENTIFIER) {
    lastIdentifier = token;
    declaredVariables.add(token);
    if (i + 1 < tokens.size()) {
        String nextToken = tokens.get(i + 1).token;
        TokenInfo nextTokenInfo = tokens.get(i + 1);
        if (nextToken.equals("=")) {
            // Check if variable is already initialized
            if (initializedVariables.contains(token)) {
                errors.add(new SyntaxError("Variable '" + token + "' is already initialized", nextTokenInfo.line,
nextTokenInfo.column));
            } else {
                initializedVariables.add(token);
            }
            if (i + 2 >= tokens.size() || getTokenType(tokens.get(i + 2).token) == TokenType.SEPARATOR)
{
                errors.add(new SyntaxError("Initialization without a value", nextTokenInfo.line,
nextTokenInfo.column));
            }
        }
    }
}

```

```

    } else {
        // Check for char type initialization
        if (lastType != null && lastType.equals("char")) {
            String valueToken = tokens.get(i + 2).token;
            if (isCharLiteral(valueToken) && valueToken.length() > 3) {
                errors.add(new SyntaxError("Invalid char initialization: char can only hold a single
character", nextTokenInfo.line, nextTokenInfo.column));
            }
        }
        // Check for string type initialization
        if (lastType != null && lastType.equals("string")) {
            String valueToken = tokens.get(i + 2).token;
            if (isCharLiteral(valueToken)) {
                errors.add(new SyntaxError("String values must be enclosed in double quotes ('...') in
C++.", nextTokenInfo.line, nextTokenInfo.column));
            }
        }
    }
}
inDeclaration = false;
continue;
}

// Enhanced assignment statement checks
if (token.equals("=") && i > 0) {
    String lhs = tokens.get(i - 1).token;
    TokenType lhsType = getTokenType(lhs);
    // 1. LHS must be a valid identifier, not a keyword, constant, or function name
    if (lhsType != TokenType.IDENTIFIER || lhs.equals("main") || KEYWORDS.contains(lhs)) {
        errors.add(new SyntaxError("Invalid assignment target: '" + lhs + "'", tokenInfo.line,
tokenInfo.column - 1));
    } else if (!declaredVariables.contains(lhs)) {

```

```

        errors.add(new SyntaxError("Assignment without prior declaration: " + lhs, tokenInfo.line,
tokenInfo.column - 1));
    }

    // 2. RHS must exist and be a valid value
    if (i + 1 >= tokens.size()) {
        errors.add(new SyntaxError("Incomplete assignment statement", tokenInfo.line,
tokenInfo.column));
    } else {
        String rhs = tokens.get(i + 1).token;
        TokenType rhsType = getTokenType(rhs);
        if (!(rhsType == TokenType.IDENTIFIER || rhsType == TokenType.CONSTANT || rhsType ==
TokenType.LITERAL)) {
            errors.add(new SyntaxError("Invalid assignment value: '" + rhs + "'", tokens.get(i + 1).line,
tokens.get(i + 1).column));
        }
    }

    // 3. Check for missing semicolon after assignment
    int j = i + 2;
    boolean foundSemicolon = false;
    while (j < tokens.size() && !tokens.get(j).token.equals(";")) {
        if (tokens.get(j).token.equals("\n")) break;
        j++;
    }
    if (j >= tokens.size() || !tokens.get(j).token.equals(";")) {
        int errorLine = tokenInfo.line;
        int errorCol = tokenInfo.column + 1;
        errors.add(new SyntaxError("Missing semicolon after assignment statement", errorLine, errorCol));
    }
    continue;
}

// Flag 'if-else' as invalid
if (token.equals("if-else")) {

```

```

        errors.add(new SyntaxError("'if-else' is not a valid statement. Use 'if (...) { ... } else { ... }' instead.",
tokenInfo.line, tokenInfo.column));
        continue;
    }

    // Enhanced if statement checks
    if (token.equals("if")) {
        inIfBlock = true;
        // Check for opening parenthesis
        if (i + 1 >= tokens.size() || !tokens.get(i + 1).token.equals("(")) {
            errors.add(new SyntaxError("if condition without proper parentheses", tokenInfo.line,
tokenInfo.column));
            continue;
        }
        // Find matching closing parenthesis
        int parenDepth = 1;
        int j = i + 2;
        boolean foundClosingParen = false;
        while (j < tokens.size()) {
            String t = tokens.get(j).token;
            if (t.equals("(")) parenDepth++;
            else if (t.equals(")")) parenDepth--;
            if (parenDepth == 0) {
                foundClosingParen = true;
                break;
            }
            j++;
        }
        if (!foundClosingParen) {
            errors.add(new SyntaxError("Missing closing parenthesis in if condition", tokenInfo.line,
tokenInfo.column));
            continue;
        }
        // Check for opening brace after condition

```

```

    int k = j + 1;
    while (k < tokens.size() && tokens.get(k).token.equals("\n")) k++; // skip newlines
    if (k >= tokens.size() || !tokens.get(k).token.equals("{")) {
        errors.add(new SyntaxError("Missing opening brace after if condition", tokenInfo.line,
tokenInfo.column));
    }
    continue;
}

// Enhanced for loop checks
if (token.equals("for")) {
    // Check for opening parenthesis
    if (i + 1 >= tokens.size() || !tokens.get(i + 1).token.equals("(")) {
        errors.add(new SyntaxError("for loop missing opening parenthesis '(', tokenInfo.line,
tokenInfo.column));
        continue;
    }
    // Find matching closing parenthesis and check for semicolons
    int parenDepth = 1;
    int j = i + 2;
    int semicolonCount = 0;
    boolean foundClosingParen = false;
    boolean foundFirstSemicolon = false, foundSecondSemicolon = false;
    while (j < tokens.size()) {
        String t = tokens.get(j).token;
        if (t.equals("(")) parenDepth++;
        else if (t.equals(")")) parenDepth--;
        if (t.equals(";")) {
            if (!foundFirstSemicolon) foundFirstSemicolon = true;
            else if (!foundSecondSemicolon) foundSecondSemicolon = true;
            semicolonCount++;
        }
    }
    if (parenDepth == 0) {

```



```

        foundClosingParen = true;
        break;
    }
    j++;
}
if (!foundClosingParen) {
    errors.add(new SyntaxError("for loop missing closing parenthesis '", tokenInfo.line,
tokenInfo.column));
    continue;
}
if (semicolonCount < 2) {
    errors.add(new SyntaxError("for loop must have two semicolons in header (for(init;cond;inc))",
tokenInfo.line, tokenInfo.column));
}
// Check for opening brace after condition
int k = j + 1;
while (k < tokens.size() && tokens.get(k).token.equals("\n")) k++; // skip newlines
if (k >= tokens.size() || !tokens.get(k).token.equals("{")) {
    errors.add(new SyntaxError("for loop missing opening brace '{' after header", tokenInfo.line,
tokenInfo.column));
}
continue;
}

// Enhanced while loop checks
if (token.equals("while")) {
    // Check for opening parenthesis
    if (i + 1 >= tokens.size() || !tokens.get(i + 1).token.equals("(")) {
        errors.add(new SyntaxError("while loop missing opening parenthesis '(', tokenInfo.line,
tokenInfo.column));
        continue;
    }
    // Find matching closing parenthesis
    int parenDepth = 1;

```

```

int j = i + 2;
boolean foundClosingParen = false;
boolean foundCondition = false;
while (j < tokens.size()) {
    String t = tokens.get(j).token;
    if (t.equals("(")) parenDepth++;
    else if (t.equals(")") parenDepth--;
    if (!t.equals("(") && !t.equals(")") && parenDepth == 1) foundCondition = true;
    if (parenDepth == 0) {
        foundClosingParen = true;
        break;
    }
    j++;
}
if (!foundClosingParen) {
    errors.add(new SyntaxError("while loop missing closing parenthesis ')", tokenInfo.line,
tokenInfo.column));
    continue;
}
if (!foundCondition) {
    errors.add(new SyntaxError("while loop missing condition in parentheses", tokenInfo.line,
tokenInfo.column));
}
// Check for opening brace after condition
int k = j + 1;
while (k < tokens.size() && tokens.get(k).token.equals("\n")) k++; // skip newlines
if (k >= tokens.size() || !tokens.get(k).token.equals("{")) {
    errors.add(new SyntaxError("while loop missing opening brace '{' after condition", tokenInfo.line,
tokenInfo.column));
}
continue;
}

// Enhanced do-while loop checks

```

```

if (token.equals("do")) {
    // Check for opening brace after do
    int j = i + 1;
    while (j < tokens.size() && tokens.get(j).token.equals("\n")) j++; // skip newlines
    if (j >= tokens.size() || !tokens.get(j).token.equals("{")) {
        errors.add(new SyntaxError("do-while loop missing opening brace '{' after 'do'", tokenInfo.line,
tokenInfo.column));
        continue;
    }
    // Find matching closing brace
    int braceDepth = 1;
    int k = j + 1;
    boolean foundClosingBrace = false;
    while (k < tokens.size()) {
        String t = tokens.get(k).token;
        if (t.equals("{")) braceDepth++;
        else if (t.equals("}")) braceDepth--;
        if (braceDepth == 0) {
            foundClosingBrace = true;
            break;
        }
        k++;
    }
    if (!foundClosingBrace) {
        errors.add(new SyntaxError("do-while loop missing closing brace '}'", tokenInfo.line,
tokenInfo.column));
        continue;
    }
    // After closing brace, expect 'while', '(', condition, ')', and ';'
    int m = k + 1;
    while (m < tokens.size() && tokens.get(m).token.equals("\n")) m++; // skip newlines
    if (m >= tokens.size() || !tokens.get(m).token.equals("while")) {
        errors.add(new SyntaxError("do-while loop missing 'while' after block", tokenInfo.line,
tokenInfo.column));
    }
}

```

```

        continue;
    }
    if (m + 1 >= tokens.size() || !tokens.get(m + 1).token.equals("(")) {
        errors.add(new SyntaxError("do-while loop missing opening parenthesis '(' after 'while'",
tokenInfo.line, tokenInfo.column));
        continue;
    }
    // Find matching closing parenthesis for while
    int parenDepth = 1;
    int n = m + 2;
    boolean foundClosingParen = false;
    boolean foundCondition = false;
    while (n < tokens.size()) {
        String t = tokens.get(n).token;
        if (t.equals("(")) parenDepth++;
        else if (t.equals(")") parenDepth--;
        if (!t.equals("(") && !t.equals(")") && parenDepth == 1) foundCondition = true;
        if (parenDepth == 0) {
            foundClosingParen = true;
            break;
        }
        n++;
    }
    if (!foundClosingParen) {
        errors.add(new SyntaxError("do-while loop missing closing parenthesis ')' after 'while'",
tokenInfo.line, tokenInfo.column));
        continue;
    }
    if (!foundCondition) {
        errors.add(new SyntaxError("do-while loop missing condition in parentheses after 'while'",
tokenInfo.line, tokenInfo.column));
    }
    // After closing parenthesis, expect semicolon
    int p = n + 1;

```

```

while (p < tokens.size() && tokens.get(p).token.equals("\n")) p++; // skip newlines
if (p < tokens.size() && tokens.get(p).token.equals("{")) {
    errors.add(new SyntaxError("Unexpected brace after while in do-while loop. In C++, the condition
after while should be followed by a semicolon, not a block.", tokenInfo.line, tokenInfo.column));
} else if (p >= tokens.size() || !tokens.get(p).token.equals(";")) {
    errors.add(new SyntaxError("do-while loop missing semicolon ';' after while condition",
tokenInfo.line, tokenInfo.column));
}
// Advance i to after the while (...);
i = p;
continue;
}

// Check for common keyword typos
if (type == TokenType.IDENTIFIER) {
    if (token.equals("el")) {
        errors.add(new SyntaxError("Unknown or invalid keyword 'el'. Did you mean 'else'?",
tokenInfo.line, tokenInfo.column));
    }
    // You can add more typo checks here if needed
}
}

// Check for unmatched braces at end of file
if (!blockStack.isEmpty()) {
    errors.add(new SyntaxError("Unmatched opening brace", line, column));
}

// Check if main function exists and is unique
boolean foundMain = false;
int mainCount = 0;
for (int i = 0; i < tokens.size(); i++) {
    // Look for 'int main' followed by '(' (allowing for whitespace/newlines)
    if (tokens.get(i).token.equals("int")) {

```

```

int j = i + 1;
while (j < tokens.size() && tokens.get(j).token.equals("\n")) j++;
if (j < tokens.size() && tokens.get(j).token.equals("main")) {
    int k = j + 1;
    while (k < tokens.size() && tokens.get(k).token.equals("\n")) k++;
    // Check for opening parenthesis
    if (k >= tokens.size() || !tokens.get(k).token.equals("(")) {
        errors.add(new SyntaxError("main function missing opening parenthesis '(', tokens.get(j).line,
tokens.get(j).column));
        continue;
    }
    // Check for closing parenthesis
    int parenDepth = 1;
    int m = k + 1;
    boolean foundClosingParen = false;
    while (m < tokens.size()) {
        String t = tokens.get(m).token;
        if (t.equals("(")) parenDepth++;
        else if (t.equals(")")) parenDepth--;
        if (parenDepth == 0) {
            foundClosingParen = true;
            break;
        }
        m++;
    }
    if (!foundClosingParen) {
        errors.add(new SyntaxError("main function missing closing parenthesis ')'", tokens.get(j).line,
tokens.get(j).column));
        continue;
    }
    // Check for opening brace after main()
    int n = m + 1;
    while (n < tokens.size() && tokens.get(n).token.equals("\n")) n++;

```

```

        if (n >= tokens.size() || !tokens.get(n).token.equals("{")) {
            errors.add(new SyntaxError("main function missing opening brace '{' after main()",
tokens.get(j).line, tokens.get(j).column));
            continue;
        }
        foundMain = true;
        mainCount++;
    }
}
}
if (!foundMain) {
    errors.add(new SyntaxError("Missing main function", 1, 1));
} else if (mainCount > 1) {
    errors.add(new SyntaxError("Multiple main functions detected. Only one main function is allowed.", 1,
1));
}
}
}
}

```

LexerGUI.java

```

import javax.swing.*.*;
import javax.swing.event.DocumentEvent;
import javax.swing.event.DocumentListener;
import javax.swing.text.*;
import javax.swing.text.JTextComponent;
import java.awt.*.*;
import java.awt.event.*;
import java.awt.event.KeyEvent;
import java.awt.event.InputEvent;
import java.awt.Insets;
import java.io.*;
import java.util.*;

```

```

import java.util.List;

import javax.swing.KeyStroke;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

import javax.imageio.ImageIO;

import java.net.URL;

import java.awt.image.BufferedImage;

import java.awt.Graphics2D;

import java.awt.RenderingHints;


public class LexerGUI extends JFrame {

    private JTabbedPane tabbedPane;

    private JTabbedPane outputPanel;

    private JPanel buttonPanel;

    private JTextArea tokenTableArea;

    private JTextArea symbolTableArea;

    private JTextArea errorArea;

    private JLabel statusLabel;

    private Map<Integer, JTextPane> editors;

    private Map<Integer, String> filePaths;

    private boolean isDarkMode = false;

    private int fontSize = 14;

    private JMenuBar menuBar;

    private javax.swing.Timer highlightTimer;

    private static final Set<String> KNOWN_WORDS = new HashSet<>(Arrays.asList(
        "int", "float", "double", "char", "if", "else", "while", "for", "return",
        "string", "do", "main", "cout", "endl", "if-else"));


    // Refined color scheme with more vibrant colors based on the example image
    private static final Color DARK_BG = new Color(30, 30, 30);

    private static final Color DARK_FG = new Color(220, 220, 220);

    private static final Color LIGHT_BG = new Color(250, 250, 250);

    private static final Color LIGHT_FG = new Color(40, 40, 40);

```



```

// Enhanced colors for code elements

private static final Color KEYWORD_COLOR_DARK = new Color(197, 134, 192); // Purple for keywords
private static final Color KEYWORD_COLOR_LIGHT = new Color(175, 0, 219); // Deeper purple for light
theme

private static final Color TYPE_COLOR_DARK = new Color(86, 156, 214); // Blue for types
private static final Color TYPE_COLOR_LIGHT = new Color(0, 119, 200); // Deeper blue for light theme

private static final Color STRING_COLOR_DARK = new Color(206, 145, 120); // Orange for strings
private static final Color STRING_COLOR_LIGHT = new Color(196, 26, 22); // Red-orange for light theme

private static final Color COMMENT_COLOR_DARK = new Color(106, 153, 85); // Green for comments
private static final Color COMMENT_COLOR_LIGHT = new Color(0, 128, 0); // Deeper green for light
theme

private static final Color NUMBER_COLOR_DARK = new Color(181, 206, 168); // Light green for numbers
private static final Color NUMBER_COLOR_LIGHT = new Color(9, 136, 90); // Teal for light theme

private static final Color OPERATOR_COLOR_DARK = new Color(180, 180, 180); // Light gray for
operators
private static final Color OPERATOR_COLOR_LIGHT = new Color(64, 64, 64); // Dark gray for light theme

private static final Color PREPROCESSOR_COLOR_DARK = new Color(189, 99, 197); // Magenta for
preprocessor
private static final Color PREPROCESSOR_COLOR_LIGHT = new Color(163, 21, 21); // Deep red for light
theme

// Orange for cout/endl

private static final Color COUT_ENDL_COLOR_DARK = new Color(255, 140, 0);
private static final Color COUT_ENDL_COLOR_LIGHT = new Color(255, 140, 0);

private static final Color ERROR_COLOR = new Color(255, 50, 50);
private static final Color ERROR_BG_COLOR = new Color(100, 0, 0);

```

```

private static final Color SYNTAX_ERROR_COLOR = new Color(255, 100, 100);
private static final Color ACCENT_COLOR = new Color(0, 120, 204);

// Current theme colors that will be set based on dark/light mode
private Color keywordColor;
private Color typeColor;
private Color stringColor;
private Color commentColor;
private Color numberColor;
private Color operatorColor;
private Color preprocessorColor;
private Color separatorColor;

// At the top with other fields:
private JPanel outputContainer;

// 1. Add fields to track session actions and error counts:
private java.util.List<String> sessionActions = new java.util.ArrayList<>();
private int sessionLexicalErrors = 0;
private int sessionSyntaxErrors = 0;
private int sessionTokens = 0;

// Add icon-related fields
private Map<String, ImageIcon> iconCache;
private static final String[] ICON_CODES = {
    " 📄 ", // new
    " 📁 ", // open
    " 💾 ", // save
    " ⚙️ ", // tokenize
    " 🔍 ", // parse
    " 🔎 ", // search
    " 🔄 ", // replace

```

```

" 🗑️ ", // clear

" 📤 ", // export

" 📋 ", // tokens

" 🏴‍☠️ ", // symbols

" ⚠️ ", // errors

" 💻 " // console
};

private static final String[] ICON_NAMES = {
    "new", "open", "save", "tokenize", "parse", "search", "replace",
    "clear", "export", "tokens", "symbols", "errors", "console"
};

// Add typo correction map
private static final Map<String, String> TYPO_CORRECTIONS = new HashMap<>();
static {
    // Data types
    TYPO_CORRECTIONS.put("itn", "int");
    TYPO_CORRECTIONS.put("nit", "int");
    TYPO_CORRECTIONS.put("flaot", "float");
    TYPO_CORRECTIONS.put("flot", "float");
    TYPO_CORRECTIONS.put("doble", "double");
    TYPO_CORRECTIONS.put("duble", "double");
    TYPO_CORRECTIONS.put("cahr", "char");
    TYPO_CORRECTIONS.put("chra", "char");
    TYPO_CORRECTIONS.put("strnig", "string");
    TYPO_CORRECTIONS.put("strng", "string");
    // Control flow
    TYPO_CORRECTIONS.put("fi", "if");
    TYPO_CORRECTIONS.put("esle", "else");
    TYPO_CORRECTIONS.put("whiel", "while");
    TYPO_CORRECTIONS.put("whle", "while");
}

```

```

    TYPO_CORRECTIONS.put("rof", "for");
    TYPO_CORRECTIONS.put("retrun", "return");
    TYPO_CORRECTIONS.put("retunr", "return");
    TYPO_CORRECTIONS.put("od", "do");
    // C++ stream typo corrections
    TYPO_CORRECTIONS.put("coutt", "cout");
    TYPO_CORRECTIONS.put("coout", "cout");
    TYPO_CORRECTIONS.put("ednl", "endl");
    TYPO_CORRECTIONS.put("endl", "endl");
}

private JMenuItem parseItem; // Add this field at the top with other fields
private JButton parseBtn;    // Add this field at the top with other fields
private boolean isProgrammaticChange = false;

public LexerGUI() {
    setTitle("W++ Compiler IDE - Final Professional Edition");
    setSize(1200, 800);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLayout(new BorderLayout(0, 0));

    // Initialize icon cache
    iconCache = new HashMap<>();
    loadIcons();

    editors = new HashMap<>();
    filePaths = new HashMap<>();
    highlightTimer = new javax.swing.Timer(300, null);
    highlightTimer.setRepeats(false);

    // Set theme as default
    isDarkMode = false; // Start with light theme

```

```

// Initialize theme colors
keywordColor = isDarkMode ? KEYWORD_COLOR_DARK : KEYWORD_COLOR_LIGHT;
typeColor = isDarkMode ? TYPE_COLOR_DARK : TYPE_COLOR_LIGHT;
stringColor = isDarkMode ? STRING_COLOR_DARK : STRING_COLOR_LIGHT;
commentColor = isDarkMode ? COMMENT_COLOR_DARK : COMMENT_COLOR_LIGHT;
numberColor = isDarkMode ? NUMBER_COLOR_DARK : NUMBER_COLOR_LIGHT;
operatorColor = isDarkMode ? OPERATOR_COLOR_DARK : OPERATOR_COLOR_LIGHT;
preprocessorColor = isDarkMode ? PREPROCESSOR_COLOR_DARK :
PREPROCESSOR_COLOR_LIGHT;

separatorColor = isDarkMode ? new Color(180, 120, 180) : new Color(140, 0, 160);

// Initialize all UI components in the correct order
initializeOutputPanel();
initializeMenuBar();
initializeToolBar();
initializeTabbedPane();
applyTheme();

// Hide Parse button and menu item by default
if (parseBtn != null) parseBtn.setVisible(false);
if (parseItem != null) parseItem.setVisible(false);

// In the constructor, after initializing tabbedPane and outputContainer, replace the add() calls:
// Remove or comment out these lines:
// add(tabbedPane, BorderLayout.CENTER);
// add(outputContainer, BorderLayout.SOUTH);

// Add this instead:
JSplitPane splitPane = new JSplitPane(JSplitPane.VERTICAL_SPLIT, tabbedPane, outputContainer);
splitPane.setResizeWeight(0.8); // 80% editor, 20% output by default
splitPane.setOneTouchExpandable(true);
splitPane.setDividerSize(8);
add(splitPane, BorderLayout.CENTER);

```

```

        setVisible(true);
    }

    private void loadIcons() {
        // Create icons using Unicode symbols
        for (int i = 0; i < ICON_NAMES.length; i++) {
            JLabel iconLabel = new JLabel(ICON_CODES[i]);
            iconLabel.setFont(new Font("Dialog", Font.PLAIN, 20)); // Increased font size
            iconLabel.setForeground(new Color(0, 120, 215));

            // Create a larger image for better visibility
            BufferedImage image = new BufferedImage(24, 24, BufferedImage.TYPE_INT_ARGB);
            Graphics2D g2d = image.createGraphics();
            g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
            g2d.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_ON);

            // Draw the icon
            iconLabel.setBounds(0, 0, 24, 24);
            iconLabel.paint(g2d);
            g2d.dispose();

            iconCache.put(ICON_NAMES[i], new ImageIcon(image));
        }
    }

    private void initializeMenuBar() {
        menuBar = new JMenuBar();

        // Simple menu bar with just File, Run, Help
        JMenu fileMenu = new JMenu("File");
    }

```

```

JMenu runMenu = new JMenu("Run");

JMenu helpMenu = new JMenu("Help");


// File menu items with keyboard shortcuts

JMenuItem newFileItem = new JMenuItem("New File");
newFileItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_N,
InputEvent.CTRL_DOWN_MASK));


JMenuItem openFileItem = new JMenuItem("Open File");
openFileItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_O,
InputEvent.CTRL_DOWN_MASK));


JMenuItem saveFileItem = new JMenuItem("Save File");
saveFileItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_S,
InputEvent.CTRL_DOWN_MASK));


JMenuItem deleteFileItem = new JMenuItem("Delete File");
deleteFileItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_D,
InputEvent.CTRL_DOWN_MASK));


JMenuItem renameFileItem = new JMenuItem("Rename File");
renameFileItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_R,
InputEvent.CTRL_DOWN_MASK));


JMenuItem exitItem = new JMenuItem("Exit");


fileMenu.add(newFileItem);
fileMenu.add(openFileItem);
fileMenu.add(saveFileItem);
fileMenu.add(deleteFileItem);
fileMenu.add(renameFileItem);
fileMenu.addSeparator();
fileMenu.add(exitItem);

```

```

// Run menu items

JMenuItem tokenizeItem = new JMenuItem("Tokenize");
parseItem = new JMenuItem("Parse"); // Store in field
parseItem.setVisible(false); // Hide by default

runMenu.add(tokenizeItem);
runMenu.add(parseItem);

// Help menu items
JMenuItem aboutItem = new JMenuItem("About");
helpMenu.add(aboutItem);

// Add event handlers
newFileItem.addActionListener(e -> createNewTab());
openFileItem.addActionListener(e -> openFile());
saveFileItem.addActionListener(e -> saveFile(false));
deleteFileItem.addActionListener(e -> deleteCurrentFile());
renameFileItem.addActionListener(e -> renameCurrentFile());
exitItem.addActionListener(e -> System.exit(0));

tokenizeItem.addActionListener(e -> {
    runLexer();
    if (parseItem != null) parseItem.setVisible(true); // Show Parse after Tokenize
    if (parseBtn != null) parseBtn.setVisible(true); // Show Parse button in toolbar
});

parseItem.addActionListener(e -> debugCode());

aboutItem.addActionListener(e -> showAboutDialog());

// Add menus to the menu bar
menuBar.add(fileMenu);
menuBar.add(runMenu);

```



```

JMenu editMenu = new JMenu("Edit");

// Add menu items to Edit menu

JMenuItem searchMenuItem = new JMenuItem("Search");

searchMenuItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_H,
InputEvent.CTRL_DOWN_MASK));

searchMenuItem.addActionListener(e -> searchInCode());

JMenuItem replaceMenuItem = new JMenuItem("Replace");

replaceMenuItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_P,
InputEvent.CTRL_DOWN_MASK));

replaceMenuItem.addActionListener(e -> replaceInCode());

JMenuItem clearConsoleMenuItem = new JMenuItem("Clear Console");

clearConsoleMenuItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_L,
InputEvent.CTRL_DOWN_MASK));

clearConsoleMenuItem.addActionListener(e -> clearConsole());

editMenu.add(searchMenuItem);

editMenu.add(replaceMenuItem);

editMenu.add(clearConsoleMenuItem);

menuBar.add(editMenu);

menuBar.add(helpMenu);

setJMenuBar(menuBar);
}

```

```

private void initializeToolBar() {

    JToolBar toolBar = new JToolBar();

    toolBar.setFloatable(false);

    toolBar.setBorderPainted(true);

    toolBar.setBackground(new Color(220, 220, 240));

    toolBar.setBorder(BorderFactory.createMatteBorder(0, 0, 1, 0, new Color(180, 180, 200)));

    // Create toolbar buttons with icons

    JButton newBtn = createStyledButton("New", "new");

    JButton openBtn = createStyledButton("Open", "open");

    JButton saveBtn = createStyledButton("Save", "save");
}

```

```

JButton tokenizeBtn = createStyledButton("Tokenize", "tokenize");
parseBtn = createStyledButton("Parse", "parse"); // Store in field
parseBtn.setVisible(false); // Hide by default
JButton searchBtn = createStyledButton("Search", "search");
JButton replaceBtn = createStyledButton("Replace", "replace");
JButton clearConsoleBtn = createStyledButton("Clear Console", "clear");
JButton exportBtn = createStyledButton("Export to CSV", "export");

// Theme selector (light/dark)
JLabel themeLabel = new JLabel("Theme:");
themeLabel.setBorder(BorderFactory.createEmptyBorder(0, 10, 0, 5));
JComboBox<String> themeCombo = new JComboBox<>(new String[]{"Light", "Dark"});
themeCombo.setSelectedIndex(isDarkMode ? 1 : 0);
themeCombo.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createLineBorder(new Color(150, 150, 180), 1, true),
    BorderFactory.createEmptyBorder(2, 5, 2, 5)
));

// Add event handlers
newBtn.addActionListener(e -> createNewTab());
openBtn.addActionListener(e -> openFile());
saveBtn.addActionListener(e -> saveFile(false));
tokenizeBtn.addActionListener(e -> {
    runLexer();
    if (parseBtn != null) parseBtn.setVisible(true); // Show Parse button after Tokenize
    if (parseItem != null) parseItem.setVisible(true); // Show Parse menu item
});
parseBtn.addActionListener(e -> debugCode());
searchBtn.addActionListener(e -> searchInCode());
replaceBtn.addActionListener(e -> replaceInCode());
clearConsoleBtn.addActionListener(e -> clearConsole());
exportBtn.addActionListener(e -> exportToCSV());
themeCombo.addActionListener(e -> {

```

```

        isDarkMode = themeCombo.getSelectedIndex() == 1;
        applyTheme();
    });

    // Add components to toolbar with proper spacing
    toolBar.add(Box.createHorizontalStrut(5));
    toolBar.add(new Btn);
    toolBar.add(Box.createHorizontalStrut(2));
    toolBar.add(openBtn);
    toolBar.add(Box.createHorizontalStrut(2));
    toolBar.add(saveBtn);
    toolBar.add(new JSeparator(SwingConstants.VERTICAL));
    toolBar.add(Box.createHorizontalStrut(5));
    toolBar.add(tokenizeBtn);
    toolBar.add(Box.createHorizontalStrut(2));
    toolBar.add(parseBtn);
    toolBar.add(Box.createHorizontalStrut(2));
    toolBar.add(searchBtn);
    toolBar.add(Box.createHorizontalStrut(2));
    toolBar.add(replaceBtn);
    toolBar.add(Box.createHorizontalStrut(2));
    toolBar.add(clearConsoleBtn);
    toolBar.add(Box.createHorizontalStrut(2));
    toolBar.add(exportBtn);
    toolBar.add(Box.createHorizontalStrut(2));
    toolBar.add(themeLabel);
    toolBar.add(themeCombo);
    toolBar.add(Box.createHorizontalGlue());

    add(toolBar, BorderLayout.NORTH);
}

// Helper method to create styled buttons with icons

```

```

private JButton createStyledButton(String text, String iconName) {
    JButton button = new JButton(text);
    ImageIcon icon = iconCache.get(iconName);
    if (icon != null) {
        button.setIcon(icon);
        button.setHorizontalTextPosition(SwingConstants.RIGHT);
        button.setIconTextGap(8);
        button.setVerticalTextPosition(SwingConstants.CENTER);
        button.setHorizontalAlignment(SwingConstants.LEFT);
    }
    button.setFocusPainted(false);
    button.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(new Color(150, 150, 180), 1, true),
        BorderFactory.createEmptyBorder(4, 8, 4, 8)
    ));
    button.setBackground(new Color(240, 240, 250));
    button.setForeground(new Color(50, 50, 80));
    button.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 12));

    // Add hover effect
    button.addMouseListener(new MouseAdapter() {
        @Override
        public void mouseEntered(MouseEvent e) {
            button.setBackground(new Color(220, 220, 240));
            button.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createLineBorder(new Color(100, 100, 160), 1, true),
                BorderFactory.createEmptyBorder(4, 8, 4, 8)
            ));
        }

        @Override
        public void mouseExited(MouseEvent e) {
            button.setBackground(new Color(240, 240, 250));
        }
    });
}

```

```

        button.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(150, 150, 180), 1, true),
            BorderFactory.createEmptyBorder(4, 8, 4, 8)
        ));
    }
});

return button;
}

private void initializeTabbedPane() {
    tabbedPane = new JTabbedPane();
    tabbedPane.setBorder(BorderFactory.createEmptyBorder(5, 5, 5, 5));
    tabbedPane.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 12));

    // Add a custom tab renderer for more professional look
    UIManager.put("TabbedPane.selected", new Color(230, 230, 250));
    UIManager.put("TabbedPane.borderHightlightColor", ACCENT_COLOR);

    add(tabbedPane, BorderLayout.CENTER);
    createNewTab();
}

private void initializeOutputPanel() {
    // Initialize text areas first to avoid null references
    tokenTextArea = new JTextArea();
    symbolTextArea = new JTextArea();
    errorArea = new JTextArea();
    JTextArea consoleArea = new JTextArea();

    // Configure text areas
    tokenTextArea.setEditable(false);
    symbolTextArea.setEditable(false);

```

```

errorArea.setEditable(false);
consoleArea.setEditable(false);

// Add monospaced font for better readability
Font monoFont = new Font(Font.MONOSPACED, Font.PLAIN, 13);
tokenTableArea.setFont(monoFont);
symbolTableArea.setFont(monoFont);
errorArea.setFont(monoFont);
consoleArea.setFont(monoFont);

// Create container with border
outputContainer = new JPanel(new BorderLayout(0, 0));
outputContainer.setPreferredSize(new Dimension(getWidth(), 200));
outputContainer.setBorder(BorderFactory.createMatteBorder(1, 0, 0, 0, new Color(180, 180, 200)));

// Create custom styled tabbed pane for output panel
outputPanel = new JTabbedPane(JTabbedPane.TOP, JTabbedPane.SCROLL_TAB_LAYOUT);
outputPanel.setBorder(BorderFactory.createEmptyBorder(5, 5, 0, 5));
outputPanel.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 12));

// Apply custom styling to the tabs
UIManager.put("TabbedPane.tabInsets", new Insets(6, 10, 6, 10));
UIManager.put("TabbedPane.selectedTabPadInsets", new Insets(0, 0, 0, 0));
UIManager.put("TabbedPane.contentBorderInsets", new Insets(1, 1, 1, 1));
UIManager.put("TabbedPane.focus", new Color(0, 0, 0, 0)); // Make focus invisible
UIManager.put("TabbedPane.selected", new Color(235, 235, 250)); // Light background for selected tab
content
UIManager.put("TabbedPane.contentAreaColor", new Color(245, 245, 250)); // Light background for tab
content

// Configure the error panel with column headers
JPanel errorPanel = new JPanel(new BorderLayout());
JPanel headerPanel = new JPanel(new GridLayout(1, 3));

```

```
headerPanel.setBorder(BorderFactory.createMatteBorder(0, 0, 1, 0, new Color(180, 180, 200)));
```

```
TextField lineHeader = new TextField("Line");
```

```
TextField errorTypeHeader = new TextField("Error Type");
```

```
TextField messageHeader = new TextField("Message");
```

```
// Make headers stand out
```

```
Font headerFont = new Font(Font.SANS_SERIF, Font.BOLD, 12);
```

```
lineHeader.setFont(headerFont);
```

```
errorTypeHeader.setFont(headerFont);
```

```
messageHeader.setFont(headerFont);
```

```
lineHeader.setEditable(false);
```

```
errorTypeHeader.setEditable(false);
```

```
messageHeader.setEditable(false);
```

```
lineHeader.setBackground(new Color(220, 220, 220));
```

```
errorTypeHeader.setBackground(new Color(220, 220, 220));
```

```
messageHeader.setBackground(new Color(220, 220, 220));
```

```
lineHeader.setBorder(BorderFactory.createEmptyBorder(3, 5, 3, 5));
```

```
errorTypeHeader.setBorder(BorderFactory.createEmptyBorder(3, 5, 3, 5));
```

```
messageHeader.setBorder(BorderFactory.createEmptyBorder(3, 5, 3, 5));
```

```
headerPanel.add(lineHeader);
```

```
headerPanel.add(errorTypeHeader);
```

```
headerPanel.add(messageHeader);
```

```
errorPanel.add(headerPanel, BorderLayout.NORTH);
```

```
errorPanel.add(new JScrollPane(errorArea), BorderLayout.CENTER);
```

```
// Create styled scroll panes
```

```
JScrollPane tokenScroll = createStyledScrollPane(tokenTableArea);
```

```
JScrollPane symbolScroll = createStyledScrollPane(symbolTableArea);
```

```

JScrollPane consoleScroll = createStyledScrollPane(consoleArea);

// Add custom styled tabs with icons
addStyledTab(outputPanel, "Tokens", tokenScroll, "Token analysis output");
addStyledTab(outputPanel, "  Symbol Table", symbolScroll, "Symbol table contents");
addStyledTab(outputPanel, "  Syntax Errors", errorPanel, "Syntax error details");
addStyledTab(outputPanel, "Console", consoleScroll, "Console output");

// Add status label with border and padding
statusLabel = new JLabel("Lexer Done");
statusLabel.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createMatteBorder(1, 0, 0, 0, new Color(180, 180, 200)),
    BorderFactory.createEmptyBorder(5, 10, 5, 10)
));
statusLabel.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 12));
statusLabel.setOpaque(true);
statusLabel.setBackground(new Color(240, 240, 245));

outputContainer.add(outputPanel, BorderLayout.CENTER);
outputContainer.add(statusLabel, BorderLayout.SOUTH);

add(outputContainer, BorderLayout.SOUTH);
}

// Helper method for adding styled tabs to output panel
private void addStyledTab(JTabbedPane tabbedPane, String title, Component component, String tooltip) {
    int index = tabbedPane.getTabCount();
    tabbedPane.addTab(null, component);

    // Create a custom tab component with border and background
    JPanel tabPanel = new JPanel();
    tabPanel.setLayout(new FlowLayout(FlowLayout.CENTER, 8, 2));
    tabPanel.setOpaque(true);

```



```

tabPanel.setBackground(new Color(240, 240, 250));
tabPanel.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createMatteBorder(1, 1, 0, 1, new Color(190, 190, 210)),
    BorderFactory.createEmptyBorder(4, 8, 4, 8)
));

// Add icon to tab with color
JLabel tabLabel;
if (title.contains("Tokens")) {
    tabLabel = new JLabel("\uD83D\uDCCB"); // 📁 or 🗂️ or 📁 or 📄 or 📁 or 📁 or 📁 or 📁
    tabLabel.setForeground(new Color(0, 120, 215));
} else if (title.contains("Symbol Table")) {
    tabLabel = new JLabel("\uD83D\uDCCA"); // 📊
    tabLabel.setForeground(new Color(0, 120, 215));
} else if (title.contains("Syntax Errors")) {
    tabLabel = new JLabel("\u26A0\uFE0F"); // ⚠️
    tabLabel.setForeground(new Color(255, 140, 0));
} else if (title.contains("Console")) {
    tabLabel = new JLabel("\uD83D\uDCBB"); // 💻
    tabLabel.setForeground(new Color(80, 80, 80));
} else {
    tabLabel = new JLabel("");
}

tabLabel.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 18));
JLabel textLabel = new JLabel(title.replaceAll("^[^ ]+ ", ""));
textLabel.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 12));
textLabel.setForeground(new Color(40, 40, 80));
tabPanel.add(tabLabel);
tabPanel.add(Box.createHorizontalStrut(4));
tabPanel.add(textLabel);
tabbedPane.setTabComponentAt(index, tabPanel);
tabbedPane.setToolTipTextAt(index, tooltip);

```

```

// Set background color of the selected tab
tabbedPane.addChangeListener(e -> {
    int selectedIndex = tabbedPane.getSelectedIndex();
    for (int i = 0; i < tabbedPane.getTabCount(); i++) {
        Component c = tabbedPane.getTabComponentAt(i);
        if (c instanceof JPanel) {
            JPanel panel = (JPanel) c;
            if (i == selectedIndex) {
                panel.setBackground(new Color(220, 225, 255));
                for (Component comp : panel.getComponents()) {
                    if (comp instanceof JLabel && panel.getComponent(0) != comp) {
                        ((JLabel) comp).setForeground(new Color(0, 92, 179));
                    }
                }
            } else {
                panel.setBackground(new Color(240, 240, 250));
                for (Component comp : panel.getComponents()) {
                    if (comp instanceof JLabel && panel.getComponent(0) != comp) {
                        ((JLabel) comp).setForeground(new Color(40, 40, 80));
                    }
                }
            }
        }
    }
});
}

```

```

// Helper method to create styled scroll panes
private JScrollPane createStyledScrollPane(JComponent component) {
    JScrollPane scrollPane = new JScrollPane(component);
    scrollPane.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(new Color(200, 200, 210), 1),

```

```

        BorderFactory.createEmptyBorder(2, 2, 2, 2)
    ));
    return scrollPane;
}

private void clearConsole() {
    // Clear all output panels
    tokenTextArea.setText("");
    symbolTextArea.setText("");
    errorArea.setText("");
    // Also clear the Console tab if it exists
    if (outputPanel.getTabCount() >= 4) {
        JScrollPane scrollPane = (JScrollPane)outputPanel.getComponentAt(3);
        if (scrollPane.getViewPort().getView() instanceof JTextArea) {
            ((JTextArea)scrollPane.getViewPort().getView()).setText("");
        }
    }
    statusLabel.setText("Console cleared");
    logToConsole("[INFO] Console cleared.");
}

private void showAboutDialog() {
    JOptionPane.showMessageDialog(this,
        "W++ Compiler IDE - Final Professional Edition\n" +
        "Version 1.0\n\n" +
        "A comprehensive C-like language compiler\n" +
        "for syntax analysis and error detection.\n\n" +
        "© 2023 Compiler Design Project",
        "About W++ Compiler IDE",
        JOptionPane.INFORMATION_MESSAGE);
}

private void createNewTab() {

```

```

JTextPane editor = new JTextPane();
Font font = new Font(Font.MONOSPACED, Font.PLAIN, fontSize);
editor.setFont(font);
editor.setBorder(BorderFactory.createEmptyBorder(5, 5, 5, 5));
// Add auto-correction functionality
setupAutoCorrection(editor);
// Add auto-completion functionality
setupAutoCompletion(editor);
JScrollPane scrollPane = new JScrollPane(editor);
scrollPane.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createLineBorder(new Color(200, 200, 210), 1),
    BorderFactory.createEmptyBorder(0, 0, 0, 0)
));
LineNumberView lineNumberView = new LineNumberView(editor);
scrollPane.setRowHeaderView(lineNumberView);
int tabIndex = tabbedPane.getTabCount();
// Generate sequential file name: united0.txt, united1.txt, etc.
String fileName = "united" + tabIndex + ".txt";
tabbedPane.addTab(fileName, scrollPane);
tabbedPane.setSelectedIndex(tabIndex);
// Create a tab component with close button
JPanel tabPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
tabPanel.setOpaque(false);
JLabel tabLabel = new JLabel(fileName);
tabLabel.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 12));
// Create a nicer looking close button
JButton closeButton = new JButton("×");
closeButton.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 12));
closeButton.setFocusPainted(false);
closeButton.setBorderPainted(false);
closeButton.setContentAreaFilled(false);
closeButton.setForeground(new Color(100, 100, 120));
closeButton.setBorder(BorderFactory.createEmptyBorder(0, 5, 0, 0));

```

```

closeButton.setPreferredSize(new Dimension(20, 20));
closeButton.addActionListener(e -> closeTab(tabIndex));

// Add hover effect to close button
closeButton.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseEntered(MouseEvent e) {
        closeButton.setForeground(new Color(180, 50, 50));
    }

    @Override
    public void mouseExited(MouseEvent e) {
        closeButton.setForeground(new Color(100, 100, 120));
    }
});

tabPanel.add(tabLabel);
tabPanel.add(closeButton);
tabbedPane.setTabComponentAt(tabIndex, tabPanel);

// Add document listener for syntax highlighting and error detection
editor.getDocument().addDocumentListener(new DocumentListener() {
    @Override
    public void insertUpdate(DocumentEvent e) {
        highlightTimer.restart();
        if (!isProgrammaticChange) {
            if (parseBtn != null) parseBtn.setVisible(false);
            if (parseItem != null) parseItem.setVisible(false);
        }
    }

    @Override
    public void removeUpdate(DocumentEvent e) {
        highlightTimer.restart();
    }
});

```

```

        if (!isProgrammaticChange) {
            if (parseBtn != null) parseBtn.setVisible(false);
            if (parseItem != null) parseItem.setVisible(false);
        }
    }

    @Override
    public void changedUpdate(DocumentEvent e) {
        highlightTimer.restart();
        if (!isProgrammaticChange) {
            if (parseBtn != null) parseBtn.setVisible(false);
            if (parseItem != null) parseItem.setVisible(false);
        }
    }
});

highlightTimer.addActionListener(e -> applySyntaxHighlighting(editor));
// Immediately apply syntax highlighting and error detection after opening
applySyntaxHighlighting(editor);

editors.put(tabIndex, editor);

// Save the file path with the sequential name
try {
    File file = new File(fileName);
    filePaths.put(tabIndex, file.getAbsolutePath());

    // Create the empty file on disk
    if (!file.exists()) {
        try (FileWriter writer = new FileWriter(file)) {
            writer.write("");
        }
        statusLabel.setText("Created new file: " + file.getAbsolutePath());
    }
} catch (IOException ex) {

```

```

        ex.printStackTrace();
        statusLabel.setText("Error creating file: " + ex.getMessage());
    }

    applyTheme();

    // Focus the editor
    editor.requestFocusInWindow();

    // In createNewTab(), after creating the editor and adding the tab:
    tokenTableArea.setText("");
    symbolTableArea.setText("");
    errorArea.setText("");
    logToConsole("[INFO] New file created: " + fileName);
}

private void closeTab(int index) {
    if (index >= 0 && index < tabbedPane.getTabCount()) {
        tabbedPane.remove(index);
        editors.remove(index);
        filePaths.remove(index);

        // Reorganize remaining tabs
        Map<Integer, JTextPane> newEditors = new HashMap<>();
        Map<Integer, String> newPaths = new HashMap<>();

        for (int i = 0; i < tabbedPane.getTabCount(); i++) {
            if (i >= index) {
                JTextPane editor = editors.get(i + 1);
                String path = filePaths.get(i + 1);

                if (editor != null) {
                    newEditors.put(i, editor);

```

```

    }
    if (path != null) {
        newPaths.put(i, path);
    }
} else {
    newEditors.put(i, editors.get(i));
    if (filePaths.containsKey(i)) {
        newPaths.put(i, filePaths.get(i));
    }
}
}

editors = newEditors;
filePaths = newPaths;

// If no tabs left, create a new one
if (tabbedPane.getTabCount() == 0) {
    createNewTab();
}
}
}

private void applyTheme() {
    // Set colors based on the current theme
    Color bgColor = isDarkMode ? DARK_BG : LIGHT_BG;
    Color fgColor = isDarkMode ? DARK_FG : LIGHT_FG;
    // Set theme-specific syntax colors
    keywordColor = isDarkMode ? KEYWORD_COLOR_DARK : KEYWORD_COLOR_LIGHT;
    typeColor = isDarkMode ? TYPE_COLOR_DARK : TYPE_COLOR_LIGHT;
    stringColor = isDarkMode ? STRING_COLOR_DARK : STRING_COLOR_LIGHT;
    commentColor = isDarkMode ? COMMENT_COLOR_DARK : COMMENT_COLOR_LIGHT;
    numberColor = isDarkMode ? NUMBER_COLOR_DARK : NUMBER_COLOR_LIGHT;
    operatorColor = isDarkMode ? OPERATOR_COLOR_DARK : OPERATOR_COLOR_LIGHT;
}

```



```

        preprocessorColor      =      isDarkMode      ?      PREPROCESSOR_COLOR_DARK      :
PREPROCESSOR_COLOR_LIGHT;

        separatorColor = isDarkMode ? new Color(180, 120, 180) : new Color(140, 0, 160);

// Apply theme to editors
if (editors != null) {
    for (JTextPane editor : editors.values()) {
        if (editor != null) {
            editor.setBackground(bgColor);
            editor.setForeground(fgColor);
            editor.setCaretColor(fgColor);
            applySyntaxHighlighting(editor);
        }
    }
}

// Apply theme to output panels
if (tokenTableArea != null) {
    tokenTableArea.setBackground(bgColor);
    tokenTableArea.setForeground(fgColor);
}

if (symbolTableArea != null) {
    symbolTableArea.setBackground(bgColor);
    symbolTableArea.setForeground(fgColor);
}

if (errorArea != null) {
    errorArea.setBackground(bgColor);
    errorArea.setForeground(fgColor);
}

// Apply theme to Console tab
if (outputPanel != null && outputPanel.getTabCount() >= 4) {
    JScrollPane consoleScroll = (JScrollPane) outputPanel.getComponentAt(3);
    if (consoleScroll.getViewPort().getView() instanceof JTextArea) {

```

```

        JTextArea consoleArea = (JTextArea) consoleScroll.getViewport().getView();
        consoleArea.setBackground(bgColor);
        consoleArea.setForeground(fgColor);
    }
}

// Apply theme to other UI components
if (tabbedPane != null) {
    tabbedPane.setBackground(bgColor);
    tabbedPane.setForeground(fgColor);
}

if (outputPanel != null) {
    outputPanel.setBackground(bgColor);
    outputPanel.setForeground(fgColor);
}

if (statusLabel != null) {
    statusLabel.setBackground(isDarkMode ? new Color(40, 40, 45) : new Color(240, 240, 245));
    statusLabel.setForeground(fgColor);
}

revalidate();
repaint();
}

private void applySyntaxHighlighting(JTextPane editor) {
    try {
        isProgrammaticChange = true;
        String text = editor.getText();
        StyledDocument doc = editor.getStyledDocument();
        // Reset foreground and background
        SimpleAttributeSet defaultStyle = new SimpleAttributeSet();

```

```

StyleConstants.setForeground(defaultStyle, isDarkMode ? DARK_FG : LIGHT_FG);
StyleConstants.setBackground(defaultStyle, isDarkMode ? DARK_BG : LIGHT_BG);
doc.setCharacterAttributes(0, text.length(), defaultStyle, true);
// First highlight comments to prevent them from being tokenized
highlightComments(text, doc);
// Then highlight preprocessor directives
highlightPreprocessor(text, doc);
// Now tokenize and highlight other elements
java.util.List<Lexer.TokenInfo> tokens = new ArrayList<>();
java.util.List<Lexer.LexicalError> lexicalErrors = new ArrayList<>();
Lexer.tokenizeWithErrors(text, tokens, lexicalErrors);
// Perform syntax analysis
java.util.List<Lexer.SyntaxError> syntaxErrors = new ArrayList<>();
Lexer.analyzeSyntax(tokens, syntaxErrors);
// Highlight tokens
List<String> tokenStrings = new ArrayList<>();
for (Lexer.TokenInfo ti : tokens) tokenStrings.add(ti.token);
highlightTokens(text, tokenStrings, doc);
// Highlight lexical errors with red underline
for (Lexer.LexicalError error : lexicalErrors) {
    highlightErrorToken(doc, text, error.token, error.line, error.column, ERROR_COLOR);
}
// Update error area with detailed error messages
StringBuilder errorOutput = new StringBuilder();
if (!lexicalErrors.isEmpty() || !syntaxErrors.isEmpty()) {
    errorOutput.append("W++ COMPILATION ERRORS\n");
    errorOutput.append("=====\n");

    if (!lexicalErrors.isEmpty()) {
        errorOutput.append("LEXICAL ERRORS:\n");
        errorOutput.append("-----\n");
        for (Lexer.LexicalError error : lexicalErrors) {
            errorOutput.append(error.toString()).append("\n");
        }
    }
}

```

```

    }
    errorOutput.append("\n");
}

if (!syntaxErrors.isEmpty()) {
    errorOutput.append("SYNTAX ERRORS:\n");
    errorOutput.append("-----\n");
    for (Lexer.SyntaxError error : syntaxErrors) {
        errorOutput.append(error.toString()).append("\n");
    }
}

errorOutput.append("-----\n");
errorOutput.append("Total Errors: ").append(lexicalErrors.size() + syntaxErrors.size()).append("\n");
errorArea.setText(errorOutput.toString());
} else {
    errorArea.setText("No errors found in W++ code.\n\nCompilation successful!");
}

isProgrammaticChange = false;
} catch (Exception e) {
    e.printStackTrace();
    statusLabel.setText("Error in syntax highlighting");
}
}

private void highlightTokens(String text, List<String> tokens, StyledDocument doc) {
    int position = 0;
    boolean inComment = false;
    boolean inMultiLineComment = false;
    boolean inString = false;
    boolean inChar = false;
    String lastType = null;
    String lastIdentifier = null;

```

```

boolean inDeclaration = false;
boolean inIfBlock = false;
boolean inElseBlock = false;
boolean inMainFunction = false;

for (int i = 0; i < tokens.size(); i++) {
    String token = tokens.get(i);
    int tokenStart = text.indexOf(token, position);

    if (tokenStart < 0) {
        position = 0;
        continue;
    }

    SimpleAttributeSet style = new SimpleAttributeSet();
    Lexer.TokenType type = Lexer.getTokenType(token);

    // Special color for cout/endl
    if (token.equals("cout") || token.equals("endl")) {
        StyleConstants.setForeground(style, isDarkMode ? COUT_ENDL_COLOR_DARK :
COUT_ENDL_COLOR_LIGHT);
        StyleConstants.setBold(style, true);
    }
    // Handle comments
    else if (token.equals("//")) {
        inComment = true;
        StyleConstants.setForeground(style, commentColor);
        StyleConstants.setItalic(style, true);
    } else if (token.equals("/*")) {
        inMultiLineComment = true;
        StyleConstants.setForeground(style, commentColor);
        StyleConstants.setItalic(style, true);
    } else if (token.equals("*/")) {

```

```

    inMultiLineComment = false;
    StyleConstants.setForeground(style, commentColor);
    StyleConstants.setItalic(style, true);
} else if (inComment || inMultiLineComment) {
    StyleConstants.setForeground(style, commentColor);
    StyleConstants.setItalic(style, true);
    if (token.equals("\n")) inComment = false;
}
// Handle string and char literals
else if (token.startsWith("\'")) {
    inString = !inString;
    StyleConstants.setForeground(style, stringColor);
} else if (token.startsWith("\"")) {
    inChar = !inChar;
    StyleConstants.setForeground(style, stringColor);
} else if (inString || inChar) {
    StyleConstants.setForeground(style, stringColor);
}
// Handle main function
else if (token.equals("main") && i > 0 && tokens.get(i-1).equals("int")) {
    inMainFunction = true;
    StyleConstants.setForeground(style, keywordColor);
    StyleConstants.setBold(style, true);
}
// Handle variable declarations and types
else if (isDataType(token)) {
    lastType = token;
    inDeclaration = true;
    StyleConstants.setForeground(style, typeColor);
    StyleConstants.setBold(style, true);
}
// Handle identifiers (variables)
else if (type == Lexer.TokenType.IDENTIFIER) {

```

```

    if (inDeclaration) {
        lastIdentifier = token;

        StyleConstants.setForeground(style, isDarkMode ? DARK_FG : LIGHT_FG);
    } else {
        StyleConstants.setForeground(style, isDarkMode ? DARK_FG : LIGHT_FG);
    }
}

// Handle control flow keywords
else if (isControlFlow(token)) {
    StyleConstants.setForeground(style, keywordColor);
    StyleConstants.setBold(style, true);
    if (token.equals("if")) inIfBlock = true;
    else if (token.equals("else")) inElseBlock = true;
}

// Handle operators
else if (type == Lexer.TokenType.OPERATOR) {
    StyleConstants.setForeground(style, operatorColor);
    if (isArithmeticOperator(token)) {
        StyleConstants.setBold(style, true);
    }
}

// Handle separators
else if (type == Lexer.TokenType.SEPARATOR) {
    StyleConstants.setForeground(style, separatorColor);
}

// Handle literals and constants
else if (type == Lexer.TokenType.LITERAL || type == Lexer.TokenType.CONSTANT) {
    StyleConstants.setForeground(style, numberColor);
}

// Apply the style
doc.setCharacterAttributes(tokenStart, token.length(), style, false);
position = tokenStart + token.length();

```

```

    }
}

private boolean isDataType(String token) {
    return token.equals("int") || token.equals("float") || token.equals("double") ||
        token.equals("char") || token.equals("void") || token.equals("string") ||
        token.equals("bool") || token.equals("long") || token.equals("short");
}

private boolean isControlFlow(String token) {
    return token.equals("if") || token.equals("else") || token.equals("while") ||
        token.equals("for") || token.equals("do") || token.equals("return") ||
        token.equals("break") || token.equals("continue") || token.equals("switch") ||
        token.equals("case") || token.equals("default");
}

private boolean isArithmeticOperator(String token) {
    return token.equals("+") || token.equals("-") || token.equals("*") ||
        token.equals("/") || token.equals("++") || token.equals("--") ||
        token.equals("+=") || token.equals("-=") || token.equals("*=") ||
        token.equals("/=") || token.equals("%=") || token.equals("%");
}

private boolean isStringLiteral(String token) {
    return token.startsWith("\"") && token.endsWith("\"");
}

private boolean isCharLiteral(String token) {
    return token.startsWith("'") && token.endsWith("'");
}

private void highlightComments(String text, StyledDocument doc) {
    try {

```



```

SimpleAttributeSet commentStyle = new SimpleAttributeSet();
StyleConstants.setForeground(commentStyle, commentColor);
    StyleConstants.setItalic(commentStyle, true);

    // Handle single-line comments
    Pattern singleLineComment = Pattern.compile("//.*");
    Matcher matcher = singleLineComment.matcher(text);
    while (matcher.find()) {
        doc.setCharacterAttributes(matcher.start(), matcher.end() - matcher.start(), commentStyle, false);
    }

    // Handle multi-line comments
    Pattern multiLineComment = Pattern.compile("/\\*.?*\\*/", Pattern.DOTALL);
    matcher = multiLineComment.matcher(text);
    while (matcher.find()) {
        doc.setCharacterAttributes(matcher.start(), matcher.end() - matcher.start(), commentStyle, false);
    }
} catch (Exception e) {
    e.printStackTrace();
}
}

private void highlightPreprocessor(String text, StyledDocument doc) {
    try {
        SimpleAttributeSet preprocessorStyle = new SimpleAttributeSet();
        StyleConstants.setForeground(preprocessorStyle, preprocessorColor);
        StyleConstants.setBold(preprocessorStyle, true);

        Pattern preprocessor = Pattern.compile("^\\s*#.*$", Pattern.MULTILINE);
        Matcher matcher = preprocessor.matcher(text);
        while (matcher.find()) {
            doc.setCharacterAttributes(matcher.start(), matcher.end() - matcher.start(), preprocessorStyle, false);
        }
    }
}

```

```

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

private void highlightError(String text, String token, int line, int column, StyledDocument doc, Color color) {
    try {
        // Calculate position of error token in document
        String[] lines = text.split("\n", -1);
        int position = 0;

        // Add up lengths of lines before the error line
        for (int i = 0; i < line - 1 && i < lines.length; i++) {
            position += lines[i].length() + 1; // +1 for newline
        }

        // Add column offset (adjusted for 1-based indexing)
        position += column - 1;

        // Don't exceed document length
        if (position >= 0 && position < text.length()) {
            // Find actual token in the text
            int start = Math.max(position - token.length(), 0);
            int end = Math.min(position + token.length(), text.length());
            String contextText = text.substring(start, end);

            int tokenStart = contextText.indexOf(token);
            if (tokenStart >= 0) {
                tokenStart += start; // Adjust for the substring offset

                SimpleAttributeSet errorStyle = new SimpleAttributeSet();
                StyleConstants.setForeground(errorStyle, color);
                StyleConstants.setUnderline(errorStyle, true);
            }
        }
    }
}

```

```

        doc.setCharacterAttributes(tokenStart, token.length(), errorStyle, false);
    }
}
} catch (Exception e) {
    e.printStackTrace();
}
}

private void runLexer() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }
    JTextPane editor = editors.get(selectedTab);
    String code = editor.getText();
    java.util.List<Lexer.TokenInfo> tokens = new ArrayList<>();
    java.util.List<Lexer.LexicalError> lexicalErrors = new ArrayList<>();
    Lexer.tokenizeWithErrors(code, tokens, lexicalErrors);
    Map<String, String[]> symbolTable = new HashMap<>();
    Lexer.processTokensForSymbolTable(tokens, symbolTable);
    Map<Lexer.TokenType, Map<String, Integer>> tokenCounts = new HashMap<>();
    for (Lexer.TokenInfo ti : tokens) {
        String token = ti.token;
        Lexer.TokenType type = Lexer.getTokenType(token);
        tokenCounts.computeIfAbsent(type, k -> new HashMap<>()).merge(token, 1, Integer::sum);
    }
    // Check for syntax errors
    java.util.List<Lexer.SyntaxError> syntaxErrors = new ArrayList<>();
    Lexer.analyzeSyntax(tokens, syntaxErrors);
    // Display token table
    isProgrammaticChange = true;
    StringBuilder tokenOutput = new StringBuilder();

```

```

tokenOutput.append("W++ LANGUAGE TOKEN ANALYSIS\n");

tokenOutput.append("=====  

=====\\n");

    tokenOutput.append(String.format("%-15s %-20s %-10s %s\\n", "TOKEN TYPE", "LEXEME", "COUNT",  

"DESCRIPTION"));

    tokenOutput.append("-----  

-----\\n");

    if (tokenCounts.containsKey(Lexer.TokenType.KEYWORD)) {
        for (Map.Entry<String, Integer> entry : tokenCounts.get(Lexer.TokenType.KEYWORD).entrySet()) {
            String description = getTokenDescription(entry.getKey(), Lexer.TokenType.KEYWORD);
            tokenOutput.append(String.format("%-15s %-20s %-10d %s\\n", "KEYWORD", entry.getKey(),  

entry.getValue(), description));
        }
    }

    if (tokenCounts.containsKey(Lexer.TokenType.IDENTIFIER)) {
        for (Map.Entry<String, Integer> entry : tokenCounts.get(Lexer.TokenType.IDENTIFIER).entrySet()) {
            String description = getTokenDescription(entry.getKey(), Lexer.TokenType.IDENTIFIER);
            tokenOutput.append(String.format("%-15s %-20s %-10d %s\\n", "IDENTIFIER", entry.getKey(),  

entry.getValue(), description));
        }
    }

    if (tokenCounts.containsKey(Lexer.TokenType.OPERATOR)) {
        for (Map.Entry<String, Integer> entry : tokenCounts.get(Lexer.TokenType.OPERATOR).entrySet()) {
            String description = getTokenDescription(entry.getKey(), Lexer.TokenType.OPERATOR);
            tokenOutput.append(String.format("%-15s %-20s %-10d %s\\n", "OPERATOR", entry.getKey(),  

entry.getValue(), description));
        }
    }

    for (Lexer.TokenType type : new Lexer.TokenType[] {Lexer.TokenType.CONSTANT,  

Lexer.TokenType.LITERAL}) {
        if (tokenCounts.containsKey(type)) {
            for (Map.Entry<String, Integer> entry : tokenCounts.get(type).entrySet()) {
                String description = getTokenDescription(entry.getKey(), type);
                tokenOutput.append(String.format("%-15s %-20s %-10d %s\\n", type.toString(), entry.getKey(),  

entry.getValue(), description));
            }
        }
    }

```

```

    }
}
}
if (tokenCounts.containsKey(Lexer.TokenType.SEPARATOR)) {
    for (Map.Entry<String, Integer> entry : tokenCounts.get(Lexer.TokenType.SEPARATOR).entrySet()) {
        String description = getTokenDescription(entry.getKey(), Lexer.TokenType.SEPARATOR);
        tokenOutput.append(String.format("%-15s %-20s %-10d %s\n", "SEPARATOR", entry.getKey(),
entry.getValue(), description));
    }
}
tokenOutput.append("-----\n");
tokenOutput.append("Total Tokens: ").append(tokens.size()).append("\n");
tokenTableArea.setText(tokenOutput.toString());
// Display symbol table
StringBuilder symbolOutput = new StringBuilder();
symbolOutput.append("W++ LANGUAGE SYMBOL TABLE\n");

symbolOutput.append("=====
=====
\n");

symbolOutput.append("Name      Type      Size   Dimension   Line of Declaration   Line of Usage
Address \n");

symbolOutput.append("-----
-----\n");

for (Map.Entry<String, String[]> entry : symbolTable.entrySet()) {
    String name = entry.getKey();
    String[] info = entry.getValue();
    String type = info[0];
    String value = info[1].equals("Not initialized") ? "Uninitialized" : info[1];
    int size = Lexer.TYPE_SIZES.getOrDefault(type, 0);
    String line = info[2];
    String lineOfUsage = line; // Placeholder, same as declaration
    String address = "0x" + Integer.toHexString(name.hashCode() & 0xFFFF);

    symbolOutput.append(String.format("%-15s %-15s %-8d %-12d %-18s %-15s %-10s\n", name, type,
size, 0, line, lineOfUsage, address));
}

```

```

symbolTableArea.setText(symbolOutput.toString());
statusLabel.setText("Lexical analysis complete");
logToConsole("[INFO] Tokenization complete. " + tokens.size() + " tokens found.");
if (!lexicalErrors.isEmpty()) logToConsole("[WARNING] " + lexicalErrors.size() + " lexical errors
detected.");

// Update sessionTokens/sessionLexicalErrors/sessionSyntaxErrors:
sessionTokens = tokens.size();
sessionLexicalErrors = lexicalErrors.size();
isProgrammaticChange = false;
}

private void debugCode() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }
    JTextPane editor = editors.get(selectedTab);
    String code = editor.getText();
    tokenTableArea.setText("");
    symbolTableArea.setText("");
    // If the code is empty, clear the error area and return
    if (code.trim().isEmpty()) {
        errorArea.setText("");
        statusLabel.setText("No code to analyze");
        return;
    }
    // Check for duplicate main functions
    int mainCount = 0;
    Pattern mainPattern = Pattern.compile("\\bmain\\s*\\(");
    Matcher mainMatcher = mainPattern.matcher(code);
    while (mainMatcher.find()) {

```

```

    mainCount++;
}
if (mainCount > 1) {
    errorArea.setText("W++ COMPILATION ERRORS\n" +
        "=====\n\n" +
        "SYNTAX ERRORS:\n" +
        "-----\n" +
        "Multiple main functions detected. Only one main function is allowed.\n" +
        "Please remove the duplicate main functions.\n" +
        "-----\n" +
        "Total Errors: 1\n");
    outputPanel.setSelectedIndex(2); // Switch to Errors tab
    statusLabel.setText("Syntax analysis complete - Multiple main functions detected");
    return;
}

```

```

StyledDocument doc = editor.getStyledDocument();
// 1. Reset all background colors first
SimpleAttributeSet resetStyle = new SimpleAttributeSet();
StyleConstants.setBackground(resetStyle, isDarkMode ? DARK_BG : LIGHT_BG);
doc.setCharacterAttributes(0, code.length(), resetStyle, true);
java.util.List<Lexer.TokenInfo> tokens = new ArrayList<>();
java.util.List<Lexer.LexicalError> lexicalErrors = new ArrayList<>();
Lexer.tokenizeWithErrors(code, tokens, lexicalErrors);
java.util.List<Lexer.SyntaxError> syntaxErrors = new ArrayList<>();
Lexer.analyzeSyntax(tokens, syntaxErrors);
// 2. Highlight error lines for both lexical and syntax errors
String[] lines = code.split("\n", -1);
Color errorLineColor = isDarkMode ? new Color(60, 0, 0, 120) : new Color(255, 200, 200, 180);
for (Lexer.LexicalError error : lexicalErrors) {
    highlightErrorLine(doc, lines, error.line, errorLineColor);
}
for (Lexer.SyntaxError error : syntaxErrors) {

```

```

    highlightErrorLine(doc, lines, error.line, errorLineColor);
}

// Show errors in errorArea with better formatting
StringBuilder errorOutput = new StringBuilder();
if (!lexicalErrors.isEmpty() || !syntaxErrors.isEmpty()) {
    errorOutput.append("W++ COMPILATION ERRORS\n");
    errorOutput.append("=====\n");

    if (!lexicalErrors.isEmpty()) {
        errorOutput.append("LEXICAL ERRORS:\n");
        errorOutput.append("-----\n");
        for (Lexer.LexicalError error : lexicalErrors) {
            errorOutput.append(String.format("Line %d: %s\n", error.line, error.message));
        }
        errorOutput.append("\n");
    }

    if (!syntaxErrors.isEmpty()) {
        errorOutput.append("SYNTAX ERRORS:\n");
        errorOutput.append("-----\n");
        for (Lexer.SyntaxError error : syntaxErrors) {
            // Format the error message to be more readable
            String formattedMessage = error.message;
            if (formattedMessage.contains("Variable") && formattedMessage.contains("not declared")) {
                // Make undeclared variable errors more prominent
                formattedMessage = " ⚠ " + formattedMessage + " - Please declare the variable before using it";
            } else if (formattedMessage.contains("Missing semicolon")) {
                formattedMessage = " ⚠ " + formattedMessage + " - Add a semicolon (;) at the end of the statement";
            } else if (formattedMessage.contains("already initialized")) {
                // Make re-initialization errors more prominent
                formattedMessage = " ⚠ " + formattedMessage + " - A variable can only be initialized once";
            } else if (formattedMessage.contains("char can only hold a single character")) {

```



```

        // Make char initialization errors more prominent

        formattedMessage = " ⚠️ " + formattedMessage + " - Example: char c = 'a'; (single character
only)";

    } else if (formattedMessage.contains("Invalid consecutive keywords")) {

        // Make consecutive keyword errors more prominent

        formattedMessage = " ⚠️ " + formattedMessage + " - Only 'int main' is allowed as consecutive
keywords in C++ function declaration.";

    }

    errorOutput.append(String.format("Line %d: %s\n", error.line, formattedMessage));

}

}

errorOutput.append("-----\n");
errorOutput.append("Total Errors: ").append(lexicalErrors.size() + syntaxErrors.size()).append("\n");
errorArea.setText(errorOutput.toString());

// Stay on the Syntax Errors tab if there are errors
outputPanel.setSelectedIndex(2);
} else {

    errorArea.setText("No errors found in W++ code.\n\nCompilation successful!");

}

// Ensure the Syntax Errors tab is shown if any error is present in the error area
if (errorArea.getText().contains("ERROR") || errorArea.getText().contains("error") ||
errorArea.getText().contains("Unknown or invalid keyword")) {

    outputPanel.setSelectedIndex(2);

}

statusLabel.setText("Syntax analysis complete");
logToConsole("[INFO] Syntax analysis complete. " + syntaxErrors.size() + " syntax errors detected.");
if (!syntaxErrors.isEmpty()) logToConsole("[WARNING] Syntax errors present.");
if (!lexicalErrors.isEmpty()) logToConsole("[WARNING] Lexical errors present.");
if (syntaxErrors.isEmpty() && lexicalErrors.isEmpty()) logToConsole("[INFO] No errors found.");

// Update sessionSyntaxErrors:
sessionSyntaxErrors = syntaxErrors.size();

}

```

```

private int getLineNumber(String code, int tokenIndex) {
    String[] lines = code.split("\n");
    int currentPos = 0;
    for (int i = 0; i < lines.length; i++) {
        currentPos += lines[i].length() + 1; // +1 for newline
        if (currentPos > tokenIndex) {
            return i + 1;
        }
    }
    return 1;
}

```

```

private void highlightErrorToken(StyledDocument doc, String text, String token, int line, int column, Color
color) {
    try {
        String[] lines = text.split("\n", -1);
        int start = 0;
        for (int i = 0; i < line - 1; i++) {
            start += lines[i].length() + 1; // +1 for newline
        }
        start += column - 1;

        SimpleAttributeSet errorStyle = new SimpleAttributeSet();
        StyleConstants.setForeground(errorStyle, color);
        StyleConstants.setUnderline(errorStyle, true);
        doc.setCharacterAttributes(start, token.length(), errorStyle, false);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

private void highlightErrorLine(StyledDocument doc, String[] lines, int line, Color color) {
    try {

```

```

    if (line < 1 || line > lines.length) return; // Prevent out-of-bounds
    int start = 0;
    for (int i = 0; i < line - 1; i++) {
        start += lines[i].length() + 1; // +1 for newline
    }
    int length = lines[line - 1].length();
    if (length == 0) return; // Don't highlight empty lines
    SimpleAttributeSet errorStyle = new SimpleAttributeSet();
    StyleConstants.setBackground(errorStyle, color);
    doc.setCharacterAttributes(start, length, errorStyle, false);
} catch (Exception e) {
    e.printStackTrace();
}
}

private void createErrorNavigation(String errorText, List<Lexer.LexicalError> lexicalErrors,
                                   List<Lexer.SyntaxError> syntaxErrors, JTextPane editor) {
    // Create a new text area with clickable error links
    JTextArea errorNavArea = new JTextArea();
    errorNavArea.setEditable(false);
    errorNavArea.setText(errorText);
    errorNavArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, fontSize));
    errorNavArea.setBackground(isDarkMode ? DARK_BG : LIGHT_BG);
    errorNavArea.setForeground(isDarkMode ? DARK_FG : LIGHT_FG);

    // Add click handler for error navigation
    errorNavArea.addMouseListener(new MouseAdapter() {
        @Override
        public void mouseClicked(MouseEvent e) {
            try {
                // Use reflection to handle both old and new API
                int clickPos = getTextPositionAt(errorNavArea, e.getPoint());
                int lineStart = errorNavArea.getText().lastIndexOf('\n', clickPos);

```

```

int lineEnd = errorNavArea.getText().indexOf('\n', clickPos);

if (lineEnd == -1) lineEnd = errorNavArea.getText().length();
if (lineStart == -1) lineStart = 0;
else lineStart++; // Skip the '\n'

String clickedLine = errorNavArea.getText().substring(lineStart, lineEnd);

// Extract line number from error message
if (clickedLine.contains("line")) {
    int lineIndex = clickedLine.indexOf("line") + 5;
    int commaIndex = clickedLine.indexOf(',', lineIndex);
    if (commaIndex == -1) commaIndex = clickedLine.indexOf(':', lineIndex);
    if (commaIndex != -1) {
        String lineNumber = clickedLine.substring(lineIndex, commaIndex).trim();
        try {
            int errorLine = Integer.parseInt(lineNumber);
            navigateToLine(editor, errorLine);
        } catch (NumberFormatException ex) {
            // Ignore if we can't parse the line number
        }
    }
}
} catch (Exception ex) {
    ex.printStackTrace();
}
});

// Update the error area with the new component
errorArea.setText("");
errorArea = errorNavArea;

// Replace the component in the scroll pane

```

```

JScrollPane scrollPane = (JScrollPane) outputPanel.getComponentAt(2);
scrollPane.setViewportView(errorNavArea);
}

// Helper method to get text position at a point that works across Java versions
@SuppressWarnings("deprecation")
private int getTextPositionAt(JTextComponent textComponent, Point point) {
    try {
        // Try using the newer API first (Java 9+)
        try {
            // Use reflection to avoid compile errors on older Java versions
            java.lang.reflect.Method method = JTextComponent.class.getMethod("viewToModelPosition",
Point.class);

            Object position = method.invoke(textComponent, point);
            // Get the offset from the Position object
            return (int) position.getClass().getMethod("getOffset").invoke(position);
        } catch (NoSuchMethodException e) {
            // Fall back to the deprecated method for older Java versions
            return textComponent.viewToModel(point);
        }
    } catch (Exception e) {
        // Last resort fallback
        return 0;
    }
}

private void navigateToLine(JTextPane editor, int lineNumber) {
    try {
        String text = editor.getText();
        String[] lines = text.split("\n", -1);

        int position = 0;
        for (int i = 0; i < lineNumber - 1 && i < lines.length; i++) {

```

```

        position += lines[i].length() + 1; // +1 for newline
    }

    editor.setCaretPosition(position);
    editor.requestFocus();

    // Highlight the line
    int lineEnd = position + (lineNumber <= lines.length ? lines[lineNumber - 1].length() : 0);
    editor.select(position, lineEnd);
} catch (Exception e) {
    e.printStackTrace();
}
}

private void saveFile(boolean saveAs) {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }

    // Always prompt for a file path when saving
    JFileChooser fileChooser = new JFileChooser();
    String filePath = filePaths.get(selectedTab);
    String defaultName;
    if (filePath != null) {
        defaultName = new File(filePath).getName();
    } else {
        defaultName = "file" + (selectedTab + 1) + ".txt";
    }
    fileChooser.setSelectedFile(new File(defaultName));
    fileChooser.setFileFilter(new javax.swing.filechooser.FileNameExtensionFilter("Text Files", "txt"));
    if (fileChooser.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {

```

```

filePath = fileChooser.getSelectedFile().getPath();
// Ensure .txt extension
if (!filePath.toLowerCase().endsWith(".txt")) {
    filePath += ".txt";
}
filePaths.put(selectedTab, filePath);
// Update tab title
tabbedPane.setTitleAt(selectedTab, new File(filePath).getName());
// Update tab component
Component tabComp = tabbedPane.getTabComponentAt(selectedTab);
if (tabComp instanceof JPanel) {
    Component[] components = ((JPanel)tabComp).getComponents();
    for (Component comp : components) {
        if (comp instanceof JLabel) {
            ((JLabel)comp).setText(new File(filePath).getName());
            break;
        }
    }
} else {
    tabbedPane.setTabComponentAt(selectedTab, createTabComponent(new File(filePath).getName()));
}
} else {
    statusLabel.setText("Save cancelled");
    return;
}

try (FileWriter writer = new FileWriter(filePath)) {
    writer.write(editors.get(selectedTab).getText());
    statusLabel.setText("File saved: " + filePath);
} catch (IOException e) {
    JOptionPane.showMessageDialog(this, "Error saving file: " + e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
    statusLabel.setText("Error saving file");
}

```

```

    }

    logToConsole("[INFO] File saved: " + filePath);
}

private void openFile() {
    JFileChooser fileChooser = new JFileChooser();

    if (fileChooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
        try (BufferedReader reader = new BufferedReader(new FileReader(fileChooser.getSelectedFile()))) {
            JTextPane editor = new JTextPane();
            Font font = new Font("JetBrains Mono", Font.PLAIN, fontSize);
            if (font.getFamily().equals("Dialog")) {
                font = new Font("Consolas", Font.PLAIN, fontSize);
            }
            editor.setFont(font);

            StringBuilder content = new StringBuilder();
            String line;
            while ((line = reader.readLine()) != null) {
                content.append(line).append("\n");
            }
            editor.setText(content.toString());

            // Add auto-correction and auto-completion
            setupAutoCorrection(editor);
            setupAutoCompletion(editor);

            JScrollPane scrollPane = new JScrollPane(editor);
            scrollPane.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createLineBorder(new Color(200, 200, 210), 1),
                BorderFactory.createEmptyBorder(0, 0, 0, 0)
            ));

            LineNumberView lineNumberView = new LineNumberView(editor);
            scrollPane.setRowHeaderView(lineNumberView);

            int tabIndex = tabbedPane.getTabCount();
            tabbedPane.addTab(fileChooser.getSelectedFile().getName(), scrollPane);
        }
    }
}

```



```

        tabbedPane.setTabComponentAt(tabIndex,
createTabComponent(fileChooser.getSelectedFile().getName()));

        editors.put(tabIndex, editor);

        filePaths.put(tabIndex, fileChooser.getSelectedFile().getPath());


        tokenTableArea.setText("");
        symbolTableArea.setText("");
        errorArea.setText("");
        outputPanel.setSelectedIndex(0); // Select Tokens tab by default


        applySyntaxHighlighting(editor);
        applyTheme();

        statusLabel.setText("File opened: " + fileChooser.getSelectedFile().getPath());
        logToConsole("[INFO] File opened: " + fileChooser.getSelectedFile().getName());
    } catch (IOException e) {
        JOptionPane.showMessageDialog(this, "Error opening file: " + e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
        statusLabel.setText("Error opening file");
    }
}

private void renameCurrentFile() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }

    String newName = JOptionPane.showInputDialog(this, "Enter new file name:");
    if (newName != null && !newName.trim().isEmpty()) {
        String oldPath = filePaths.get(selectedTab);
        if (oldPath != null) {

```

```

File oldFile = new File(oldPath);
File newFile = new File(oldFile.getParent(), newName);
if (oldFile.renameTo(newFile)) {
    filePaths.put(selectedTab, newFile.getPath());
    tabbedPane.setTitleAt(selectedTab, newName);
    tabbedPane.setTabComponentAt(selectedTab, createTabComponent(newName));
    statusLabel.setText("File renamed to: " + newName);
} else {
    JOptionPane.showMessageDialog(this, "Failed to rename file on disk", "Error",
JOptionPane.ERROR_MESSAGE);
    statusLabel.setText("Error renaming file");
}
} else {
    tabbedPane.setTitleAt(selectedTab, newName);
    tabbedPane.setTabComponentAt(selectedTab, createTabComponent(newName));
    statusLabel.setText("Tab renamed to: " + newName);
}
}
}

```

```

private void deleteCurrentFile() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }
}

```

```

String filePath = filePaths.get(selectedTab);
if (filePath != null) {
    File file = new File(filePath);
    if (file.exists() && file.delete()) {
        tabbedPane.remove(selectedTab);
        editors.remove(selectedTab);
    }
}

```

```

        filePaths.remove(selectedTab);

        statusLabel.setText("File deleted: " + filePath);

    } else {

        JOptionPane.showMessageDialog(this, "Failed to delete file from disk", "Error",
JOptionPane.ERROR_MESSAGE);

        statusLabel.setText("Error deleting file");

    }

    } else {

        tabbedPane.remove(selectedTab);

        editors.remove(selectedTab);

        statusLabel.setText("Tab closed");

    }

}

```

```

private void showThemeOptions() {

    String[] options = {"Light Mode", "Dark Mode"};

    int choice = JOptionPane.showOptionDialog(this, "Select Theme:", "Theme Selection",

        JOptionPane.YES_NO_OPTION,      JOptionPane.QUESTION_MESSAGE,      null,      options,
options[isDarkMode ? 1 : 0]);

    if (choice == 0 && isDarkMode) {

        isDarkMode = false;

        applyTheme();

        statusLabel.setText("Switched to Light Mode");

    } else if (choice == 1 && !isDarkMode) {

        isDarkMode = true;

        applyTheme();

        statusLabel.setText("Switched to Dark Mode");

    }

}

```

```

private void changeFontSize() {

    JSlider fontSlider = new JSlider(8, 24, fontSize);

```

```

fontSlider.setMajorTickSpacing(4);
fontSlider.setMinorTickSpacing(1);
fontSlider.setPaintTicks(true);
fontSlider.setPaintLabels(true);

int result = JOptionPane.showConfirmDialog(this, fontSlider, "Select Font Size",
JOptionPane.OK_CANCEL_OPTION);

if (result == JOptionPane.OK_OPTION) {
    fontSize = fontSlider.getValue();
    Font font = new Font("JetBrains Mono", Font.PLAIN, fontSize);
    if (font.getFamily().equals("Dialog")) {
        font = new Font("Consolas", Font.PLAIN, fontSize);
    }
    for (JTextPane editor : editors.values()) {
        editor.setFont(font);
        applySyntaxHighlighting(editor);
    }
    statusLabel.setText("Font size changed to: " + fontSize);
}
}

```

// Line number view for editor

```

class LineNumberView extends JTextArea {
    private final JTextPane editor;

    public LineNumberView(JTextPane editor) {
        this.editor = editor;
        setEditable(false);
        setBackground(isDarkMode ? Color.DARK_GRAY : Color.LIGHT_GRAY);
        setForeground(isDarkMode ? Color.LIGHT_GRAY : Color.DARK_GRAY);
        setFont(editor.getFont());
        editor.getDocument().addDocumentListener(new DocumentListener() {
            public void insertUpdate(DocumentEvent e) { refresh(); }
            public void removeUpdate(DocumentEvent e) { refresh(); }
        });
    }
}

```

```

        public void changedUpdate(DocumentEvent e) { refresh(); }
    });
    refresh();
}

```

```

private void refresh() {
    if (editor == null) return;
    String text = editor.getText();
    int lines = text.isEmpty() ? 1 : text.split("\n").length;
    StringBuilder sb = new StringBuilder();
    for (int i = 1; i <= lines; i++) {
        sb.append(i).append("\n");
    }
    setText(sb.toString());
}
}

```

// New methods for W++ specific functionality

```

private void insertWppTemplate() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }
}

```

```

JTextPane editor = editors.get(selectedTab);

```

```

String template = "#include <iostream>\n\n"
    + "int main() {\n"
    + "    // W++ program starts here\n"
    + "    std::cout << \"Hello from W++\" << std::endl;\n"
    + "    \n"
    + "    return 0;\n"
    + "}\n";

```

```

editor.setText(template);
applySyntaxHighlighting(editor);
statusLabel.setText("W++ template inserted");
}

```

```

private void insertMainFunction() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }

```

```

JTextPane editor = editors.get(selectedTab);

```

```

String mainFunction = "int main() {\n"
    + "    // Main function code\n"
    + "    \n"
    + "    return 0;\n"
    + "}\n";

```

```

try {
    editor.getDocument().insertString(editor.getCaretPosition(), mainFunction, null);
    applySyntaxHighlighting(editor);
    statusLabel.setText("Main function inserted");
} catch (BadLocationException e) {
    e.printStackTrace();
    statusLabel.setText("Error inserting main function");
}
}

```

```

private void insertIncludeDirective() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {

```

```

        statusLabel.setText("No tab selected");

        return;
    }

    String[] options = {"<iostream>", "<string>", "<cmath>", "<vector>", "<algorithm>"};
    String selected = (String) JOptionPane.showInputDialog(
        this,
        "Select library to include:",
        "Include Directive",
        JOptionPane.QUESTION_MESSAGE,
        null,
        options,
        options[0]
    );

    if (selected != null) {
        JTextPane editor = editors.get(selectedTab);
        String include = "#include " + selected + "\n";

        try {
            editor.getDocument().insertString(editor.getCaretPosition(), include, null);
            applySyntaxHighlighting(editor);
            statusLabel.setText("Include directive inserted");
        } catch (BadLocationException e) {
            e.printStackTrace();
            statusLabel.setText("Error inserting include directive");
        }
    }
}

private void analyzeWppCode() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {

```

```

        statusLabel.setText("No tab selected");
        return;
    }

    JTextPane editor = editors.get(selectedTab);
    String code = editor.getText();

    java.util.List<Lexer.TokenInfo> tokens = new ArrayList<>();
    java.util.List<Lexer.LexicalError> lexicalErrors = new ArrayList<>();
    Lexer.tokenizeWithErrors(code, tokens, lexicalErrors);

    // Generate statistics about the code
    int lines = code.split("\n", -1).length;
    int chars = code.length();

    // Count different token types
    int keywords = 0;
    int identifiers = 0;
    int operators = 0;
    int literals = 0;
    int constants = 0;
    int separators = 0;

    for (Lexer.TokenInfo ti : tokens) {
        String token = ti.token;
        Lexer.TokenType type = Lexer.getTokenType(token);
        switch (type) {
            case KEYWORD: keywords++; break;
            case IDENTIFIER: identifiers++; break;
            case OPERATOR: operators++; break;
            case LITERAL: literals++; break;
            case CONSTANT: constants++; break;
            case SEPARATOR: separators++; break;
        }
    }

```



```

    }
}

// Create analysis report
StringBuilder report = new StringBuilder();
report.append("W++ CODE ANALYSIS REPORT\n");
report.append("=====\n\n");
report.append("CODE STATISTICS:\n");
report.append("-----\n");
report.append(String.format("Lines of code: %d\n", lines));
report.append(String.format("Total characters: %d\n", chars));
report.append(String.format("Total tokens: %d\n\n", tokens.size()));

report.append("TOKEN DISTRIBUTION:\n");
report.append("-----\n");
report.append(String.format("Keywords: %d (%.1f%%)\n", keywords, 100.0 * keywords / tokens.size()));
report.append(String.format("Identifiers: %d (%.1f%%)\n", identifiers, 100.0 * identifiers / tokens.size()));
report.append(String.format("Operators: %d (%.1f%%)\n", operators, 100.0 * operators / tokens.size()));
report.append(String.format("Literals: %d (%.1f%%)\n", literals, 100.0 * literals / tokens.size()));
report.append(String.format("Constants: %d (%.1f%%)\n", constants, 100.0 * constants / tokens.size()));
report.append(String.format("Separators: %d (%.1f%%)\n", separators, 100.0 * separators / tokens.size()));

if (!lexicalErrors.isEmpty()) {
    report.append("\nLEXICAL ISSUES FOUND: ").append(lexicalErrors.size()).append("\n");
    report.append("-----\n");
    for (Lexer.LexicalError error : lexicalErrors) {
        report.append(error.toString()).append("\n");
    }
}

// Create analysis window
JDialog analysisDialog = new JDialog(this, "W++ Code Analysis", true);
analysisDialog.setSize(600, 500);

```

```

analysisDialog.setLocationRelativeTo(this);

JTextArea analysisArea = new JTextArea(report.toString());
analysisArea.setEditable(false);
analysisArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, fontSize));
analysisArea.setBackground(isDarkMode ? DARK_BG : LIGHT_BG);
analysisArea.setForeground(isDarkMode ? DARK_FG : LIGHT_FG);

JScrollPane scrollPane = new JScrollPane(analysisArea);
analysisDialog.add(scrollPane);

analysisDialog.setVisible(true);
}

private void searchInCode() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }

    JTextPane editor = editors.get(selectedTab);
    String searchText = JOptionPane.showInputDialog(this, "Enter text to search:");

    if (searchText != null && !searchText.isEmpty()) {
        String text = editor.getText();
        int caretPos = editor.getCaretPosition();

        // Search from current position
        int foundPos = text.indexOf(searchText, caretPos);

        // If not found, wrap around to beginning
        if (foundPos == -1) {

```

```

        foundPos = text.indexOf(searchText);
    }

    if (foundPos != -1) {
        editor.setCaretPosition(foundPos);
        editor.select(foundPos, foundPos + searchText.length());
        editor.requestFocus();
        statusLabel.setText("Found " + searchText + " at position " + foundPos);
    } else {
        statusLabel.setText("Text " + searchText + " not found");
    }
}
}

```

```

private void replaceInCode() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }
}

```

```

JTextPane editor = editors.get(selectedTab);

```

```

// Create a dialog with search and replace fields

```

```

JPanel panel = new JPanel(new GridLayout(2, 2, 5, 5));

```

```

JLabel searchLabel = new JLabel("Search for:");

```

```

JLabel replaceLabel = new JLabel("Replace with:");

```

```

JTextField searchField = new JTextField(20);

```

```

JTextField replaceField = new JTextField(20);

```

```

panel.add(searchLabel);

```

```

panel.add(searchField);

```

```

panel.add(replaceLabel);

```

```

panel.add(replaceField);

int result = JOptionPane.showConfirmDialog(this, panel, "Search and Replace",
                                           JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);

if (result == JOptionPane.OK_OPTION) {
    String searchText = searchField.getText();
    String replaceText = replaceField.getText();

    if (searchText != null && !searchText.isEmpty()) {
        String text = editor.getText();
        String newText = text.replace(searchText, replaceText);

        if (!newText.equals(text)) {
            editor.setText(newText);
            applySyntaxHighlighting(editor);
            statusLabel.setText("Replaced all occurrences of '" + searchText + "' with '" + replaceText + "'");
        } else {
            statusLabel.setText("Text '" + searchText + "' not found");
        }
    }
}

private void exportToCSV() {
    int selectedTab = outputPanel.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No output tab selected");
        return;
    }
    // Determine which content to export
    String content = "";
    String fileNameSuffix = "";

```

```

switch (selectedTab) {
    case 0: // Tokens tab
        content = tokenTableArea.getText();
        fileNameSuffix = "tokens";
        break;
    case 1: // Symbol Table tab
        content = symbolTableArea.getText();
        fileNameSuffix = "symbols";
        break;
    case 2: // Errors tab
        content = errorArea.getText();
        fileNameSuffix = "errors";
        break;
    default:
        JScrollPane scrollPane = (JScrollPane)outputPanel.getComponentAt(selectedTab);
        if (scrollPane.getViewPort().getView() instanceof JTextArea) {
            content = ((JTextArea)scrollPane.getViewPort().getView()).getText();
            fileNameSuffix = "output";
        }
}

if (content.isEmpty()) {
    statusLabel.setText("No content to export");
    return;
}

// Convert text to CSV format
String csvContent = convertToCSV(content);

// Save to file
JFileChooser fileChooser = new JFileChooser();
fileChooser.setSelectedFile(new File("wpp_" + fileNameSuffix + ".csv"));
fileChooser.setFileFilter(new javax.swing.filechooser.FileNameExtensionFilter("CSV Files", "csv"));

```

```

if (fileChooser.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {
    String filePath = fileChooser.getSelectedFile().getPath();
    if (!filePath.toLowerCase().endsWith(".csv")) {
        filePath += ".csv";
    }

    try (FileWriter writer = new FileWriter(filePath)) {
        writer.write(csvContent);
        statusLabel.setText("Exported to " + filePath);
    } catch (IOException e) {
        JOptionPane.showMessageDialog(this, "Error exporting to CSV: " + e.getMessage(),
            "Export Error", JOptionPane.ERROR_MESSAGE);
        statusLabel.setText("Error exporting to CSV");
    }
}

private String convertToCSV(String content) {
    StringBuilder csv = new StringBuilder();
    String[] lines = content.split("\n");

    for (String line : lines) {
        // Skip empty lines and separator lines
        if (line.trim().isEmpty() || line.startsWith("--") || line.startsWith("==")) {
            continue;
        }

        // Replace multiple consecutive spaces with a single comma
        String csvLine = line.replaceAll("\\s{2,}", ",");
        csv.append(csvLine).append("\n");
    }
}

```

```

    return csv.toString();
}

private JPanel createTabComponent(String title) {
    JPanel panel = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
    panel.setOpaque(false);

    JLabel label = new JLabel(title);
    label.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 12));

    JButton closeButton = new JButton("×");
    closeButton.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 12));
    closeButton.setFocusPainted(false);
    closeButton.setBorderPainted(false);
    closeButton.setContentAreaFilled(false);
    closeButton.setForeground(new Color(100, 100, 120));
    closeButton.setBorder(BorderFactory.createEmptyBorder(0, 5, 0, 0));
    closeButton.setPreferredSize(new Dimension(20, 20));

    closeButton.addMouseListener(new MouseAdapter() {
        @Override
        public void mouseEntered(MouseEvent e) {
            closeButton.setForeground(new Color(180, 50, 50));
        }

        @Override
        public void mouseExited(MouseEvent e) {
            closeButton.setForeground(new Color(100, 100, 120));
        }
    });

    closeButton.addActionListener(e -> {
        int index = tabbedPane.indexOfTabComponent(panel);

```

```

    if (index != -1) {
        tabbedPane.remove(index);
        editors.remove(index);
        filePaths.remove(index);
    }
});

```

```

panel.add(label);
panel.add(closeButton);
return panel;
}

```

```

private String getTokenDescription(String token, Lexer.TokenType type) {
    switch (type) {
        case KEYWORD:
            if (token.equals("cout")) return "C++ output stream";
            if (token.equals("endl")) return "C++ end of line";
            if (isDataType(token)) return "Data type";
            if (token.equals("if") || token.equals("else")) return "Conditional";
            if (token.equals("for") || token.equals("while") || token.equals("do")) return "Loop control";
            if (token.equals("return")) return "Return statement";
            return "Reserved word";
        case OPERATOR:
            if (token.equals("+") || token.equals("-") || token.equals("*") || token.equals("/") || token.equals("%"))
                return "Arithmetic operator";
            if (token.equals("++") || token.equals("--"))
                return "Increment/decrement";
            if (token.equals("==") || token.equals("!=") || token.equals("<") || token.equals(">") ||
                token.equals("<=") || token.equals(">="))
                return "Comparison operator";
            if (token.equals("&&") || token.equals("||") || token.equals("!"))
                return "Logical operator";
            if (token.equals("=") || token.contains("="))

```



```

        return "Assignment operator";
    return "Operator";
case SEPARATOR:
    if (token.equals("(") || token.equals("))")) return "Parenthesis";
    if (token.equals("{") || token.equals("}")) return "Brace";
    if (token.equals("[") || token.equals("]")) return "Bracket";
    if (token.equals(";")) return "Statement terminator";
    if (token.equals(",")) return "Separator";
    if (token.equals("#")) return "Preprocessor directive";
    return "Separator";
case CONSTANT:
    return "Numeric constant";
case LITERAL:
    if (isStringLiteral(token)) return "String literal";
    if (token.startsWith("\"") && token.endsWith("\"")) return "Character literal";
    if (token.contains(".")) return "Floating-point literal";
    return "Literal";
case IDENTIFIER:
    if (token.equals("main")) return "Main function identifier";
    return "User-defined identifier";
default:
    return "";
}
}

private void simulateOutput() {
    int selectedTab = tabbedPane.getSelectedIndex();
    if (selectedTab == -1) {
        statusLabel.setText("No tab selected");
        return;
    }
    JTextPane editor = editors.get(selectedTab);
    String code = editor.getText();

```

```

StringBuilder output = new StringBuilder();

// Simple simulation: extract cout, printf, and print statements

// C++ cout << "text";

java.util.regex.Pattern    coutPattern    =    java.util.regex.Pattern.compile("cout\\s*<<\\s*\"(.*)\""",
java.util.regex.Pattern.DOTALL);

java.util.regex.Matcher coutMatcher = coutPattern.matcher(code);

while (coutMatcher.find()) {

    output.append(coutMatcher.group(1)).append("\n");

}

// C printf("text");

java.util.regex.Pattern    printfPattern    =    java.util.regex.Pattern.compile("printf\\s*\\(\\s*\"(.*)\""",
java.util.regex.Pattern.DOTALL);

java.util.regex.Matcher printfMatcher = printfPattern.matcher(code);

while (printfMatcher.find()) {

    output.append(printfMatcher.group(1)).append("\n");

}

// Python-like print("text")

java.util.regex.Pattern    printPattern    =    java.util.regex.Pattern.compile("print\\s*\\(\\s*\"(.*)\""",
java.util.regex.Pattern.DOTALL);

java.util.regex.Matcher printMatcher = printPattern.matcher(code);

while (printMatcher.find()) {

    output.append(printMatcher.group(1)).append("\n");

}

// Show output in Console tab

JScrollPane consoleScroll = (JScrollPane) outputPanel.getComponentAt(3);
JTextArea consoleArea = (JTextArea) consoleScroll.getViewPort().getView();

if (output.length() == 0) {

    consoleArea.setText("[No output simulated]");

    statusLabel.setText("No output statements found");

} else {

    consoleArea.setText(output.toString());

    statusLabel.setText("Simulation complete");

}

}

```

```

// Add this method to append messages to the Console tab:
private void logToConsole(String message) {
    JScrollPane consoleScroll = (JScrollPane) outputPanel.getComponentAt(3);
    JTextArea consoleArea = (JTextArea) consoleScroll.getViewport().getView();
    if (consoleArea.getText().equals("[No output simulated]")) {
        consoleArea.setText("");
    }
    consoleArea.append(message + "\n");
    sessionActions.add(message);
}

// Add auto-correction functionality
private void setupAutoCorrection(JTextPane editor) {
    editor.getDocument().addDocumentListener(new DocumentListener() {
        private boolean isCorrecting = false;

        @Override
        public void insertUpdate(DocumentEvent e) {
            if (!isCorrecting) {
                try {
                    String text = editor.getText();
                    int caretPos = editor.getCaretPosition();
                    if (caretPos == 0) return;
                    char lastChar = text.charAt(caretPos - 1);
                    // Define separators that should trigger correction
                    boolean isSeparator = !Character.isLetterOrDigit(lastChar);
                    int wordEnd = caretPos;
                    int wordStart = wordEnd;

                    // Always look back for the previous word
                    while (wordStart > 0 && Character.isLetterOrDigit(text.charAt(wordStart - 1))) {
                        wordStart--;
                    }

                    if (wordStart < wordEnd) {
                        String word = text.substring(wordStart, wordEnd);

```

```

String correction = TYPO_CORRECTIONS.get(word.toLowerCase());
if (correction != null && !correction.equals(word)) {
    isCorrecting = true;
    editor.getDocument().remove(wordStart, word.length());
    editor.getDocument().insertString(wordStart, correction, null);
    // Move caret after the separator if one was typed
    if (isSeparator && caretPos < text.length()) {
        editor.setCaretPosition(wordStart + correction.length() + 1);
    } else {
        editor.setCaretPosition(wordStart + correction.length());
    }
    isCorrecting = false;
    statusLabel.setText("Auto-corrected: " + word + " → " + correction);
    logToConsole("[INFO] Auto-corrected: " + word + " → " + correction);
}
}
} catch (Exception ex) {
    ex.printStackTrace();
}
}
}

```

```

@Override

```

```

public void removeUpdate(DocumentEvent e) {}

```

```

@Override

```

```

public void changedUpdate(DocumentEvent e) {}

```

```

});

```

```

}

```

```

// Auto-complete popup for keywords

```

```

private void setupAutoCompletion(JTextPane editor) {

```

```

    JPopupMenu suggestionPopup = new JPopupMenu();

```

```

JList<String> suggestionList = new JList<>();
suggestionList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
suggestionList.setFont(editor.getFont());
JScrollPane scrollPane = new JScrollPane(suggestionList);
scrollPane.setBorder(null);
suggestionPopup.setBorder(BorderFactory.createLineBorder(new Color(180,180,200)));
suggestionPopup.add(scrollPane);
suggestionPopup.setFocusable(false);
suggestionList.setFocusable(false);
// Helper to show suggestions
Runnable showSuggestions = () -> {
    try {
        int caretPos = editor.getCaretPosition();
        String text = editor.getText(0, caretPos);
        int wordStart = caretPos;
        while (wordStart > 0 && Character.isLetterOrDigit(text.charAt(wordStart - 1))) {
            wordStart--;
        }
        String prefix = text.substring(wordStart, caretPos);
        if (prefix.length() == 0) {
            suggestionPopup.setVisible(false);
            return;
        }
        java.util.List<String> matches = new ArrayList<>();
        for (String kw : KNOWN_WORDS) {
            if (prefix.equals("if")) {
                if (kw.equals("if")) matches.add(kw);
            } else if (prefix.equals("if-")) {
                if (kw.equals("if-else")) matches.add(kw);
            } else if (kw.startsWith(prefix) && !kw.equals(prefix)) {
                // Don't suggest 'main' if it already exists in the code
                if (kw.equals("main") && text.contains("main()")) {
                    continue;
                }
            }
        }
    }
};

```

```

        }
        matches.add(kw);
    }
}

// Sort so 'if' comes before 'if-else' and shorter keywords come first
matches.sort((a, b) -> {
    if (a.equals("if") && b.equals("if-else")) return -1;
    if (a.equals("if-else") && b.equals("if")) return 1;
    return Integer.compare(a.length(), b.length());
});

if (matches.isEmpty()) {
    suggestionPopup.setVisible(false);
    return;
}

suggestionList.setListData(matches.toArray(new String[0]));
suggestionList.setSelectedIndex(0);
Rectangle caretRect = editor.modelToView(caretPos);
Point popupLoc = new Point(caretRect.x, caretRect.y + caretRect.height);
SwingUtilities.convertPointToScreen(popupLoc, editor);
suggestionPopup.setPreferredSize(new Dimension(120, Math.min(120, matches.size() * 22)));
suggestionPopup.pack();
suggestionPopup.setLocation(popupLoc);
suggestionPopup.setVisible(true);
editor.requestFocusInWindow();
} catch (Exception ex) {
    suggestionPopup.setVisible(false);
}
};

// Key listener for auto-complete
editor.addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {

```

```

// Prevent Enter/Tab from propagating if popup is visible
if ((e.getKeyCode() == KeyEvent.VK_TAB || e.getKeyCode() == KeyEvent.VK_ENTER) &&
suggestionPopup.isVisible()) {
    e.consume();
}
}

@Override
public void keyReleased(KeyEvent e) {
    if (e.getKeyCode() == KeyEvent.VK_DOWN && suggestionPopup.isVisible()) {
        int sel = suggestionList.getSelectedIndex();
        if (sel < suggestionList.getModel().getSize() - 1) {
            suggestionList.setSelectedIndex(sel + 1);
            suggestionList.ensureIndexIsVisible(sel + 1);
        }
        e.consume();
    } else if (e.getKeyCode() == KeyEvent.VK_UP && suggestionPopup.isVisible()) {
        int sel = suggestionList.getSelectedIndex();
        if (sel > 0) {
            suggestionList.setSelectedIndex(sel - 1);
            suggestionList.ensureIndexIsVisible(sel - 1);
        }
        e.consume();
    } else if ((e.getKeyCode() == KeyEvent.VK_TAB || e.getKeyCode() == KeyEvent.VK_ENTER) &&
suggestionPopup.isVisible()) {
        int sel = suggestionList.getSelectedIndex();
        if (sel >= 0) {
            String completion = suggestionList.getModel().getElementAt(sel);
            try {
                int caretPos = editor.getCaretPosition();
                String text = editor.getText(0, caretPos);
                int wordStart = caretPos;
                while (wordStart > 0 && Character.isLetterOrDigit(text.charAt(wordStart - 1))) {
                    wordStart--;
                }
            }

```

```

}

// Special handling for main function
if (completion.equals("main")) {
    // Check if main already exists
    if (text.contains("main()")) {
        JOptionPane.showMessageDialog(editor,
            "A main function already exists in the code.\nOnly one main function is allowed.",
            "Duplicate Main Function",
            JOptionPane.WARNING_MESSAGE);
        suggestionPopup.setVisible(false);
        return;
    }

    // Get the current line's text
    int lineStart = wordStart;
    while (lineStart > 0 && text.charAt(lineStart - 1) != '\n') {
        lineStart--;
    }

    String currentLine = text.substring(lineStart, caretPos);

    // Check if we're in a function declaration (after a type)
    String[] words = currentLine.trim().split("\\s+");
    if (words.length > 0 && isDataType(words[0])) {
        // Keep the type and add main with parentheses
        String type = words[0];
        editor.getDocument().remove(wordStart, caretPos - wordStart);
        editor.getDocument().insertString(wordStart, "main() {\n  \n}", null);
        editor.setCaretPosition(wordStart + 7); // Position after "main() {"
    } else {
        // Normal completion for main
        editor.getDocument().remove(wordStart, caretPos - wordStart);
        editor.getDocument().insertString(wordStart, "main() {\n  \n}", null);
    }
}

```



```

        editor.setCaretPosition(wordStart + 7); // Position after "main() {"
    }
} else if (completion.equals("if")) {
    // Auto-complete for if statement
    editor.getDocument().remove(wordStart, caretPos - wordStart);
    String snippet = "if () {\n    \n}";
    editor.getDocument().insertString(wordStart, snippet, null);
    // Place caret inside the parentheses
    editor.setCaretPosition(wordStart + 4);
} else if (completion.equals("if-else")) {
    // Auto-complete for if-else statement
    editor.getDocument().remove(wordStart, caretPos - wordStart);
    String snippet = "if () {\n    \n} else {\n    \n}";
    editor.getDocument().insertString(wordStart, snippet, null);
    // Place caret inside the first parentheses
    editor.setCaretPosition(wordStart + 4);
} else if (completion.equals("else")) {
    // Auto-complete for else statement
    editor.getDocument().remove(wordStart, caretPos - wordStart);
    String snippet = "else {\n    \n}";
    editor.getDocument().insertString(wordStart, snippet, null);
    editor.setCaretPosition(wordStart + 7);
} else if (completion.equals("for")) {
    // Auto-complete for for statement
    editor.getDocument().remove(wordStart, caretPos - wordStart);
    String snippet = "for () {\n    \n}";
    editor.getDocument().insertString(wordStart, snippet, null);
    editor.setCaretPosition(wordStart + 5);
} else if (completion.equals("while")) {
    // Auto-complete for while statement
    editor.getDocument().remove(wordStart, caretPos - wordStart);
    String snippet = "while () {\n    \n}";
    editor.getDocument().insertString(wordStart, snippet, null);

```

```

        editor.setCaretPosition(wordStart + 7);
    } else if (completion.equals("do")) {
        // Auto-complete for do statement
        editor.getDocument().remove(wordStart, caretPos - wordStart);
        String snippet = "do {\n    \n} while ();";

        editor.getDocument().insertString(wordStart, snippet, null);
        editor.setCaretPosition(wordStart + 15);
    } else {
        // Normal completion for other keywords
        editor.getDocument().remove(wordStart, caretPos - wordStart);
        editor.getDocument().insertString(wordStart, completion, null);
        editor.setCaretPosition(wordStart + completion.length());
    }

    suggestionPopup.setVisible(false);
    statusLabel.setText("Auto-completed: " + completion);
    logToConsole("[INFO] Auto-completed: " + completion);
} catch (Exception ex) {
    ex.printStackTrace();
}

e.consume();
} else if (Character.isLetterOrDigit(e.getKeyChar()) || e.getKeyCode() ==
KeyEvent.VK_BACK_SPACE) {
    SwingUtilities.invokeLater(showSuggestions);
} else {
    suggestionPopup.setVisible(false);
}
}

});

// Hide popup if focus lost
editor.addFocusListener(new FocusAdapter() {

```

```

        public void focusLost(FocusEvent e) { suggestionPopup.setVisible(false); }
    });
}

public static void main(String[] args) {
    try {
        // Set system properties for better UI rendering
        System.setProperty("awt.useSystemAAFontSettings", "on");
        System.setProperty("swing.aatext", "true");

        // Set Look and Feel to system look and feel
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());

        // Custom UI tweaks
        UIManager.put("TabbedPane.selected", new Color(0, 120, 215, 80));
        UIManager.put("TabbedPane.contentAreaColor", new Color(0, 120, 215, 80));
        UIManager.put("TabbedPane.highlight", new Color(0, 120, 215, 80));
        UIManager.put("TabbedPane.tabAreaBackground", Color.LIGHT_GRAY);
        UIManager.put("TabbedPane.borderHightlightColor", new Color(0, 120, 215));

        // Start the application
        SwingUtilities.invokeLater(LexerGUI::new);
    } catch (Exception e) {
        e.printStackTrace();
        // Fall back to default if there's an error
        SwingUtilities.invokeLater(LexerGUI::new);
    }
}
}

```